A Cookbook of SPL Recipes

Declarative Approaches to Agentic Workflow Orchestration

Wen Gong

2026

Contents

1 Foundations **1**

1.1 Preface: The Conductor's Score **2**

1.1.1 Why This Book Exists . . . 2
1.1.2 This Book Is Written for Two Audiences . . . 2
1.1.2.1 Audience One: The SQL Professional . . . 2
1.1.2.2 Audience Two: The Global South . . . 3
1.1.3 The Bigger Vision: SPL + Momagrid . . . 3
1.1.4 The Conductor Metaphor . . . 4
1.1.5 A Note for Engineering Leaders . . . 4
1.1.6 What This Book Is Not . . . 5
1.1.7 How This Book Works . . . 5
1.1.8 A Note on Reproducibility . . . 6
1.1.9 A Note on the Collaborative Process . . . 6
1.1.10 Acknowledgments . . . 6

1.2 Why Declarative? SQL as the Mental Model **8**

1.2.1 Who This Chapter Is For . . . 8
1.2.2 The Problem You Already Solved . . . 8
1.2.3 What Imperative LLM Code Looks Like . . . 9
1.2.4 The SQL Analogy . . . 11
1.2.5 The Mapping . . . 11
1.2.6 CALL vs. GENERATE: The Most Important Distinction . . . 13
1.2.7 The Adapter Abstraction . . . 14
1.2.8 Side by Side . . . 15
1.2.9 Why This Matters for SQL Practitioners Specifically . . . 17
1.2.10 What Comes Next . . . 18

1.3 Installing spl-llm, Running Your First Workflow **19**

1.3.1 The Path to Hello World . . . 19
1.3.2 Step 1: Install the Runtime . . . 19
1.3.3 Step 2: Set Up Your Orchestra (Ollama) . . . 20
1.3.4 Step 3: Run Your First Workflow . . . 20
1.3.5 The Adapter Configuration . . . 20
1.3.6 Troubleshooting . . . 21
1.3.7 What Just Happened? . . . 21

2 Basics **22**

2.1 Hello World **23**
2.1.1 The Pattern . . . 23
2.1.2 The SPL Approach . . . 23
2.1.3 The .spl File (Annotated) . . . 24
2.1.3.1 (1) `PROMPT hello_world` . . . 24
2.1.3.2 (2) `SELECT system_role(...)` . . . 24
2.1.3.3 (3) `GENERATE greeting()` . . . 25
2.1.4 Running It . . . 25
2.1.5 What Just Happened . . . 26
2.1.6 Reproducibility Note . . . 26
2.1.7 When to Use This Pattern . . . 27
2.1.8 Exercises . . . 27

2.2 Ollama Proxy **28**
2.2.1 The Pattern . . . 28
2.2.2 The Open-Source Engine Room: Ollama and Llama.cpp . . . 28
2.2.3 The SPL Approach . . . 29
2.2.4 The .spl File (Annotated) . . . 29
2.2.4.1 (1) `PROMPT ollama_proxy` . . . 29
2.2.4.2 (2) `system_role(...)` . . . 30
2.2.4.3 (3) `context.prompt AS prompt` . . . 30
2.2.4.4 (4) `GENERATE answer(prompt)` . . . 30
2.2.5 Running It . . . 30
2.2.6 What Just Happened . . . 31
2.2.7 Reproducibility Note . . . 31
2.2.8 When to Use This Pattern . . . 31
2.2.9 Exercises . . . 31

2.3 Multilingual Greeting **33**
2.3.1 The Pattern . . . 33
2.3.2 The SPL Approach . . . 33
2.3.3 The .spl File (Annotated) . . . 34
2.3.3.1 (1) `PROMPT multilingual_greeting` . . . 34
2.3.3.2 (2) `system_role(...)` . . . 34
2.3.3.3 (3) & (4) `context.user_input AS input` and `context.lang AS lang` 34
2.3.3.4 (5) `GENERATE greeting(input, lang)` . . . 34
2.3.4 Running It . . . 34
2.3.5 What Just Happened . . . 35
2.3.6 Reproducibility Note . . . 35
2.3.7 When to Use This Pattern . . . 35
2.3.8 Exercises . . . 36

2.4 Few-Shot Learning **37**
2.4.1 The Pattern . . . 37
2.4.2 The SPL Approach . . . 37
2.4.3 The .spl File (Annotated) . . . 37

2.4.3.1 (1) Dynamic Examples via `SELECT CASE` . . . 38
2.4.3.2 (2) Dynamic Context Injection . . . 38
2.4.3.3 (3) Calibrated Generation . . . 38
2.4.4 Running It . . . 39
2.4.5 What Just Happened . . . 39
2.4.6 Reproducibility Note . . . 39
2.4.7 When to Use This Pattern . . . 40
2.4.8 Exercises . . . 40

3 Agentic Patterns **41**
3.0.1 Self-Refine . . . 42
3.0.1.1 The Pattern . . . 42
3.0.1.2 The SPL Approach . . . 42
3.0.1.3 The .spl File (Annotated) . . . 42
3.0.1.4 The SQL Analogy in Depth . . . 44
3.0.1.5 Running It . . . 44
3.0.1.6 What Just Happened . . . 45
3.0.1.7 Reproducibility Note . . . 45
3.0.1.8 When to Use This Pattern . . . 46
3.0.1.9 Exercises . . . 46

3.1 ReAct Agent **47**
3.1.1 The Pattern . . . 47
3.1.2 The SPL Approach . . . 47
3.1.3 The .spl File (Annotated) . . . 47
3.1.3.1 (1) `PROCEDURE name(...)` . . . 48
3.1.3.2 (2) Arithmetic in SPL . . . 48
3.1.3.3 (3) `CALL procedure_name(...)` . . . 49
3.1.3.4 (4) `CALL calc_growth_rate(...)` . . . 49
3.1.3.5 (5) `GENERATE growth_report(...)` . . . 49
3.1.4 Running It . . . 49
3.1.5 What Just Happened . . . 49
3.1.6 Reproducibility Note . . . 50
3.1.7 When to Use This Pattern . . . 50
3.1.8 Exercises . . . 50

3.2 Plan and Execute **51**
3.2.1 The Pattern . . . 51
3.2.2 The SPL Approach . . . 51
3.2.3 The .spl File (Annotated) . . . 52
3.2.3.1 (1) Phase 1: Planning (The Blueprint) . . . 52
3.2.3.2 (2) Phase 2: Design Loop (Lightweight Reasoning) . . . 52
3.2.3.3 (3) Automatic Re-planning . . . 53
3.2.3.4 (4) Phase 3: Implementation Loop (Heavy Lifting) . . . 53
3.2.3.5 (5) Deterministic I/O (`CALL write_code_files`) . . . 53
3.2.4 Running It . . . 53
3.2.5 What Just Happened . . . 53

3.2.6 Reproducibility Note . . . 53
3.2.7 When to Use This Pattern . . . 54
3.2.8 Exercises . . . 54

3.3 Reflection Agent 55
3.3.1 The Pattern . . . 55
3.3.2 The SPL Approach . . . 55
3.3.3 The .spl File (Annotated) . . . 55
3.3.3.1 (1) The Initial Solve . . . 56
3.3.3.2 (2) The "Mirror" Step (`reflect`) . . . 56
3.3.3.3 (3) Quantifying Doubt (`confidence_score`) . . . 57
3.3.3.4 (4) The Confidence Threshold . . . 57
3.3.4 Running It . . . 57
3.3.5 What Just Happened . . . 57
3.3.6 Reproducibility Note . . . 57
3.3.7 When to Use This Pattern . . . 58
3.3.8 Exercises . . . 58

3.4 Multi-Model Pipeline 59
3.4.1 The Pattern . . . 59
3.4.2 The SPL Approach . . . 59
3.4.3 The .spl File (Annotated) . . . 59
3.4.3.1 (1) Step 1: Fast Retrieval . . . 60
3.4.3.2 (2) Step 2: Deep Reasoning . . . 60
3.4.3.3 (3) Step 3: Creative Writing . . . 60
3.4.3.4 (4) Step 4: The Neutral Judge . . . 61
3.4.4 Running It . . . 61
3.4.5 What Just Happened . . . 61
3.4.6 Reproducibility Note . . . 61
3.4.7 When to Use This Pattern . . . 62
3.4.8 Exercises . . . 62

3.5 Nested Procedures 63
3.5.1 The Pattern . . . 63
3.5.2 The SPL Approach . . . 63
3.5.3 The .spl File (Annotated) . . . 63
3.5.3.1 (1) The "Leaf" Procedure . . . 64
3.5.3.2 (2) The "Intermediate" Procedure . . . 64
3.5.3.3 (3) The Root Workflow . . . 65
3.5.3.4 (4) Delegation (`CALL`) . . . 65
3.5.4 Running It . . . 65
3.5.5 What Just Happened . . . 65
3.5.6 Reproducibility Note . . . 65
3.5.7 When to Use This Pattern . . . 66
3.5.8 Exercises . . . 66

4 Reasoning **67**
4.0.1 Chain of Thought . . . 68
4.0.1.1 The Pattern . . . 68
4.0.1.2 The SPL Approach . . . 68
4.0.1.3 The .spl File (Annotated) . . . 68
4.0.1.4 Running It . . . 70
4.0.1.5 What Just Happened . . . 70
4.0.1.6 Reproducibility Note . . . 71
4.0.1.7 When to Use This Pattern . . . 71
4.0.1.8 Exercises . . . 72

4.1 Tree of Thought **73**
4.1.1 The Pattern . . . 73
4.1.2 The SPL Approach . . . 73
4.1.3 The .spl File (Annotated) . . . 73
4.1.3.1 (1) Phase 1: Branching . . . 74
4.1.3.2 (2) Phase 2: Developing . . . 74
4.1.3.3 (3) Phase 3: Scoring . . . 75
4.1.3.4 (4) The Final Check . . . 75
4.1.4 Running It . . . 75
4.1.5 What Just Happened . . . 75
4.1.6 Reproducibility Note . . . 75
4.1.7 When to Use This Pattern . . . 76
4.1.8 Exercises . . . 76

4.2 Hypothesis Tester **77**
4.2.1 The Pattern . . . 77
4.2.2 The SPL Approach . . . 77
4.2.3 The .spl File (Annotated) . . . 77
4.2.3.1 (1), (2) & (3) The Confidence Ladder . . . 78
4.2.4 Running It . . . 79
4.2.5 What Just Happened . . . 79
4.2.6 Reproducibility Note . . . 79
4.2.7 When to Use This Pattern . . . 79
4.2.8 Exercises . . . 80

5 Safety & Reliability **81**

5.1 Safe Generation **82**
5.1.1 The Pattern . . . 82
5.1.2 The SPL Approach . . . 82
5.1.3 The .spl File (Annotated) . . . 82
5.1.3.1 (1) & (2) Generation and Assessment . . . 83
5.1.3.2 (3) `EVALUATE` (Semantic Branching) . . . 83
5.1.3.3 (4) Refinement . . . 84
5.1.3.4 (5) `EXCEPTION` . . . 84
5.1.3.5 (6) `HallucinationDetected` . . . 84

5.1.3.6 (7) `RETRY WITH ...` . . . 84
5.1.4 Running It . . . 84
5.1.5 What Just Happened . . . 84
5.1.6 Reproducibility Note . . . 85
5.1.7 When to Use This Pattern . . . 85
5.1.8 Exercises . . . 85

5.2 Guardrails Pipeline 86
5.2.1 The Pattern . . . 86
5.2.2 The SPL Approach . . . 86
5.2.3 The .spl File (Annotated) . . . 86
5.2.3.1 (1) Gate 1a: The Firewall (`CALL`) . . . 87
5.2.3.2 (2) Gate 1b: The Nuanced Check (`GENERATE`) . . . 88
5.2.3.3 (3) Gate 2: Data Masking (`CALL`) . . . 88
5.2.3.4 (4) Gate 4: The Final Audit (`GENERATE`) . . . 88
5.2.4 Running It . . . 88
5.2.5 What Just Happened . . . 88
5.2.6 Reproducibility Note . . . 89
5.2.7 When to Use This Pattern . . . 89
5.2.8 Exercises . . . 89

6 Memory & Retrieval 90

6.1 RAG Query 91
6.1.1 The Pattern . . . 91
6.1.2 The SPL Approach . . . 92
6.1.3 The .spl File (Annotated) . . . 92
6.1.3.1 (1) `PROMPT rag_answer` . . . 92
6.1.3.2 (2) `system_role(...)` . . . 92
6.1.3.3 (3) `rag.query(context.question, top_k=3) AS background` . . . 92
6.1.3.4 (4) & (5) `question` and `GENERATE` . . . 92
6.1.4 Running It . . . 93
6.1.5 What Just Happened . . . 93
6.1.6 Reproducibility Note . . . 93
6.1.7 When to Use This Pattern . . . 93
6.1.8 Exercises . . . 94

6.2 Memory Conversation 95
6.2.1 The Pattern . . . 95
6.2.2 The SPL Approach . . . 95
6.2.3 The .spl File (Annotated) . . . 95
6.2.3.1 (1) Phase 1: Retrieval (`memory.get`) . . . 96
6.2.3.2 (2) Phase 2: Fact Extraction . . . 96
6.2.3.3 (3) Updating the State (`merge_profile`) . . . 96
6.2.4 Running It . . . 97
6.2.5 What Just Happened . . . 97
6.2.6 Reproducibility Note . . . 97

6.2.7 When to Use This Pattern . . . 97
6.2.8 Exercises . . . 97

7 Multi-Agent Systems 99

7.1 Debate Arena 100
7.1.1 The Pattern . . . 100
7.1.2 The SPL Approach . . . 100
7.1.3 The .spl File (Annotated) . . . 100
7.1.3.1 (1) & (2) State Management . . . 101
7.1.3.2 (3) `WHILE` (Iterative Reasoning) . . . 101
7.1.3.3 (4) The Final Judgment . . . 102
7.1.4 Running It . . . 102
7.1.5 What Just Happened . . . 102
7.1.6 Reproducibility Note . . . 102
7.1.7 When to Use This Pattern . . . 102
7.1.8 Exercises . . . 103

7.2 Multi-Agent Collaboration 104
7.2.1 The Pattern . . . 104
7.2.2 The SPL Approach . . . 104
7.2.3 The .spl File (Annotated) . . . 104
7.2.3.1 (1), (2) & (3) Procedures as Agents . . . 105
7.2.3.2 (4) Linear Delegation . . . 105
7.2.4 Running It . . . 106
7.2.5 What Just Happened . . . 106
7.2.6 Reproducibility Note . . . 106
7.2.7 When to Use This Pattern . . . 106
7.2.8 Exercises . . . 106

7.3 Ensemble Voting 108
7.3.1 The Pattern . . . 108
7.3.2 The SPL Approach . . . 108
7.3.3 The .spl File (Annotated) . . . 108
7.3.3.1 (1) Phase 1: Multi-Sampling . . . 109
7.3.3.2 (2) Phase 2: Scoring . . . 109
7.3.3.3 (3) Phase 3: Finding Consensus . . . 109
7.3.3.4 (4) Phase 4: Polishing . . . 109
7.3.4 Running It . . . 110
7.3.5 What Just Happened . . . 110
7.3.6 Reproducibility Note . . . 110
7.3.7 When to Use This Pattern . . . 110
7.3.8 Exercises . . . 110

7.4 Socratic Tutor 112
7.4.1 The Pattern . . . 112
7.4.2 The SPL Approach . . . 112

7.4.3 The .spl File (Annotated) . . . 112
7.4.3.1 (1) The "Dummy Student" (`simulate_student`) . . . 113
7.4.3.2 (2) Measuring Learning (`assess_understanding`) . . . 114
7.4.3.3 (3) & (4) Adaptive Branching . . . 114
7.4.4 Running It . . . 114
7.4.5 What Just Happened . . . 114
7.4.6 Reproducibility Note . . . 114
7.4.7 When to Use This Pattern . . . 115
7.4.8 Exercises . . . 115

7.5 Interview Sim 116
7.5.1 The Pattern . . . 116
7.5.2 The SPL Approach . . . 116
7.5.3 The .spl File (Annotated) . . . 117
7.5.3.1 (1) Upfront Planning (`generate_questions`) . . . 117
7.5.3.2 (2) Role-play Generation . . . 117
7.5.3.3 (3) Applying the Rubric (`evaluation_rubric`) . . . 117
7.5.3.4 (4) Python Math Tool (`aggregate_scores`) . . . 118
7.5.4 Running It . . . 118
7.5.5 What Just Happened . . . 118
7.5.6 Reproducibility Note . . . 118
7.5.7 When to Use This Pattern . . . 118
7.5.8 Exercises . . . 119

8 Applications 120

8.1 Map-Reduce Summarizer 121
8.1.1 The Pattern . . . 121
8.1.2 The SPL Approach . . . 121
8.1.3 The .spl File (Annotated) . . . 122
8.1.3.1 (1) Phase 1: The Map Strategy . . . 122
8.1.3.2 (2) Phase 2: The Map Step . . . 123
8.1.3.3 (3) Phase 3: The Reduce Step . . . 123
8.1.3.4 (4) Phase 4: Conditional Refinement . . . 123
8.1.4 Running It . . . 123
8.1.5 What Just Happened . . . 123
8.1.6 Reproducibility Note . . . 123
8.1.7 When to Use This Pattern . . . 124
8.1.8 Exercises . . . 124

8.2 Code Review 125
8.2.1 The Pattern . . . 125
8.2.2 The SPL Approach . . . 125
8.2.3 The .spl File (Annotated) . . . 126
8.2.3.1 (1) Specialized Passes . . . 126
8.2.3.2 (2) Quantifying Risk (`severity_score`) . . . 126
8.2.3.3 (3) Threshold Branching . . . 127

8.2.4 Running It . . . 127
8.2.5 What Just Happened . . . 127
8.2.6 Reproducibility Note . . . 127
8.2.7 When to Use This Pattern . . . 127
8.2.8 Exercises . . . 128

8.3 Text2SPL Demo 129
8.3.1 The Pattern . . . 129
8.3.2 The SPL Approach . . . 129
8.3.3 The Compilation Process (Annotated) . . . 129
8.3.4 Running It . . . 130
8.3.5 What Just Happened . . . 130
8.3.6 Reproducibility Note . . . 130
8.3.7 When to Use This Pattern . . . 131
8.3.8 Exercises . . . 131

8.4 Structured Output 132
8.4.1 The Pattern . . . 132
8.4.2 The SPL Approach . . . 132
8.4.3 The .spl File (Annotated) . . . 133
8.4.3.1 (1) Reusable Schema Definition . . . 133
8.4.3.2 (2) Enforcing Numeric Types . . . 133
8.4.3.3 (3) Schema Context Injection . . . 133
8.4.3.4 (4) Constrained Generation . . . 134
8.4.4 Running It . . . 134
8.4.5 What Just Happened . . . 134
8.4.6 Reproducibility Note . . . 134
8.4.7 When to Use This Pattern . . . 135
8.4.8 Exercises . . . 135

8.5 Data Extraction 136
8.5.1 The Pattern . . . 136
8.5.2 The SPL Approach . . . 136
8.5.3 The .spl File (Annotated) . . . 136
8.5.3.1 (1) & (2) The Blueprint Library . . . 137
8.5.3.2 (3) Injecting the right blueprint . . . 137
8.5.3.3 (4) Precision Extraction . . . 138
8.5.4 Running It . . . 138
8.5.5 What Just Happened . . . 138
8.5.6 Reproducibility Note . . . 138
8.5.7 When to Use This Pattern . . . 139
8.5.8 Exercises . . . 139

8.6 Support Triage 140
8.6.1 The Pattern . . . 140
8.6.2 The SPL Approach . . . 140
8.6.3 The .spl File (Annotated) . . . 141
8.6.3.1 (1) & (2) Fact Finding (`CALL`) . . . 141

8.6.3.2 (3) Grounded Classification . . . 142
8.6.3.3 (4) Critical Escalation . . . 142
8.6.3.4 (5) Citing the Ground Truth . . . 142
8.6.4 Running It . . . 142
8.6.5 What Just Happened . . . 142
8.6.6 Reproducibility Note . . . 142
8.6.7 When to Use This Pattern . . . 143
8.6.8 Exercises . . . 143

8.7 Meeting Actions 144
8.7.1 The Pattern . . . 144
8.7.2 The SPL Approach . . . 144
8.7.3 The .spl File (Annotated) . . . 145
8.7.3.1 (1) Finding the Actors (`extract_speakers`) . . . 145
8.7.3.2 (2) Noise Reduction (`normalize_transcript`) . . . 145
8.7.3.3 (3) The Schema Constraint (`action_item_schema`) . . . 145
8.7.3.4 (4) Date Fixing (`normalize_dates`) . . . 146
8.7.4 Running It . . . 146
8.7.5 What Just Happened . . . 146
8.7.6 Reproducibility Note . . . 146
8.7.7 When to Use This Pattern . . . 147
8.7.8 Exercises . . . 147

8.8 Code Gen + Tests 148
8.8.1 The Pattern . . . 148
8.8.2 The SPL Approach . . . 148
8.8.3 The .spl File (Annotated) . . . 149
8.8.3.1 (1) Injecting Style Guides (`language_conventions`) . . . 149
8.8.3.2 (2) Passing Code as Context . . . 149
8.8.3.3 (3) Choosing the Runner (`test_framework_guide`) . . . 150
8.8.4 Running It . . . 150
8.8.5 What Just Happened . . . 150
8.8.6 Reproducibility Note . . . 150
8.8.7 When to Use This Pattern . . . 150
8.8.8 Exercises . . . 151

8.9 Sentiment Pipeline 152
8.9.1 The Pattern . . . 152
8.9.2 The SPL Approach . . . 152
8.9.3 The .spl File (Annotated) . . . 153
8.9.3.1 (1) Python Splitting Tool (`split_items`) . . . 153
8.9.3.2 (2) The Extraction Schema (`sentiment_schema`) . . . 153
8.9.3.3 (3) Python Math Tool (`compute_stats`) . . . 153
8.9.4 Running It . . . 154
8.9.5 What Just Happened . . . 154
8.9.6 Reproducibility Note . . . 154
8.9.7 When to Use This Pattern . . . 154
8.9.8 Exercises . . . 155

8.10 Progressive Summary **156**
8.10.1 The Pattern . . . 156
8.10.2 The SPL Approach . . . 156
8.10.3 The .spl File (Annotated) . . . 157
8.10.3.1 (1) Layer 1: The Core . . . 157
8.10.3.2 (2) Layer 2: The Brief (`expand`) . . . 157
8.10.3.3 (3) Layer 3: The Report . . . 157
8.10.4 Running It . . . 158
8.10.5 What Just Happened . . . 158
8.10.6 Reproducibility Note . . . 158
8.10.7 When to Use This Pattern . . . 158
8.10.8 Exercises . . . 159

9 Benchmarking & Evaluation **160**

9.1 Model Showdown **161**
9.1.1 The Pattern . . . 161
9.1.2 The SPL Approach . . . 161
9.1.3 The .spl File (Annotated) . . . 161
9.1.3.1 (1) `CREATE FUNCTION ... AS $$ ... $$` . . . 163
9.1.3.2 (2) & (3) `WORKFLOW` and `INPUT` . . . 163
9.1.3.3 (4) `WITH ... AS (...)` (Parallel CTEs) . . . 163
9.1.3.4 (5) `USING MODEL @model_1` . . . 163
9.1.3.5 (6) `SELECT ... INTO ...` . . . 163
9.1.3.6 (7) The "Judge" Step . . . 163
9.1.4 Running It . . . 163
9.1.5 What Just Happened . . . 164
9.1.6 Reproducibility Note . . . 164
9.1.7 When to Use This Pattern . . . 164
9.1.8 Exercises . . . 164

9.2 Batch Test **166**
9.2.1 The Pattern . . . 166
9.2.2 The SPL Approach . . . 166
9.2.3 The .spl File (Annotated) . . . 166
9.2.3.1 (1) Extreme Fan-Out . . . 167
9.2.3.2 (2) Synchronization Point . . . 168
9.2.3.3 (3) The Reporter . . . 168
9.2.3.4 (4) Final Commit . . . 168
9.2.4 Running It . . . 168
9.2.5 What Just Happened . . . 168
9.2.6 Reproducibility Note . . . 169
9.2.7 When to Use This Pattern . . . 169
9.2.8 Exercises . . . 169

9.3 A/B Test **170**
9.3.1 The Pattern . . . 170

9.3.2 The SPL Approach . . . 170
9.3.3 The .spl File (Annotated) . . . 171
9.3.3.1 (1) & (2) The Tournament . . . 171
9.3.3.2 (3) Prose to Numbers (`extract_total`) . . . 171
9.3.3.3 (4) The Margin Check . . . 172
9.3.4 Running It . . . 172
9.3.5 What Just Happened . . . 172
9.3.6 Reproducibility Note . . . 172
9.3.7 When to Use This Pattern . . . 172
9.3.8 Exercises . . . 173

10 The Road Ahead 174

10.1 The Recipe Maker: SPL Eating Its Own Cake 175
10.1.1 The Concept: What 37 Recipes Taught Us . . . 175
10.1.2 Cooking as Science and Art . . . 176
10.1.3 The Full Lifecycle . . . 176
10.1.4 The CLI Contract . . . 177
10.1.5 Recipe 00: Recipe Maker . . . 178
10.1.5.1 The Pattern . . . 178
10.1.5.2 The SPL Approach . . . 178
10.1.5.3 The .spl File (Annotated) . . . 178
10.1.5.4 Running It . . . 180
10.1.5.5 What Just Happened . . . 181
10.1.5.6 Reproducibility Note . . . 182
10.1.5.7 When to Use This Pattern . . . 182
10.1.5.8 Exercises . . . 183
10.1.6 A Note on What This Means . . . 183

10.2 About the Author 184
10.2.1 From Anhui to CERN: A Journey of Access . . . 184
10.2.2 The Physicist's Instinct: Declarative Thinking . . . 184
10.2.3 The "Gang of Four" and the 48-Hour Sprint . . . 185
10.2.4 The Vision: AI as a Public Utility . . . 185

Part 1

Foundations

Chapter 1.1

Preface: The Conductor's Score

"Every human can conduct their own symphony — the models are the orchestra, SPL is the score."

1.1.1 Why This Book Exists

In early 2025, I watched a colleague — a skilled data engineer with a decade of SQL experience — struggle for three weeks to build a simple multi-step AI workflow. She had read the LangChain documentation, attended an AutoGen workshop, and completed an online course on AI agents. Yet there she was, drowning in callback functions, state machine definitions, and framework-specific abstractions that multiplied with every tutorial she followed.

"I just want to chain three LLM calls together," she told me over coffee. "Why does this require a hundred lines of Python?"

That question haunted me. Because the answer — *because that's how the frameworks are designed* — wasn't good enough. Not when millions of people already know a declarative language powerful enough to express complex logic. Not when SQL has been solving orchestration problems for fifty years. Not when the mental model for joining tables, filtering results, and aggregating data maps so naturally to chaining prompts, routing outputs, and orchestrating agents.

This book is my answer to her question. And to every SQL practitioner, data analyst, and domain expert who has looked at the agentic AI landscape and thought: *there has to be a simpler way.*

There is.

1.1.2 This Book Is Written for Two Audiences

I want to be direct about who this book is for, because each group deserves to hear it said plainly.

1.1.2.1 Audience One: The SQL Professional

You have written SQL for years — maybe decades. You know what it feels like to describe *what* you want and let the engine figure out *how* to produce it. You know stored procedures, cursors,

exception handlers. You may have written PL/SQL, T-SQL, or PostgreSQL procedural extensions. You understand declarative thinking in your hands, not just in your head.

You have been largely left out of the agentic AI conversation. The frameworks assume Python. The tutorials assume you want to write classes, manage state machines, and wire callback functions. You don't. You want to describe a workflow the same way you describe a query — clearly, concisely, and portably — and let the runtime handle the mechanics.

SPL is SQL for LLM workflows. The transfer is not a metaphor. SELECT assembles context going into the model. GENERATE invokes it and captures the output. WORKFLOW is a stored procedure. EVALUATE is a semantic WHERE clause. EXCEPTION is BEGIN...EXCEPTION...END. If you have written PL/SQL, you will recognize the pattern within the first recipe. The domain is new. The discipline is not.

1.1.2.2 Audience Two: The Global South

This book is dedicated to the engineers, researchers, students, and builders of the Global South — who have every right to participate in the AI revolution, and who face barriers their counterparts in San Francisco, London, or Shanghai never encounter.

Access to frontier AI today requires one of three things: expensive cloud API credits, enterprise GPU clusters, or a university affiliation that provides both. None of these are accessible to a student in Lagos, a researcher in Hefei, or a developer in São Paulo building tools for their community.

This book rejects that premise.

Every recipe in this book was benchmarked on a **GTX 1080 Ti** — a graphics card you can find on eBay or in a used gaming PC for under $200. Not because we couldn't access better equipment. Because we chose not to. Because accessibility is not an afterthought in this project — it is a design principle baked into every benchmark, every recipe, every line of SPL.

If you are reading this from a university in Jakarta, a startup in Nairobi, a government office in São Paulo, or a home in any of the hundred countries where GPU clouds are expensive, slow, or unavailable: **this book was written for you.** The code runs on your hardware. The patterns work within your constraints. The knowledge is yours to keep, share, and build upon.

1.1.3 The Bigger Vision: SPL + Momagrid

This book teaches SPL — the language. But the full vision is larger, and you deserve to understand it.

SPL solves the *programming* problem: it gives you a declarative language to express agentic workflows without Python, without framework lock-in, without infrastructure decisions baked into the source code. The same `.spl` file runs on any model, any provider, any backend — by changing a CLI flag.

Momagrid solves the *infrastructure* problem: it is a decentralized inference network where GPU nodes contributed by independent operators — students, hobbyists, universities, cooperatives — form a shared compute grid. Anyone with a GPU can contribute. Anyone with a workflow can run it.

Together, SPL and Momagrid form a full stack for democratic AI:

```
Momagrid — Decentralized Inference Network
(GPU nodes contributed by anyone, anywhere)

SPL 2.0 — Declarative Agentic Workflow Orchestration
(WORKFLOW, EVALUATE, WHILE, EXCEPTION, PROCEDURE)

SPL 1.0 — Declarative Context Management
(PROMPT, SELECT, GENERATE)
```

A student in Anhui with a used GPU can contribute inference capacity to Momagrid and earn compute credits. A researcher in Lagos with an SPL workflow can run it against the grid — no cloud account, no API key, no enterprise contract. The same language that runs on your local Ollama instance runs on the decentralized grid. You change one flag.

This is not a distant roadmap. The SPL runtime is published and tested. The Momagrid adapter is live. The vision is operational today on modest hardware, and it will scale as the network grows.

The AI revolution should not belong only to those who can afford H100 clusters. SPL and Momagrid are our answer to that problem.

1.1.4 The Conductor Metaphor

Throughout this book, a single metaphor organizes the ideas:

- **The Human is the Conductor**: You provide the vision, the intent, and the final judgment. You decide what the workflow should accomplish. You set the quality bar.
- **The LLMs are the Orchestra**: Gemma, Claude, Llama, Mistral, GPT, Qwen — a diverse ensemble of models, each with different strengths, costs, and latency profiles. You can swap any instrument without rewriting the score.
- **SPL is the Score**: A declarative specification that describes *what* the performance should be, not *how* each musician should move their fingers. The same score performs differently with a string quartet versus a full symphony — but it is the same score.

When you change the orchestra (e.g., from −−adapter ollama to −−adapter openrouter), the score — your .spl file — remains exactly the same. This is the promise of adapter portability.

1.1.5 A Note for Engineering Leaders

If you are a CTO, engineering lead, or technical decision-maker evaluating AI orchestration tools for your team, this section is for you.

The core question is not "which framework?" It is: **who on your team can build and maintain these workflows?**

Imperative frameworks (LangGraph, AutoGen, CrewAI) require Python programming, framework-specific abstractions, and careful state management. Your senior engineers can build with them. Your data analysts, BI developers, domain experts, and SQL practitioners cannot — not without significant retraining.

SPL changes the denominator. If your team includes SQL practitioners — and most data-adjacent teams do — they can write, read, and review SPL workflows without Python training. More importantly, **business stakeholders can review an SPL script and approve it** without executing it, because the intent is explicit in the syntax. This is a governance model that imperative code cannot provide, regardless of documentation quality.

The operational model also simplifies: the same .spl file promotes unchanged through Dev, Test, and Production by swapping CLI flags (−−adapter, −−model, −−tools, −−datasets). No environment-specific code paths. No re-review across environments.

The LOC reduction (5–7× versus imperative frameworks for equivalent patterns) is a secondary benefit. The primary benefit is that more of your team can participate — and that the artifacts they produce are human-reviewable by the people who understand the business logic best.

1.1.6 What This Book Is Not

This is not a book about machine learning theory. Transformer architectures, attention mechanisms, and backpropagation are not covered here.

This is not a book about Python. SPL workflows are expressed in .spl files — not Python files. If you want the Python API, the spl−llm package documentation covers it. This book focuses on the declarative artifact.

This is not a book that claims SPL is always the right tool. LangGraph, AutoGen, and CrewAI have their place. If you are building production systems with complex dynamic state that changes at the millisecond level, those frameworks may serve you better. This book is for the workflows — the large majority — where the logic is expressible declaratively, and where that expressibility matters for maintainability, reviewability, and reach.

And this is not a book with toy examples. Every recipe runs on real hardware, produces real output, and includes latency benchmarks, variability notes, and honest "when not to use this" guidance.

1.1.7 How This Book Works

Every chapter follows the same structure. You can open to any recipe and start there, or read sequentially. Each recipe includes:

- **The Pattern** — What problem does this solve? Why would you need it?
- **The SPL Approach** — How does SPL make this natural to express?
- **The .spl File (Annotated)** — Full source with line-by-line explanations
- **The SQL Analogy** — The structural parallel for SQL practitioners
- **Running It** — Exact commands and expected output

- **What Just Happened** — An execution trace showing what the runtime did
- **Reproducibility Note** — Latency, variability, and hardware details
- **When to Use This Pattern** — Concrete use cases, anti-patterns, and alternatives
- **Exercises** — Modifications to try on your own hardware

The SQL Analogy section is present in every recipe because it is the fastest path for data practitioners to build a correct mental model. If you are not a SQL practitioner, you can skip it — the recipe stands alone. If you are, it is the section that makes the language click.

1.1.8 A Note on Reproducibility

Every recipe was tested multiple times on the same hardware to verify consistency. The reference environment:

- **GPU**: NVIDIA GTX 1080 Ti (11 GB VRAM) — available on eBay for under $200
- **CPU**: AMD Ryzen 7 3700X
- **RAM**: 32 GB DDR4
- **OS**: Ubuntu 22.04 LTS
- **Primary Model**: Gemma 3 via Ollama

Why Gemma 3 on a 1080 Ti? Because it fits. Because it is open-weights. Because it represents what is achievable on commodity hardware today — not what requires enterprise infrastructure. If you have better hardware, recipes will run faster. If you have worse hardware, smaller models (phi4, Llama 3.2 3B) will work for most patterns.

The logic is what matters. The hardware is a floor, not a ceiling.

1.1.9 A Note on the Collaborative Process

This book was written in partnership with AI assistants — Claude, Gemini, and Z.ai — in an iterative loop that mirrors the Conductor metaphor. Wen provides the vision, the architectural intuition, and the lived experience of building declarative systems. The AI partners contribute drafting speed, breadth of reference, and adversarial review.

Every technical claim was verified by running the actual code. Every benchmark was measured on real hardware. The AI partners accelerated the work; they did not replace the judgment.

This process is itself a demonstration of the book's thesis: humans conducting orchestras of AI, producing work neither could produce alone.

1.1.10 Acknowledgments

To my colleague who asked the question that started this journey — you know who you are.

To the readers in the Global South: may these recipes serve you well. May your hardware be sufficient, your latencies low, and your workflows reliable. May you build systems that help your communities.

To the SQL practitioners who have been waiting for an AI language that respects the mental model you have spent years building: this is it. Welcome to the orchestra.

"Every atom in a crystal is equally fundamental. Every recipe in this book is equally real — no toy examples, no omitted edge cases, no hardware you cannot buy on eBay."

— Wen Gong March 2026

Chapter 1.2

Why Declarative? SQL as the Mental Model

1.2.1 Who This Chapter Is For

This chapter is written for two audiences, and I want to name them directly.

If you are a SQL practitioner — a data engineer, BI developer, analyst, or anyone who has spent years writing queries, stored procedures, or ETL pipelines — this chapter will feel like recognition. The declarative mental model you already use for data is the same model SPL applies to AI workflows. The transfer is structural, not metaphorical. By the end of this chapter, SPL will not feel like a new language. It will feel like an extension of one you already know.

If you are in the Global South, or anywhere that frontier AI infrastructure is expensive, slow, or inaccessible — this chapter is equally for you. The reason SPL matters for accessibility is not just that it is simpler to write. It is that it decouples the *logic* of a workflow from the *infrastructure* that runs it. A workflow you develop on a GTX 1080 Ti in Nairobi runs on a Momagrid node in São Paulo, a cloud API in Singapore, or a local Ollama instance in Hefei — without changing a single line of code. You write against the language. The runtime figures out the rest.

Both audiences are building the same thing: AI workflows that are readable, portable, and maintainable. The path just looks different depending on where you are starting from.

1.2.2 The Problem You Already Solved

You have written SQL for years. Maybe a decade. You know what it feels like to sit down with a business question — "how many customers churned last quarter by region?" — and translate it directly into a query that returns exactly that, without worrying about B-tree traversal strategies, page cache management, or network round-trips to the storage layer.

You wrote:

```sql
SELECT region, COUNT(*) AS churned
FROM customers
WHERE status = 'churned'
  AND churned_at BETWEEN '2025-10-01' AND '2025-12-31'
GROUP BY region
ORDER BY churned DESC;
```

The database figured out the rest.

This is declarative programming. You describe *what* you want. The engine decides *how* to produce it. The physical details — indexes, join algorithms, parallelism — are someone else's problem. Your job is the query.

Now contrast that with what most engineers write the first time they try to build an LLM workflow.

1.2.3 What Imperative LLM Code Looks Like

Here is a realistic Python script for a simple draft-critique-refine loop. Three steps: draft a proposal, critique it, refine based on the critique. Straightforward in description. Watch what happens in implementation.

```python
import anthropic
import json
import sys
import time

client = anthropic.Anthropic(api_key=os.environ["ANTHROPIC_API_KEY"])

def draft_proposal(topic: str) -> str:
    response = client.messages.create(
        model="claude-opus-4-5",
        max_tokens=1024,
        messages=[
            {
                "role": "user",
                "content": f"Write a one-paragraph proposal for: {topic}"
            }
        ]
    )
    return response.content[0].text

def critique_proposal(draft: str) -> str:
    response = client.messages.create(
        model="claude-opus-4-5",
        max_tokens=512,
        messages=[
            {
                "role": "user",
```

```
                "content": f"Critique this proposal. Be specific about weaknesses:\
            }
        ]
    )
    return response.content[0].text

def refine_proposal(draft: str, critique: str) -> str:
    response = client.messages.create(
        model="claude-opus-4-5",
        max_tokens=1024,
        messages=[
            {
                "role": "user",
                "content": (
                    f"Revise this proposal based on the critique below.\n\n"
                    f"Original:\n{draft}\n\n"
                    f"Critique:\n{critique}"
                )
            }
        ]
    )
    return response.content[0].text

if __name__ == "__main__":
    topic = sys.argv[1] if len(sys.argv) > 1 else "a new employee onboarding proces

    print("Drafting...")
    draft = draft_proposal(topic)

    print("Critiquing...")
    critique = critique_proposal(draft)

    print("Refining...")
    result = refine_proposal(draft, critique)

    # Save output
    with open("output.json", "w") as f:
        json.dump({
            "topic": topic,
            "draft": draft,
            "critique": critique,
            "result": result,
            "timestamp": time.time()
        }, f, indent=2)

    print(result)
```

This works. It runs. A developer reviewing it can follow the logic. But look at what you had to

decide:

Which model. What max_tokens. How to format the messages list. How to extract response.content[0].text — the right accessor for this SDK version. How to pass context between steps. How to handle output. Whether to save intermediate states. What to do if the API call fails (this script does nothing — it crashes).

None of those decisions are about your *intent*. They are all about the *mechanics* of talking to one specific provider's API in one specific programming language using one specific SDK version that may have breaking changes in six months.

The business logic — draft, critique, refine — is buried inside three functions that look nearly identical. The signal is drowning in noise.

And if you want to switch from Claude to Gemini to test whether it produces better proposals? You rewrite three functions, update the import, change the accessor, and pray the response schema is compatible. Then you maintain two versions of the same logic.

This is the imperative LLM problem. You are managing infrastructure details when you should be thinking about intent.

1.2.4 The SQL Analogy

SQL solved this problem for data in the 1970s. The insight was simple but radical: separate *what* you want from *how* the system produces it.

Before SQL, you wrote navigational database code. You traversed records manually, managed cursors, tracked positions in file hierarchies. To get the churn report, you wrote a loop. You opened a file handle. You read records. You checked dates. You aggregated. The query and the execution plan were the same thing.

SQL said: stop. Describe the result set. The query optimizer will figure out whether to use a nested loop join or a hash join, whether to scan the index forward or backward, whether to parallelize across partitions. That is the engine's job. Your job is the query.

SPL applies the same insight to LLM workflows.

You describe the workflow — what steps exist, what each step produces, how data flows between steps, what conditions branch the logic. SPL figures out how to execute it: which model to call, how to format the prompt for that model's API, how to parse the response, how to pass context forward.

The mental model shift is not about learning new syntax. It is about recognizing a pattern you already know and applying it to a new domain.

1.2.5 The Mapping

SQL and SPL share more than a philosophy. They share a structural vocabulary.

SELECT → GENERATE

In SQL, SELECT defines what you want to retrieve or compute. In SPL, GENERATE defines what you want the LLM to produce.

```
-- SQL: retrieve a computed column
SELECT summarize(article_text) AS summary
FROM articles
WHERE published_at > '2025-01-01';

-- SPL: generate a summary using an LLM
GENERATE summarize(article_text) INTO @summary
```

Both express *what* you want. Neither tells the engine *how* to produce it.

WHERE → EVALUATE

In SQL, WHERE filters rows based on a condition. In SPL, EVALUATE branches execution based on the content of an LLM output.

```
-- SQL: filter based on a computed value
SELECT *
FROM proposals
WHERE sentiment_score > 0.7;

-- SPL: branch based on an LLM judgment
EVALUATE @sentiment
  WHEN 'positive' THEN
    COMMIT @proposal WITH status = 'approved'
  OTHERWISE
    GENERATE revision(@proposal, @feedback) INTO @proposal
END
```

SQL's WHERE is deterministic — you know the filter will behave the same every time. SPL's EVALUATE is probabilistic — the LLM decides the value of @sentiment, so the branch depends on a judgment call. The syntax looks similar because the intent is similar: route data based on a condition.

Stored Procedure → WORKFLOW

In SQL, a stored procedure encapsulates a named, parameterized sequence of operations. In SPL, a WORKFLOW does the same.

```
-- SQL stored procedure
CREATE PROCEDURE generate_churn_report(
    start_date DATE,
    end_date DATE
)
AS BEGIN
    SELECT region, COUNT(*) AS churned
    FROM customers
    WHERE churned_at BETWEEN start_date AND end_date
```

```
    GROUP BY region;
END;

-- SPL workflow
WORKFLOW draft_and_refine
  INPUT:
    @topic TEXT,
    @max_iterations INTEGER DEFAULT 3
  OUTPUT: @result TEXT
DO
  GENERATE draft(@topic) INTO @current
  -- ... loop and refine
END
```

Both are named, reusable, parameterized. Both separate the caller (who provides inputs) from the implementation (which decides how to produce outputs).

UDF → CREATE FUNCTION

In SQL, a user-defined function returns a scalar or table value that the query engine can use anywhere a value is expected. In SPL, CREATE FUNCTION defines a system context block — effectively a prompt component — that can be injected into any GENERATE call.

```
-- SQL UDF
CREATE FUNCTION business_hours(ts TIMESTAMP)
RETURNS BOOLEAN AS $$
    RETURN EXTRACT(HOUR FROM ts) BETWEEN 9 AND 17;
$$ LANGUAGE plpgsql;

-- SPL function: defines system context injected as an LLM prompt
CREATE FUNCTION expert_reviewer()
RETURNS TEXT AS $$
  You are a senior technical reviewer with 15 years of experience.
  You provide specific, actionable feedback.
  You do not hedge or soften criticism.
$$
```

The SQL UDF is called by the query engine wherever the function name appears. The SPL function is injected as system context wherever the function name appears in a GENERATE call. Same concept: composable, reusable units that plug into a larger expression.

1.2.6 CALL vs. GENERATE: The Most Important Distinction

There is one concept in SPL that has no direct SQL analog, but every SQL practitioner will immediately understand it once the motivation is clear.

SPL has two types of steps: CALL and GENERATE.

CALL invokes a deterministic Python function. It costs nothing in tokens. It runs at machine speed. It is perfectly reproducible — run it a thousand times with the same input and you get the same output every time.

GENERATE invokes an LLM. It costs tokens. It is relatively slow. It is probabilistic — run it a thousand times and you get slightly different outputs each time.

The rule is simple: **use CALL for everything code can do. Use GENERATE only for what genuinely requires reasoning or generation.**

A SQL practitioner understands this instinctively. If you can write it as WHERE date > '2025−01−01', you do not need a subquery. If you can compute it in the SELECT clause with a deterministic expression, you do not call a stored procedure. You use the cheapest, most reliable tool that produces the right result.

```
-- Wrong: asking the LLM to do arithmetic
GENERATE count_words(@document) INTO @word_count

-- Right: use a Python function
CALL count_words(@document) INTO @word_count
```

The LLM cannot count words more reliably than len(text.split()). It costs more. It is slower. It might be wrong. There is no reason to involve it.

```
-- Wrong: trying to parse a JSON field in SPL when code can do it
GENERATE extract_author(@raw_json) INTO @author

-- Right: deterministic extraction
CALL extract_field(@raw_json, 'author') INTO @author
```

The LLM cannot parse JSON more reliably than json.loads(). It should not be asked to.

```
-- Right: using the LLM for what only it can do
GENERATE assess_argument_quality(@essay, @criteria) INTO @assessment
```

Is this argument coherent? Does it address the counterargument? Is the evidence relevant? Code cannot do this. A regex cannot do this. A lookup table cannot do this. This is where LLM tokens are earned.

The CALL/GENERATE split is the single most important design decision in any SPL workflow. It determines quality, cost, and reliability all at once. A workflow that routes everything through GENERATE is like a SQL query that computes everything in a subquery when a simple expression would suffice — technically correct, unnecessarily expensive, and harder to debug.

1.2.7 The Adapter Abstraction

Here is where the SQL analogy pays its biggest dividend.

When you write SQL, you do not write "PostgreSQL." You write SQL. The same SELECT ∗ FROM customers WHI
runs on PostgreSQL, MySQL, BigQuery, and Snowflake. The syntax may have minor dialects.

The performance characteristics differ. But the query expresses the same intent and produces the same result set regardless of the engine underneath.

SPL works the same way for LLM workflows. The same .spl file runs against Ollama (local models), OpenRouter (API-based cloud models), or claude_cli (Claude directly). You change the adapter flag at the command line. The workflow does not change.

```
# Same .spl file, three different adapters
spl run workflow.spl --adapter ollama -m gemma3
spl run workflow.spl --adapter openrouter -m openai/gpt-4o
spl run workflow.spl --adapter claude_cli -m claude-opus-4-5
```

A SQL database does not expose B-tree implementations to your queries. An SPL adapter does not expose API schemas, authentication headers, response parsing, or token accounting to your workflows. These are infrastructure concerns. The workflow describes intent. The adapter handles execution.

This matters enormously in practice. When you build a workflow against a local Ollama model during development, you are not taking on technical debt. You are building the real workflow. When you move to production on a cloud model, you change one flag. The workflow file is unchanged, unmodified, and untested against the new model — but it runs, because the adapter handles the translation.

SQL practitioners know what it is like to develop against a local database and deploy against a production cluster. The adapter abstraction is the same thing.

This portability has a deeper implication that matters especially if you are in the Global South or working with constrained infrastructure. The same .spl workflow that you develop locally on Ollama can be submitted to **Momagrid** — a decentralized inference network where GPU nodes contributed by independent operators around the world form a shared compute grid. No cloud account. No API key from a company headquartered in another continent. Just change the adapter flag:

```
# Develop locally — your own GPU, zero cost
spl run workflow.spl --adapter ollama -m gemma3

# Run on the decentralized grid — community compute
spl run workflow.spl --adapter momagrid -m gemma3
```

The workflow is identical. The infrastructure is different. The language does not know or care which runtime executes it.

This is the full vision: SPL as the score, Momagrid as the distributed orchestra. Anyone with a GPU can contribute capacity. Anyone with a workflow can run it. The barrier is a used GPU and a pip install — not a credit card linked to a cloud provider in Virginia.

1.2.8 Side by Side

The clearest way to see the difference is to solve the same problem both ways. A draft-critique-refine loop: given a topic, generate a draft, have the model critique it, refine based on the critique.

Python (imperative)

```
import anthropic
import os

client = anthropic.Anthropic(api_key=os.environ["ANTHROPIC_API_KEY"])

def run(topic: str) -> str:
    # Step 1: Draft
    draft_response = client.messages.create(
        model="claude-opus-4-5",
        max_tokens=1024,
        system="You are an experienced technical writer.",
        messages=[{"role": "user", "content": f"Write a draft proposal for: {topic}
    )
    draft = draft_response.content[0].text

    # Step 2: Critique
    critique_response = client.messages.create(
        model="claude-opus-4-5",
        max_tokens=512,
        system="You are a critical editor. Find specific weaknesses.",
        messages=[{"role": "user", "content": f"Critique this draft:\n\n{draft}"}]
    )
    critique = critique_response.content[0].text

    # Step 3: Refine
    refine_response = client.messages.create(
        model="claude-opus-4-5",
        max_tokens=1024,
        system="You are an experienced technical writer.",
        messages=[{
            "role": "user",
            "content": f"Revise the draft based on this critique.\n\nDraft:\n{draft
        }]
    )
    return refine_response.content[0].text

if __name__ == "__main__":
    import sys
    result = run(sys.argv[1])
    print(result)
```

Count the decisions baked into this file: the provider (Anthropic), the model name (hardcoded), the API key source (environment variable name), the max_tokens values (guessed), the response accessor (.content[0].text), the system prompt duplication (writer appears twice), the context formatting (manual string concatenation), and the complete absence of error handling.

SPL (declarative)

```
CREATE FUNCTION technical_writer()
RETURNS TEXT AS $$
  You are an experienced technical writer.
  Your drafts are clear, specific, and actionable.
$$

CREATE FUNCTION critical_editor()
RETURNS TEXT AS $$
  You are a critical editor. Find specific weaknesses.
  Be direct. Do not soften your feedback.
$$

WORKFLOW draft_critique_refine
  INPUT:
    @topic TEXT
  OUTPUT: @result TEXT
DO
  GENERATE draft(technical_writer(), @topic) INTO @current
  GENERATE critique(critical_editor(), @current) INTO @feedback
  GENERATE refine(technical_writer(), @current, @feedback) INTO @result

  COMMIT @result WITH topic = @topic, status = 'complete'
END
```

The SPL version describes the workflow. The Python version implements it. Every line in the SPL file is about intent. Most lines in the Python file are about mechanics.

The SPL version runs on any adapter. The Python version runs on Anthropic's API with the claude-opus-4-5 model. Changing models requires editing the source. Changing providers requires rewriting the client.

The SPL version has error handling built into the runtime (the adapter layer catches API failures, retries, and surfaces them as typed exceptions the workflow can handle). The Python version crashes.

The SPL version is readable by someone who has never seen LLM code before — they will recognize the pattern immediately because it looks like a stored procedure. The Python version is readable only to someone who knows this specific SDK's response schema.

Neither version is "better" in every situation. Python gives you the full language for cases where SPL's abstraction is too constraining. But for the canonical LLM workflow patterns — draft, critique, refine, route, retry — SPL is not a workaround. It is the right tool.

1.2.9 Why This Matters for SQL Practitioners Specifically

You already know declarative thinking. You already know how to describe what you want and let an engine figure out how to produce it. You already know how to decompose a complex data transformation into named, composable steps. You already know how to think in pipelines.

The SQL-to-SPL transfer is not a leap. It is a lateral move.

The domain changes: instead of rows and columns, you are working with text and language model outputs. The stakes change: instead of a deterministic join, you are routing through a probabilistic judgment. The tooling changes: instead of psql or dbeaver, you use spl.

But the mental model is the same. Describe what you want. Let the engine handle how. Trust the abstraction until you have a reason not to.

One more thing carries over directly: the instinct to profile before optimizing. SQL practitioners do not rewrite queries on a hunch — they check the execution plan, identify the bottleneck, and fix the specific problem. SPL workflows deserve the same discipline. Do not route through GENERATE unless you have confirmed that CALL cannot do the job. Do not run a three-step GENERATE loop when a single well-constructed prompt would suffice. Spend tokens where they earn their cost.

The models are the orchestra. SPL is the score. The score says what to play and in what order. The conductor does not tune the instruments or manage the rosin — the adapter handles that. Your job is to write music worth performing.

You already know how to think about this. You have been doing it with data for years. The domain is new. The discipline is not.

1.2.10 What Comes Next

The next chapter gets you running. Install spl−llm, pull a model, and execute a workflow in under fifteen minutes. From there, every recipe in this book is open to you.

The conceptual work is done here. The rest is practice.

Chapter 1.3

Installing spl-llm, Running Your First Workflow

"Before you can conduct, you must first gather the orchestra."

1.3.1 The Path to Hello World

If you have spent any time in the modern AI ecosystem, you are likely used to a "dependency nightmare." You spend more time debugging your Python virtual environment, updating the OpenAI SDK, and managing your API keys than you do actually writing prompts.

SPL 2.0 changes the game. By moving the complexity into the runtime, it reduces the "install and run" process to a single pip install and a model pull. In this chapter, we will go from zero to a working workflow in under fifteen minutes.

1.3.2 Step 1: Install the Runtime

The spl−llm package is the heart of the ecosystem. It contains the parser, the execution engine, and all the adapters for local and cloud models.

It is available on PyPI:

```
# We recommend using a clean virtual environment
python -m venv venv
source venv/bin/activate

# Install the runtime
pip install spl-llm
```

Verify the installation by running the spl binary:

```
spl --version
# Expected: spl-llm v2.0.x
```

1.3.3 Step 2: Set Up Your Orchestra (Ollama)

While SPL can connect to cloud providers like Claude and OpenAI, the core mission of this book is to run AI on **your own hardware**. For this, we use **Ollama**, the most popular open-source tool for running models locally.

1. **Install Ollama**: Download it from ollama.com.
2. **Start the Service**: Ensure ollama serve is running in your background.
3. **Pull a Model**: For most recipes in this book, we recommend **gemma3 (27B)** for reasoning and **phi4** or **Llama 3.2** for speed.

```
# Pull our primary "conductor" model
ollama pull gemma3
```

1.3.4 Step 3: Run Your First Workflow

Navigate to the Cookbook−of−SPL−Recipes directory. We will run the **Hello World** recipe from Chapter 2.1.

```
spl run cookbook/01_hello_world/hello.spl --adapter ollama -m gemma3
```

You should see an output like this:

```
Hello! I'm Gemma, a helpful AI assistant...
SPL 2.0 is the declarative score for this performance.
```

If you see this, your "stove is on." You have a working, end-to-end environment that can parse SPL files, communicate with an adapter, and generate intelligence from a local model.

1.3.5 The Adapter Configuration

By default, SPL looks for a .spl/config.yaml file to manage your adapters. You can store your API keys and default models there to save typing.

Example configuration for cloud models:

```
adapters:
  anthropic:
    api_key: "your-key-here"
    model: "claude-3-5-sonnet"
  ollama:
    host: "localhost:11434"
    model: "gemma3"
```

Once configured, you can run any recipe without flags:

```
# Uses the defaults in your config file
spl run hello.spl
```

1.3.6 Troubleshooting

- **"Model not found"**: Ensure you have pulled the model using ollama pull.
- **"Connection refused"**: Check that ollama serve is running.
- **"VRAM Limit"**: If your model is slow or crashes, you might be exceeding your GPU's memory. Try a smaller model like phi4 or llama3.2:3b.

1.3.7 What Just Happened?

You didn't write any Python. You didn't import an SDK. You didn't manage a response envelope. You simply pointed a **Score** (hello.spl) at an **Orchestra** (ollama) and let the **Runtime** (spl) handle the performance.

In the next part, we will dive into the **Basics** and see how to pass parameters, handle multiple languages, and compare different models side-by-side.

Part 2

Basics

Chapter 2.1

Hello World

"Before you cook anything, you verify that the stove is on."

2.1.1 The Pattern

Every technology stack demands a ritual: write the simplest possible program, run it, and confirm that the scaffolding holds. SPL is no different. Before you write a multi-step workflow that CALLs tools and GENERATEs structured output, you need to know that spl is installed, the adapter is configured, the model is running, and the runtime can connect them end to end.

The naive approach is to write a Python script: import the Ollama client, define a system message, send the request, print the response. It works. It also requires you to manage the HTTP client, handle the response envelope, deal with streaming vs. non-streaming modes, and remember the exact API shape of whichever model host you are using. For a sanity check, that is too much ceremony.

In SQL, the equivalent sanity check is:

```
SELECT 'hello' FROM dual;
```

One row. One column. Confirms the connection is live, the parser understood you, and the engine returned data. No tables, no joins, no logic. Just proof that the pipe works.

SPL's hello world is exactly that: one PROMPT block, one GENERATE call, no parameters, no tools, no state management. If it runs and returns text, the stack is healthy.

2.1.2 The SPL Approach

A PROMPT block assembles context in a SELECT clause, then fires a single GENERATE call into the assembled context. The model reads the context and produces the response. One LLM call, no moving parts.

2.1.3 The .spl File (Annotated)

```
-- Recipe 01: Hello World
-- Minimal SPL program — verify spl + adapter + model work.
--
-- Usage:
--   spl run cookbook/01_hello_world/hello.spl
--   spl run cookbook/01_hello_world/hello.spl --adapter ollama
--   spl run cookbook/01_hello_world/hello.spl --adapter ollama -m gemma3

PROMPT hello_world                              -- (1) declare a named PROMPT block
SELECT
    system_role('You are a friendly assistant. Introduce yourself and SPL 2.0 in tw
                                                -- (2) assemble context in SELECT
GENERATE greeting()                             -- (3) fire the LLM call
```

Three lines of logic. Here is what each one does.

2.1.3.1 (1) `PROMPT hello_world`

PROMPT is the simplest SPL construct. It is a single-step program: select context, generate response, done. You can think of it as a named view — it has a name that the runtime uses for logging and tracing, but it takes no parameters and returns one result.

The SQL analogy is a named query or a view with no WHERE clause:

```
CREATE VIEW hello_world AS
SELECT 'hello from the database';
```

You are not filtering. You are not joining. You are simply declaring what this block does and giving it a name the runtime can track.

2.1.3.2 (2) `SELECT system_role(...)`

The SELECT clause in a PROMPT block assembles the context that will be given to the LLM when GENERATE fires. Each item in the SELECT is a piece of context: a system instruction, a user input, a retrieved document, a computed value.

system_role() is a built-in SPL function that wraps its argument as a system-level instruction. In most LLM APIs, the system message establishes the model's role, constraints, and persona — it is the part of the conversation that precedes any user turn. Putting your instruction in system_role() signals to the adapter that this text should be injected at the system level, not as user input.

The SQL analogy: the SELECT clause computes what you hand to the LLM, just as a SQL SELECT computes what you return to the caller. Here, you are computing exactly one thing: a system instruction string. That string becomes the model's full context for this prompt.

In later recipes, the SELECT clause will grow — you will add context.param AS column_name, function_call() AS context_label, and multiple joined context items. The structure is always the same: SELECT assembles a context row, GENERATE sends it to the model.

2.1.3.3 (3) `GENERATE greeting()`

GENERATE is an LLM call. It takes the context assembled by SELECT, sends it to the configured adapter, waits for the response, and makes the result available to the rest of the workflow.

greeting() is the step name, not a function you defined. It is how this call appears in logs, traces, and error messages. Choose step names that describe what the model is doing, not what you are hoping for.

The parentheses can accept arguments — in later recipes you will pass GENERATE classify(text, examples) or GENERATE translate(content, target_language). In this recipe, there are no arguments because the entire context is in the SELECT clause. The GENERATE step sends whatever SELECT assembled.

The SQL analogy:

```
-- SQL: compute a value
SELECT UPPER('hello world') AS result;

-- SPL: generate a value using the LLM
GENERATE greeting()
```

UPPER() is deterministic. greeting() is not. That is the entire difference. Both take input from what SELECT assembled. Both produce a value the runtime captures.

The critical distinction: SPL uses GENERATE for operations that require reasoning, language, or judgment — things only a model can do. It uses CALL for deterministic operations: loading files, computing checksums, looking up values, writing output. In hello world, you have one GENERATE and no CALLs, because the entire task is generative. Later recipes will mix them, and the discipline of separating them is where most of the design work lives.

2.1.4 Running It

Ensure Ollama is running and gemma3 is pulled:

```
ollama serve          # if not already running as a service
ollama pull gemma3    # one-time download, ~5 GB
```

Run the recipe:

```
spl run cookbook/01_hello_world/hello.spl --adapter ollama -m gemma3
```

Expected output (text will vary across runs):

```
Hello! I'm Gemma, a helpful AI assistant developed by Google DeepMind.
SPL 2.0, or Structured Prompt Language, is a declarative language for
orchestrating LLM workflows — think SQL for AI pipelines.
```

The response is printed to stdout. No files are written, no state is committed. This is a pure read operation.

You can also run without specifying an adapter if Ollama is your configured default:

```
spl run cookbook/01_hello_world/hello.spl
```

And you can swap models without changing the file:

```
spl run cookbook/01_hello_world/hello.spl --adapter ollama -m llama3.2
spl run cookbook/01_hello_world/hello.spl --adapter ollama -m mistral
```

The .spl file does not change. The adapter and model are runtime concerns, not workflow concerns. This is an early demonstration of a principle that pays off in later recipes: SPL workflows are model-agnostic by design.

2.1.5 What Just Happened

LLM calls: 1.

The runtime parsed hello.spl, identified the PROMPT hello_world block, assembled the context from the SELECT clause (one system instruction string), and handed it to the Ollama adapter with a request to the gemma3 model. The model generated a response. The runtime printed it and exited.

No files were read. No tools ran. No variables were set. No exception handlers were evaluated.

This is the minimal path through the SPL runtime. Every more complex recipe is this path plus additional steps layered on top:

- WORKFLOW blocks add INPUT parameters and multi-step sequences
- CALL steps add deterministic operations alongside the generative ones
- EVALUATE adds branching on model output
- COMMIT adds persistence
- EXCEPTION adds error handling

None of those are present here. Hello world establishes the floor: PROMPT + SELECT + GENERATE = one model call, one response, done.

2.1.6 Reproducibility Note

Latency on a GTX 1080 Ti with gemma3: **3–8 seconds** for a two-sentence response. The range is wide because a cold Ollama server will load the model weights into VRAM on the first call — subsequent calls in the same session are faster once the model is warm.

The response content varies across runs. The model is not seeded to a fixed random state, so each run produces a different phrasing. This is expected and correct for a generative sanity check. What you are verifying is not the exact text but that text arrives at all.

If you need reproducible output for a test: set temperature=0 via the adapter configuration, or use a workflow with structured output constraints (see Chapter 2.4). For a hello world, variability is not a problem — it is confirmation that the model is actually generating, not returning a cached string.

2.1.7 When to Use This Pattern

The PROMPT form — PROMPT name / SELECT context / GENERATE step() — is the right shape for:

- **Stack verification**: Confirm the runtime, adapter, and model are wired together correctly before writing anything more complex.
- **One-shot queries**: A single natural-language question that needs one LLM response and no post-processing. The Ollama proxy recipe (Chapter 2.2) extends this pattern with a parameter.
- **Prototyping a system message**: When you are tuning the persona or instruction in system_role(), a PROMPT block lets you iterate without running a full workflow.
- **Demonstrations and smoke tests**: CI pipelines that verify adapter connectivity use a hello-world-style PROMPT as the probe.

The PROMPT form is not the right shape when:

- You need to pass parameters from the command line (use WORKFLOW with INPUT)
- You need multiple sequential LLM calls (use WORKFLOW with multiple GENERATE steps)
- You need to call Python tools alongside the LLM (use WORKFLOW with CALL)
- You need to branch on the model's output (use WORKFLOW with EVALUATE)

Think of PROMPT as SELECT without a FROM: fast, useful for spot-checks, not a substitute for a full query.

2.1.8 Exercises

1. **Change the system instruction.** Edit the system_role() string to 'You are a gruff database administrato
   Re-run. Observe how the same GENERATE greeting() call produces a completely different response when the system context changes. The step name does not constrain the output — the context does.

2. **Swap the adapter.** If you have an Anthropic API key configured, run:

```
spl run cookbook/01_hello_world/hello.spl --adapter anthropic
```

The .spl file is unchanged. The response will reflect Claude's style rather than gemma3's. This demonstrates adapter portability: the workflow is the specification, the model is the execution engine.

3. **Promote to a WORKFLOW.** Take the PROMPT block and rewrite it as a WORKFLOW with an INPUT parameter called @persona TEXT DEFAULT 'friendly assistant'. Pass the persona into the system_role() string. Run it with persona="skeptical engineer" and persona="enthusiastic intern". This is the shape of Recipe 02.

Next: Chapter 2.2 — Ollama Proxy: Parameterized LLM Queries

Chapter 2.2

Ollama Proxy

"The simplest tool is often the most powerful when it becomes a building block."

2.2.1 The Pattern

In the previous chapter, we verified the "Hello World" of SPL: a hardcoded prompt that proves the connection between the runtime and the model. But real-world utility requires input. You don't just want the model to introduce itself; you want to ask it *your* questions.

The common way to interact with a local LLM is via a dedicated chat UI or a specific CLI tool like ollama run. These are great for interactive sessions, but they are hard to script or integrate into a larger pipeline. If you want to use the output of a model as a step in a bash script or a CI process, you often end up writing a wrapper script in Python or Node.js to handle the HTTP request, extract the text from the JSON response, and print it to stdout.

The SQL analogy is a parameterized query:

```
SELECT * FROM users WHERE user_id = ?;
```

You don't rewrite the query every time you want a different user; you pass the user_id as a parameter. The query structure remains constant, while the data varies.

The Ollama Proxy recipe is the SPL version of a parameterized query. It defines a reusable prompt structure that accepts a prompt argument from the command line, sends it to the model, and returns the result. It turns any LLM into a standard Unix-style utility.

2.2.2 The Open-Source Engine Room: Ollama and Llama.cpp

Before we dive into the code, we must acknowledge the "engine room" that makes this recipe possible: **Ollama** and **Llama.cpp**.

In the proprietary AI world, models are locked behind APIs. You pay per token, you need a credit card, and you need a high-speed, uncensored connection to a data center in the Global North. For

an engineer in Lagos, a student in rural Anhui, or a researcher in Recife, this is a barrier to entry that reproduces old inequalities.

Llama.cpp (and its user-friendly distribution, **Ollama**) changed the game. By optimizing LLM inference for consumer-grade hardware, they turned the "commodity hardware" in your closet—like a used **GTX 1080 Ti**—into a powerful AI workstation. These are not just technical achievements; they are tools of digital sovereignty.

In the Momagrid vision, these open-source foundations are the "orchestra" in our conductor metaphor. SPL 2.0 provides the "score" (the logic), but Ollama provides the "instruments" (the models). By advocating for open-source models (like Gemma, Llama, or Mistral) running on open-source runtimes, we ensure that AI remains a public utility accessible to everyone, everywhere, without a middleman.

When you run this recipe, you aren't just calling a model; you are exercising your right to run state-of-the-art intelligence on your own terms.

2.2.3 The SPL Approach

The core idea is to use the context object to bridge the gap between the command line and the SPL runtime. By selecting context.prompt, we take a named argument passed at runtime and inject it into the LLM's context.

2.2.4 The .spl File (Annotated)

```
-- Recipe 02: Ollama Proxy
-- General-purpose LLM query -- proxy any prompt to any adapter/model.
--
-- Usage:
--   spl run cookbook/02_ollama_proxy/proxy.spl prompt="Explain quantum computing"

PROMPT ollama_proxy                                -- (1) name the prompt block
SELECT
    system_role('You are a helpful, knowledgeable assistant.'),
                                                   -- (2) establish the persona
    context.prompt AS prompt                       -- (3) bind the CLI parameter
GENERATE answer(prompt)                            -- (4) execute the query
```

2.2.4.1 (1) `PROMPT ollama_proxy`

Like "Hello World," we use a PROMPT block because we only need a single LLM call. This is the "view" that we are executing.

2.2.4.2 (2) `system_role(...)`

We define a general-purpose persona. Since this script is a proxy for any question, we want a model that is helpful and knowledgeable. This stays constant regardless of the input.

2.2.4.3 (3) `context.prompt AS prompt`

This is the bridge. In SPL, the context object contains all parameters passed via the CLI (e.g., key="value"). By selecting context.prompt, we are telling SPL: "Look for a parameter named prompt in the environment and include its value in the context row."

The AS prompt alias is important. It gives the piece of context a label that we can refer to in the GENERATE step.

SQL Analogy:

```
SELECT :input_param AS user_query;
```

You are taking an external variable and bringing it into the scope of your query.

2.2.4.4 (4) `GENERATE answer(prompt)`

We trigger the LLM call. Unlike Recipe 01, we now pass an argument to the step: answer(prompt). This tells the runtime to specifically emphasize the prompt column from our SELECT statement as the primary instruction for the model.

2.2.5 Running It

You can pass the prompt parameter as a key-value pair after the file path:

```
spl run cookbook/02_ollama_proxy/proxy.spl --adapter ollama -m gemma3 \
    prompt="Explain quantum computing in one sentence"
```

Expected output:

```
Quantum computing uses the principles of subatomic physics to perform complex
calculations that are currently impossible for traditional computers.
```

Because the adapter and model are specified at the CLI level, this single .spl file can be used to test any model in your Ollama library:

```
# Test with Llama 3.2
spl run cookbook/02_ollama_proxy/proxy.spl -m llama3.2 prompt="What is 2+2?"

# Test with Mistral
spl run cookbook/02_ollama_proxy/proxy.spl -m mistral prompt="Write a haiku about S
```

2.2.6 What Just Happened

LLM calls: 1.

The runtime performed the following steps: 1. Parsed the command-line arguments and populated the context object with prompt="Explain quantum computing...". 2. Executed the SELECT clause, combining the system_role string and the context.prompt value into a single context row. 3. Passed this row to the specified adapter/model. 4. Captured the model's output and printed it to the console.

If you omit the prompt= argument, the runtime will throw an error because context.prompt is undefined. This is similar to a SQL error when a required bind variable is missing.

2.2.7 Reproducibility Note

Since this is a proxy, its performance and stability depend entirely on the underlying model you choose.

On a **GTX 1080 Ti**, running **gemma3**, you should expect: - **Time to First Token (TTFT)**: ~200ms - **Generation Speed**: ~40-60 tokens per second

The reproducibility of the *answer* depends on the prompt complexity. For factual questions ("What is 2+2?"), most models are 100% stable. For creative tasks, expect high variance unless you configure the adapter for temperature=0.

2.2.8 When to Use This Pattern

Use the **Ollama Proxy** pattern when: - **CLI Tooling**: You want to call an LLM from a shell script, e.g., git commit −m "$(spl run proxy.spl prompt='Summarize these changes...')" - **Quick Model Testing**: You want to compare how different models handle the exact same prompt without writing any code. - **Generic Utility**: You need a "swiss army knife" script that can handle any one-shot task.

Do not use this pattern when: - **Structured Logic**: You need to validate the output or take different actions based on what the model says (use EVALUATE in a WORKFLOW). - **Chain of Thought**: You need the model to "think" through steps before answering (use the patterns in Part 3).

2.2.9 Exercises

1. **Multiple Parameters**: Modify the SELECT clause to accept a second parameter context.topic. Update the system_role to say 'You are an expert in ' || context.topic. Run it with topic="Biology" prompt="What is a cell?".
2. **Default Values**: (Advanced) Research how to use the COALESCE or DEFAULT equivalent in SPL to provide a fallback if context.prompt is missing.

3. **Shell Integration**: Use the proxy in a bash pipe. For example:

```
echo "The weather is nice today" | xargs -I {} spl run proxy.spl prompt="Translate
```

Chapter 2.3

Multilingual Greeting

"Language is the first barrier to knowledge; code should be the first bridge."

2.3.1 The Pattern

In a truly global AI ecosystem, assuming the user speaks English is a design flaw. Whether you are building a support bot for a marketplace in Southeast Asia or a localized tutor for a school in West Africa, the ability to pivot between languages dynamically is a core requirement.

The naive way to handle multilingual prompts is often to hardcode multiple prompts or use complex string interpolation in a programming language. You end up with a "prompt library" that is difficult to maintain and even harder to test across different models.

The SQL analogy is a multi-parameter join:

```
SELECT t.translation
FROM dictionary d
JOIN translations t ON d.id = t.id
WHERE d.word = ? AND t.language = ?;
```

You have the "intent" (the word) and the "constraint" (the language). By providing both, you get the specific result you need.

The Multilingual Greeting recipe takes this principle and applies it to a prompt. It accepts two parameters—user_input and lang—and uses them to constrain the model's response. It's a simple demonstration of how SPL can manage multi-dimensional context without the "spaghetti code" of imperative string formatting.

2.3.2 The SPL Approach

We use the context object to capture multiple CLI arguments and then "bind" them to semantic labels in our SELECT clause. This allows us to pass structured, multi-part instructions to the GENERATE step.

2.3.3 The .spl File (Annotated)

```
-- Recipe 03: Multilingual Greeting
-- Greet in any language — demonstrates parametric context with lang.
--
-- Usage:
--   spl run cookbook/03_multilingual/multilingual.spl user_input="hello" lang="Ch

PROMPT multilingual_greeting                    -- (1) name the prompt block
SELECT
    system_role('You are a friendly assistant. Respond in the language specified.')
                                                -- (2) system-level constraint
    context.user_input AS input,                -- (3) the user's message
    context.lang AS lang                        -- (4) the target language
GENERATE greeting(input, lang)                  -- (5) context-aware generation
```

2.3.3.1 (1) `PROMPT multilingual_greeting`

A single model call. The complexity is in the input, not the execution flow.

2.3.3.2 (2) `system_role(...)`

This establishes the "rules of the game." We tell the model to be friendly and, crucially, to respect the language specification provided in the context.

2.3.3.3 (3) & (4) `context.user_input AS input` and `context.lang AS lang`

This is where we "map" our command-line parameters into our query scope. By giving them clean aliases (input, lang), we make our GENERATE step more readable.

In SPL, every column in your SELECT clause becomes a piece of evidence for the model. By labeling them explicitly, you are giving the model a structured view of the world: "Here is the input, and here is the language constraint."

2.3.3.4 (5) `GENERATE greeting(input, lang)`

We pass both aliases as arguments to the GENERATE step. This tells the model: "Produce a greeting by considering both the input message and the language requirement."

2.3.4 Running It

Test it with different languages:

```
# To Chinese
spl run cookbook/03_multilingual/multilingual.spl user_input="hello wen" lang="Chi
```

```
# To French
spl run cookbook/03_multilingual/multilingual.spl user_input="hello wen" lang="Fren

# To Spanish
spl run cookbook/03_multilingual/multilingual.spl user_input="hello wen" lang="Spa
```

Expected output (for Chinese):

```
(Hello, Wen! Nice to see you. I am your assistant.)
```

2.3.5 What Just Happened

LLM calls: 1.

The runtime combined the system_role, the input, and the lang into a single prompt package. Because we are using an open-source model like **gemma3** via **Ollama**, the model's native multilingual capabilities are triggered.

Modern open-weights models are trained on vast, multi-lingual datasets. By explicitly naming the language in the prompt, you are activating the specific language weights within the model's neural network.

2.3.6 Reproducibility Note

The stability of this recipe varies based on the "popularity" of the language in the model's training set. - **High Stability**: English, Chinese, Spanish, French, German. - **Moderate Stability**: Hindi, Japanese, Portuguese. - **Lower Stability**: Low-resource languages (e.g., local dialects).

On a **GTX 1080 Ti**, the latency for a short greeting remains consistent at **2-5 seconds** regardless of the language, as the model's internal processing time is more dependent on the number of tokens generated than the specific language family.

2.3.7 When to Use This Pattern

Use the **Multilingual** pattern when: - **Global Reach**: You need to serve users in their native tongue without duplicating your logic. - **Dynamic Formatting**: You want to change the "style" or "format" of the output based on a parameter (e.g., lang="pirate", lang="formal", lang="bullet points"). - **Context Injection**: You have more than one variable piece of data to feed into the model.

2.3.8 Exercises

1. **Add a "Tone" parameter.** Modify the SELECT to accept context.tone and update the system_role to include it. Run with lang="Chinese" tone="extremely excited" and then tone="very bored".
2. **Combine with Translation.** Instead of just a greeting, change the prompt to be a "Translator." Set the system_role to 'You are a translator .' and the GENERATE step to translate(input, lang).
3. **Regional Dialects.** Try specific dialects like lang="Brazilian Portuguese" vs. lang="European Portuguese" See if your local model can tell the difference!

Chapter 2.4

Few-Shot Learning

"Show, don't just tell."

2.4.1 The Pattern

Even the most sophisticated LLMs sometimes struggle to follow complex instructions. You might ask for a "JSON object with sentiment and summary," and the model might return a markdown-formatted string with a conversational preamble: *"Sure! Here is the JSON you requested…"*. This "chattiness" breaks automated pipelines.

Few-Shot Learning (or Few-Shot Prompting) is the technique of providing the model with 2–3 "gold-standard" examples of what a perfect input-output pair looks like. By seeing examples, the model "calibrates" its tone, format, and reasoning pattern to match your expectations. It is often more effective than writing a three-page instruction manual.

The SQL analogy is a **Lookup Table** or a **Reference Dataset**. You aren't just giving the database a schema; you are giving it a small set of "row templates" that define what valid data looks like.

The Few-Shot Learning recipe (Recipe 24) demonstrates how to embed these examples directly into the SELECT context. It uses a domain-swappable set of examples to guide the model across different industries (Finance, Ops, General).

2.4.2 The SPL Approach

This recipe uses CREATE FUNCTION with a SELECT CASE block to deliver dynamic, domain-specific examples into the context row.

2.4.3 The .spl File (Annotated)

```
-- Recipe 24: Few-Shot Prompting
-- Embed gold-standard examples in context to guide output format.

CREATE FUNCTION few_shot_examples(domain TEXT DEFAULT 'general')
RETURNS TEXT AS $$
SELECT CASE domain                              -- (1) Domain-specific example sets
  WHEN 'finance' THEN
    '### Examples
      Input: "Revenue up 12% YoY."
      Output: {"sentiment": "positive", ...}'
  WHEN 'ops' THEN
    '### Examples
      Input: "Deployment succeeded."
      Output: {"status": "healthy", ...}'
  ELSE
    '### Examples
      Input: "New feature launched!"
      Output: {"sentiment": "positive", ...}'
END
$$;

PROMPT few_shot_classifier
SELECT
    system_role('You are a precise text classifier. Follow the examples exactly.'),
    few_shot_examples(context.domain) AS examples, -- (2) Dynamic context injection
    context.text AS text,
    context.domain AS domain
GENERATE classify(text, examples)               -- (3) Calibrated generation
```

2.4.3.1 (1) Dynamic Examples via `SELECT CASE`

Instead of hardcoding examples into the prompt string, we encapsulate them in a function. This allows us to maintain different "Reference Sets" for different domains in one place.

SQL Analogy: A **Stored Function** that returns a configuration string based on a parameter.

2.4.3.2 (2) Dynamic Context Injection

In the SELECT clause, we call few_shot_examples(context.domain). - If you pass domain="finance" on the CLI, the LLM receives finance-related examples. - If you pass domain="ops", it gets operations-related examples.

This makes your workflow "Context-Aware" without duplicating the prompt logic.

2.4.3.3 (3) Calibrated Generation

The GENERATE step receives both the user's text and the examples. Because the model sees the ### Examples block right before it generates its own response, it naturally adopts the same JSON structure and field names shown in the examples.

2.4.4 Running It

Run the classifier for the finance domain:

```
spl run cookbook/24_few_shot/few_shot.spl --adapter ollama \
    text="The quarterly results exceeded all analyst forecasts" \
    domain="finance"
```

Expected output (clean JSON):

```
{"sentiment": "positive", "magnitude": 0.9, "topics": ["earnings", "forecasts"], "s
```

Now try the operations domain:

```
spl run cookbook/24_few_shot/few_shot.spl --adapter ollama \
    text="System outage detected in EU-WEST-2" \
    domain="ops"
```

Expected output:

```
{"status": "incident", "severity": "high", "action_required": true, "summary": "Sys
```

2.4.5 What Just Happened

LLM calls: 1.

The "Conductor" (SPL Runtime) managed the calibration: 1. Looked up the domain parameter. 2. Retrieved the matching "Reference Set" from the few_shot_examples function. 3. Assembled a context package containing: System Role + Reference Examples + User Input. 4. Executed a single, high-precision generation.

2.4.6 Reproducibility Note

Few-shot prompting is one of the most effective ways to increase **Stability**. While a "Zero-Shot" prompt (no examples) might fail 5% of the time, a "Two-Shot" prompt usually reaches 99%+ reliability for formatting.

On a **GTX 1080 Ti**, adding a few examples adds negligible latency (a few milliseconds for the extra tokens) but saves hours of post-processing and debugging.

2.4.7 When to Use This Pattern

Use the **Few-Shot Learning** pattern when: - **Strict Format Requirements**: You need JSON, YAML, or a specific tone that the model doesn't always hit on the first try. - **Domain Specialization**: You want the same workflow to behave differently for "Medical" vs. "Legal" vs. "Code" tasks. - **Edge Case Handling**: If the model keeps making a specific mistake, add a "Negative Example" to the few-shot set to show it what *not* to do.

2.4.8 Exercises

1. **Add a "Negative" Case.** Modify the few_shot_examples to include a case where the input is "ambiguous" and the output should be an empty JSON object.
2. **Toggle Example Count.** Modify the function to accept a second parameter @count and use it to return 1, 2, or 3 examples. See how the accuracy changes.
3. **Cross-Domain Test.** Run the recipe with a Finance text but pass domain="ops". Observe how the model tries to force-fit finance data into an operations schema. This is a great way to understand the "gravity" of examples.

Part 3

Agentic Patterns

3.0.1 Self-Refine

"The first draft is never the answer. The discipline is knowing when to stop revising."

3.0.1.1 The Pattern

You have a task that benefits from iteration: drafting a persuasive email, writing technical documentation, composing a code review comment. The naive approach is to call the LLM once and accept whatever comes back. Experienced practitioners know the first pass is rarely production-ready.

The Python equivalent looks roughly like this:

```
draft = llm.generate(task)
for i in range(max_iterations):
    feedback = llm.critique(draft)
    if feedback.strip().lower() == "satisfactory":
        break
    draft = llm.refine(draft, feedback)
```

This is manageable to write, but awkward to maintain. The loop logic, the exit condition, and the budget handling are all tangled together. When you add exception handling for budget overruns or you want to pass quality metadata to the caller, the code grows in directions that obscure the original intent. Six months later, a new team member reads this and has to reconstruct the algorithm from the implementation details.

More practically: every team that uses LLMs for iterative generation writes this loop. They each write it slightly differently. There is no shared vocabulary for the pattern, no standard way to configure max_iterations from the outside, no consistent exception surface.

SPL gives this pattern a name and a syntax.

3.0.1.2 The SPL Approach

Generate a draft, then loop: critique the draft, exit if satisfactory, otherwise refine and repeat — up to a configurable iteration limit.

3.0.1.3 The .spl File (Annotated)

```
-- SPL 2.0: Self-Refine Pattern
-- Iteratively improves output through critique and refinement

WORKFLOW self_refine
  INPUT:
    @task text,
    @max_iterations integer default 5      -- caller controls iteration budget
  OUTPUT: @result text
DO
```

```
  @iteration := 0
  -- @max_iterations := 5                    -- default shown; overridable at runtime

  -- Initial draft — one LLM call, cold start
  GENERATE draft(@task) INTO @current
  --
  -- SQL analogy: this is the seed row that the UPDATE loop will refine.
  -- Think: INSERT INTO drafts VALUES (llm_generate(task));

  -- Iterative refinement loop
  WHILE @iteration < @max_iterations DO
    -- Ask the LLM to critique its own output
    GENERATE critique(@current) INTO @feedback
    --
    -- critique() is a CREATE FUNCTION (not shown in this file, but registered
    -- in the system context). It receives @current as input and is prompted
    -- to return either "satisfactory" or a specific critique.
    -- The LLM is not deciding the control flow — EVALUATE is.

    EVALUATE @feedback
      WHEN 'satisfactory' THEN
        -- Early exit: quality threshold reached before max_iterations
        -- SQL analogy: BREAK in a cursor loop when WHERE condition is met
        COMMIT @current WITH status = 'complete', iterations = @iteration
      OTHERWISE
        -- Refine: feed the critique back into the LLM alongside the draft
        GENERATE refined(@current, @feedback) INTO @current
        @iteration := @iteration + 1
    END
  END

  -- Loop exhausted without hitting 'satisfactory' — commit best effort
  -- SQL analogy: the cursor reached EOF; commit whatever state we're in
  COMMIT @current WITH status = 'max_iterations', iterations = @iteration

EXCEPTION
  WHEN MaxIterationsReached THEN
    -- Runtime raised this before our loop could: external budget enforcer
    COMMIT @current WITH status = 'partial'
  WHEN BudgetExceeded THEN
    -- Token/cost budget hit — commit whatever is in @current
    COMMIT @current WITH status = 'budget_limit'
END
```

Key design decisions to notice:

EVALUATE @feedback / WHEN 'satisfactory' — The LLM wrote @feedback, but the LLM does not decide whether to exit the loop. EVALUATE is deterministic string matching. The LLM's

job is to produce the word “satisfactory” or a critique. The runtime’s job is to act on it. This separation is intentional and important.

@max_iterations integer default 5 — This is not hardcoded. The caller decides the iteration budget. A CI pipeline might pass max_iterations=2 for speed; an offline document editor might pass max_iterations=10 for quality. The workflow has no opinion.

COMMIT @current WITH status = 'complete', iterations = @iteration — The COMMIT carries metadata. Downstream code can see how many iterations were needed. A monitoring dashboard can track this over time: if your average iterations is creeping up, your base draft quality may be degrading, or your critique prompt has become too strict.

EXCEPTION / WHEN BudgetExceeded — The runtime enforces a token budget independently of your loop counter. If costs spike mid-run, the workflow does not crash with an unhandled exception. It commits what it has and marks the status clearly.

3.0.1.4 The SQL Analogy in Depth

SQL practitioners are comfortable with cursor loops. The self-refine pattern is a cursor loop where the exit condition is semantic rather than positional.

A conventional cursor loop:

```
DECLARE cur CURSOR FOR SELECT id, text FROM drafts WHERE status = 'pending';
OPEN cur;
LOOP
  FETCH cur INTO v_id, v_text;
  EXIT WHEN NOT FOUND;
  UPDATE drafts SET text = improve(v_text) WHERE id = v_id;
END LOOP;
CLOSE cur;
```

The self-refine SPL workflow is structurally identical, except: - There is only one row (the current draft). - The exit condition (WHEN 'satisfactory') is evaluated by the LLM at each step, not derived from a SQL predicate. - The “UPDATE” is a GENERATE that produces a new version of the same row.

A stored procedure that calls UPDATE in a loop until a quality score exceeds a threshold is doing exactly what self_refine.spl does. The syntax differs. The logic is the same.

3.0.1.5 Running It

```
spl run cookbook/05_self_refine/self_refine.spl \
    --adapter ollama -m gemma3 \
    task="Write a concise product description for a noise-cancelling travel headpho
    max_iterations=3
```

Expected output (truncated):

```
[draft] Generating initial draft...
[critique] Iteration 0: feedback received
[refined] Iteration 0: draft updated
[critique] Iteration 1: satisfactory
[commit] status=complete iterations=1

Result:
Experience travel in silence. Our noise-cancelling headphones block ambient
cabin noise while delivering rich, balanced audio across a 30-hour battery
life. Lightweight folding design fits any carry-on pocket. USB-C charging
from 0 to 80% in 45 minutes.
```

3.0.1.6 What Just Happened

LLM calls made: 3 (1 draft + 1 critique at iteration 0 + 1 critique at iteration 1).

Step-by-step:

1. GENERATE draft(@task) — Cold start. The LLM writes a first-pass product description from nothing.
2. GENERATE critique(@current) at iteration 0 — The LLM evaluates its own output. In this run it found the draft vague ("rich audio" with no specifics), returned structured feedback.
3. GENERATE refined(@current, @feedback) at iteration 0 — The LLM revised the draft incorporating the critique. Concrete battery life, charge time, and weight claim were added.
4. GENERATE critique(@current) at iteration 1 — The LLM re-evaluated. Returned "satisfactory".
5. EVALUATE 'satisfactory' — Matched the exit branch. COMMIT fired with status='complete' and iterations=1.

The key insight: the workflow ran 2 fewer LLM calls than max_iterations would have permitted. Early exit is working correctly. On poor-quality base prompts, expect to see iterations=4 or iterations=5 (max).

3.0.1.7 Reproducibility Note

Hardware: GTX 1080 Ti, 11GB VRAM. **Model:** gemma3 via Ollama.

Typical wall-clock time for a short copywriting task: 8–14 seconds for a 2-iteration run, 18–28 seconds for a 5-iteration run.

Stability: The critique function's "satisfactory" output is sensitive to prompt wording. If your critique prompt does not explicitly instruct the model to return the exact string "satisfactory" (and nothing else) when the draft meets the bar, you will see false negatives — the EVALUATE branch will always fall through to OTHERWISE, exhausting max_iterations every run. Test your critique prompt in isolation before embedding it in the loop.

Non-determinism: Two runs on identical input will often differ at iteration count. One run exits at iteration 1, the next at iteration 2. This is expected. The self-refine pattern is not designed to be deterministic at the iteration level — only at the structural level (the loop always terminates, the output is always committed, exceptions are always handled).

3.0.1.8 When to Use This Pattern

Good fit: - Content generation where quality has a measurable (if fuzzy) threshold: product copy, executive summaries, cover letters, commit messages. - Code documentation where "complete and accurate" is a binary outcome that can be evaluated by a second LLM pass. - Any task where you have seen humans naturally do "write, review, rewrite" manually.

Poor fit: - Tasks with an objective correct answer. Use CALL + a Python validator instead of a critique loop. An LLM critiquing a SQL query it generated may not catch a logical error it already made. - High-volume batch jobs where you need strict cost control. A 5-iteration loop on 1,000 inputs could mean 5,000 LLM calls in the worst case. Set max_iterations=2 for batch workloads and accept the quality tradeoff. - Real-time applications. The latency stacks multiplicatively. A 3-second LLM call becomes 15 seconds at max_iterations=5.

The right tool vs. alternatives: - Self-Refine vs. Reflection (Chapter 3.4): Self-refine evaluates the *output*. Reflection evaluates the *reasoning process*. Use self-refine when the output quality is observable; use reflection when the reasoning steps may be subtly wrong even when the output looks correct. - Self-Refine vs. Ensemble Voting (Recipe 20): Ensemble generates N drafts and picks the best. Self-refine generates one draft and improves it. Use self-refine when you have a clear critique standard; use ensemble when you want diversity.

3.0.1.9 Exercises

1. **Tighten the critique prompt.** The default critique function accepts any structured feedback. Modify the CREATE FUNCTION critique() to require a structured output: a rating 1–10 and a bullet-point list of specific issues. Change EVALUATE @feedback to check WHEN > 7 instead of WHEN 'satisfactory'. How does output quality change?

2. **Add a word-count constraint.** The current workflow has no length control. Add a CALL count_words(@current) INTO @word_count step using a Python tool, and add a second EVALUATE that truncates or expands the draft if it falls outside a target range (e.g., 50–80 words). This introduces a hybrid: deterministic length enforcement combined with LLM quality refinement.

3. **Instrument the iteration cost.** Modify the COMMIT to also record an estimated token cost per iteration. Add a Python tool estimate_tokens(text) that returns a character-count approximation. After 10 runs, plot the cost vs. iterations vs. final quality. This gives you the data to set max_iterations empirically rather than arbitrarily.

Chapter 3.1

ReAct Agent

"Don't ask a poet to do your taxes, and don't ask an LLM to do your math."

3.1.1 The Pattern

One of the most common pitfalls in AI development is over-reliance on the model. Because LLMs are so good at talking, we tend to ask them to do everything: search the web, extract data, perform calculations, and write reports. But LLMs are probabilistic—they are "guessing" the next token. When it comes to arithmetic or precise data retrieval, "guessing" is not good enough.

The **ReAct (Reason + Act)** pattern solves this by giving the model tools. The model "reasons" about what it needs to do, "acts" by calling an external tool (like a calculator or a search engine), and then observes the result before moving to the next step.

The SQL analogy is an **External Procedure Call** or a **Trigger**. Your database is great at storage and joins, but if you need to send an email or process a credit card, you don't write that logic in SQL; you call an external service.

The ReAct Agent recipe (Recipe 06) demonstrates this "80/20 Rule": - Use CALL for deterministic operations (math, web search, regex). - Use GENERATE for reasoning (deciding which year to search) and language (writing the final report).

3.1.2 The SPL Approach

This recipe introduces PROCEDURE definitions and the CALL statement. It also shows how to register external Python tools using the @spl_tool decorator.

3.1.3 The .spl File (Annotated)

```
-- Recipe 06: ReAct Agent -- Population Growth
-- Fetch population via WebSearch, compute growth rate via Python tool.

CREATE FUNCTION search_population(country TEXT, year INT) RETURNS TEXT AS $$
Use WebSearch to find the total population of {country} for the year {year}.
Return ONLY the population as a plain integer.
$$;

PROCEDURE search_population(country TEXT, year INT) -- (1) Define a reusable proced
RETURNS TEXT
DO
  GENERATE search_population(country, year) INTO @result
  COMMIT @result
END

WORKFLOW population_growth
  INPUT:
    @country TEXT DEFAULT 'China',
    @year_curr integer default 2023
  OUTPUT: @report TEXT
DO
  @year_prev := @year_curr - 1                -- (2) Arithmetic in SPL

  -- Step 1: Fetch population for each year via WebSearch
  CALL search_population(@country, @year_prev) INTO @pop_prev -- (3) Call procedure
  CALL search_population(@country, @year_curr) INTO @pop_curr

  -- Step 2: Compute growth rate -- routed to Python tool
  CALL calc_growth_rate(@pop_prev, @pop_curr) INTO @growth_rate -- (4) Call Python

  -- Step 3: Compose the final human-readable report
  GENERATE growth_report(...) INTO @report    -- (5) Generate final narrative
  COMMIT @report
END
```

3.1.3.1 (1) `PROCEDURE name(...)`

A PROCEDURE is a named block of SPL logic. Unlike a PROMPT, which is a single model call, a PROCEDURE can contain multiple steps. It's the building block of modular agentic workflows.

SQL Analogy: A **Stored Procedure**. It encapsulates a specific task (searching for population) so the main workflow stays clean.

3.1.3.2 (2) Arithmetic in SPL

Notice @year_prev := @year_curr − 1. SPL handles basic arithmetic and variable assignment directly in the runtime. You don't need an LLM to subtract 1 from a year. This is the "Conductor" doing the simple math so the "Orchestra" can focus on the music.

3.1.3.3 (3) `CALL procedure_name(...)`

The CALL statement executes a procedure or a tool. It is **deterministic**. When you CALL search_population, the runtime executes the steps inside that procedure and returns the result.

3.1.3.4 (4) `CALL calc_growth_rate(...)`

This is where the magic happens. calc_growth_rate is not an SPL procedure; it's a **Python function** decorated with @spl_tool. By using CALL, we are leaving the probabilistic world of the LLM and entering the deterministic world of Python.

The Python tool handles the string parsing (removing commas/spaces) and the floating-point math. The LLM never sees the numbers until the very end.

3.1.3.5 (5) `GENERATE growth_report(...)`

Only after the data has been fetched and the math has been verified do we go back to the LLM. We give the model the raw data and ask it to write a "concise 2-3 sentence report." This is what LLMs are best at: **Language Synthesis**.

3.1.4 Running It

This recipe requires a Python tool file and a web search capability:

```
spl run cookbook/06_react_agent/react_agent.spl \
    --adapter claude_cli \
    --allowed-tools WebSearch \
    --tools cookbook/06_react_agent/tools.py \
    country="France"
```

Expected output:

```
In 2022, France had a population of approximately 67,936,455.
By 2023, this grew to 68,170,228, representing a year-over-year
growth rate of 0.3441%.
```

3.1.5 What Just Happened

LLM calls: 3. (2 for searches, 1 for the final report) **Tool calls: 1.** (Python math tool)

The "Conductor" (SPL Runtime) managed the flow: 1. Calculated 2022 from $2023 - 1$. 2. Delegated the searching to the search_population procedure. 3. Delegated the math to the calc_growth_rate Python tool. 4. Assembled the final evidence and asked the model to write the prose.

This separation of concerns is the secret to building **reliable** AI agents. By offloading the math to Python, you eliminate the risk of "hallucinated" growth rates.

3.1.6 Reproducibility Note

The most variable part of this recipe is the **Web Search**. Different models might navigate to different sources, leading to slightly different population numbers.

However, because the **math** is handled by Python, the *relationship* between the numbers will always be mathematically sound. This is a critical distinction in SPL 2.0: we accept variability in retrieval but demand correctness in calculation.

3.1.7 When to Use This Pattern

Use the **ReAct Agent** pattern when: - **Accuracy is Critical**: Any task involving math, dates, or precise data extraction. - **External Integration**: You need to fetch data from an API, a database, or the web. - **Modular Design**: You want to build a library of reusable procedures (search_population, verify_address, etc.) that can be shared across multiple workflows.

3.1.8 Exercises

1. **Add Error Handling.** Use the EXCEPTION block to handle cases where WebSearch fails to find a number.
2. **Multi-Year Comparison.** Modify the workflow to fetch data for the last *three* years and calculate the average growth rate.
3. **Unit Conversion.** Add a Python tool that converts the population number into "millions" or "billions" before passing it to the final report generator.

Chapter 3.2

Plan and Execute

"A complex task is just a series of simple tasks that haven't been organized yet."

3.2.1 The Pattern

In the previous recipes, we focused on single tasks: generate a greeting, fetch a population, judge a debate. But real-world engineering is more complex. If you ask an LLM to "Build a REST API for a todo app," and it tries to write all the code in one go, it will likely fail. It will hit its output token limit, forget to include imports, or lose consistency between the database model and the API routes.

The **Plan and Execute** pattern is the professional way to handle large tasks. Instead of jumping straight to code, the system follows a structured lifecycle: 1. **Planning**: Decompose the big goal into a numbered list of concrete, independent steps. 2. **Execution**: Iterate through each step, making design decisions and generating code. 3. **Refinement**: If a step fails validation, re-plan the remaining steps based on the new information.

The SQL analogy is a **Long-Running Transaction** or a **Recursive Batch Job**. You don't update a million rows in one statement; you break the work into batches, verify each batch, and log your progress so you can resume if something fails.

The Plan and Execute recipe (Recipe 12) is the most advanced workflow in the basics section. It demonstrates how to use nested WHILE loops and EVALUATE logic to manage a multi-file software project from start to finish.

3.2.2 The SPL Approach

This recipe introduces the "Two-Phase Loop" pattern, which is essential for generating large artifacts without hitting model limits.

3.2.3 The .spl File (Annotated)

```
-- Recipe 12: Plan and Execute
-- A planner decomposes a task, then an executor implements each step.

WORKFLOW plan_and_execute
    INPUT: @task TEXT, @output_dir TEXT, @max_steps INTEGER DEFAULT 5
DO
    -- Phase 1: Planning
    GENERATE plan(@task) INTO @plan              -- (1) Create the blueprint
    GENERATE count_steps(@plan) INTO @step_count

    -- Phase 2: Design Loop
    WHILE @step_index < @step_count DO           -- (2) Iterative Design
        GENERATE extract_step(@plan, @step_index) INTO @current_step
        GENERATE execute_step(@current_step, @results) INTO @step_result

        EVALUATE validate_step(@current_step, @step_result)
            WHEN 'failed' THEN                   -- (3) Automatic Re-planning
                GENERATE replan(@task, @plan, @step_index, @step_result) INTO @plan
                @step_index := 0
            OTHERWISE
                @results := @results + @step_result
                @step_index := @step_index + 1
        END
    END

    -- Phase 3: Implementation Loop
    GENERATE outline_files(@task, @results) INTO @file_outline
    WHILE @file_index < @file_count DO           -- (4) The Implementation Loop
        GENERATE generate_file(...) INTO @file_code
        CALL write_code_files(@file_code, @output_dir) -- (5) Deterministic I/O
        @file_index := @file_index + 1
    END
END
```

3.2.3.1 (1) Phase 1: Planning (The Blueprint)

We start by asking a "Senior Architect" persona to break the task down. The output is a simple numbered list. This list is our "Schedule" for the rest of the workflow.

3.2.3.2 (2) Phase 2: Design Loop (Lightweight Reasoning)

Crucial Lesson: We do *not* generate code inside this first loop. Instead, we generate "Design Notes"—short descriptions of what each step should achieve. - Why? Because LLM context windows are finite. If we generated 500 lines of code in Step 1 and passed it all into Step 2, by Step 5 the model would be "drowned" in its own previous output. - By keeping the loop "lightweight," we ensure the model stays focused on the design intent.

3.2.3.3 (3) Automatic Re-planning

If a step fails (e.g., the model realizes a design choice is impossible), we use the EVALUATE block to trigger a replan. We feed the failure details back into the planner and get a revised list of steps. This is the "Self-Healing" capability of SPL 2.0.

3.2.3.4 (4) Phase 3: Implementation Loop (Heavy Lifting)

Only after the entire design is finalized do we start a second loop for the "Heavy Lifting." In this loop, we generate one file at a time. - One LLM call = One file. - This ensures that each file gets the model's full attention and output token budget.

3.2.3.5 (5) Deterministic I/O (`CALL write_code_files`)

We use a Python tool to write the generated code to disk immediately. We don't wait for the whole workflow to finish. This means that even if the workflow crashes on the last file, your previous work is safe on disk.

3.2.4 Running It

This is a "heavy" recipe. It's best to use a strong model like **qwen2.5-coder** or **Claude 3.5 Sonnet**:

```
spl run cookbook/12_plan_and_execute/plan_execute.spl \
    --adapter ollama -m qwen2.5-coder \
    --tools cookbook/12_plan_and_execute/tools.py \
    task="Build a REST API for a todo app" \
    output_dir="./todo_app"
```

Expected result: A new directory ./todo_app containing main.py, models.py, requirements.txt, and a README.md.

3.2.5 What Just Happened

LLM calls: ~15-20. (Planning, design steps, file generation, summary)

The "Conductor" (SPL Runtime) managed a complex production pipeline: 1. **Architected** the solution. 2. **Validated** each design decision. 3. **Recovered** from mid-process failures via re-planning. 4. **Serialized** the final product into individual files on disk.

3.2.6 Reproducibility Note

On a **GTX 1080 Ti**, this recipe can take **3–5 minutes** to complete. Most of the time is spent in Phase 3 (generating the actual code).

The output is a **First Draft**. While the structure and logic are usually sound, you should always review the generated files for consistency (e.g., ensuring main.py uses the same variable names as models.py).

3.2.7 When to Use This Pattern

Use the **Plan and Execute** pattern when: - **Multi-File Projects**: Generating any application that requires more than one source file. - **Complex Logic**: Tasks where the "how" is not immediately obvious and needs to be decomposed. - **Reliability over Speed**: When you are willing to wait a few minutes for a high-quality, structured result rather than a fast, messy one.

3.2.8 Exercises

1. **Add a "Test" Step.** Modify the workflow to include a step at the end that generates a test_main.py file based on the implementation in main.py.
2. **Global Consistency Check.** Add a step after Phase 2 that asks a "Reviewer" model to look at *all* the design notes at once and check for contradictions before Phase 3 begins.
3. **Language Swap.** Try running the task for different languages (e.g., task="Build a todo app in Go" or task="Write a data processing script in Rust"). Observe how the "Architect" persona adapts the steps.

Chapter 3.3

Reflection Agent

"A genius is someone who makes their mistakes quickly and catches them even faster."

3.3.1 The Pattern

When you ask an LLM a complex question, its first response is often "directionally correct" but flawed in the details. It might overlook a corner case, make a subtle logical error, or provide a generic answer when a specific one was needed. If you simply accept the first output, you are settling for the model's "System 1" thinking—fast, intuitive, but prone to error.

Reflection is the process of forcing the model to look at its own work and find its own mistakes. It is a "System 2" pattern: slow, deliberate, and meta-cognitive. Instead of just answering the problem, the model performs a three-step dance: 1. **Solve**: Produce an initial answer. 2. **Reflect**: Analyze that answer for gaps, errors, or weak reasoning. 3. **Correct**: Revise the answer based on the reflection.

The SQL analogy is a **Self-Correction Loop** or a **Recursive Validation**. You are performing a "sanity check" on a temporary result set before committing it to the final output.

The Reflection Agent recipe (Recipe 16) implements this loop using a WHILE block. It doesn't just refine the text; it calculates a "Confidence Score." If the score is too low, it iterates. If the score is high, it stops. This ensures that the workflow is only as complex as the problem requires.

3.3.2 The SPL Approach

This recipe demonstrates the use of **Meta-Cognitive Branching**—using the model's own self-assessment to decide whether to continue the loop.

3.3.3 The .spl File (Annotated)

```
-- Recipe 16: Reflection Agent
-- Solve a problem, then reflect and correct until confident.

WORKFLOW reflection_agent
    INPUT: @problem TEXT
DO
    @iteration := 0
    @max_reflections := 3
    @confidence := 0

    -- Phase 1: Initial Attempt
    GENERATE solve(@problem) INTO @answer      -- (1) The "System 1" response

    -- Phase 2: The Meta-Cognitive Loop
    WHILE @iteration < @max_reflections DO
        -- Reflect: find errors or gaps
        GENERATE reflect(@problem, @answer) INTO @reflection -- (2) The "Mirror" st

        -- Score confidence (LLM-as-Judge of its own work)
        GENERATE confidence_score(@answer, @reflection) INTO @confidence -- (3) Qu

        EVALUATE @confidence
            WHEN > 0.85 THEN                    -- (4) The Confidence Threshold
                COMMIT @answer WITH status = 'confident'
            OTHERWISE
                -- Extract issues and correct
                GENERATE extract_issues(@reflection) INTO @issues
                GENERATE correct(@answer, @issues, @problem) INTO @answer
                @iteration := @iteration + 1
        END
    END

    -- Final Commit (Best Effort)
    COMMIT @answer WITH status = 'best_effort'
END
```

3.3.3.1 (1) The Initial Solve

The first GENERATE call is the baseline. We ask the model to solve the problem as it normally would. This result is stored in @answer.

3.3.3.2 (2) The "Mirror" Step (`reflect`)

Instead of asking for a better answer immediately, we ask the model to be its own "Critical Editor." - "What did you miss?" - "Are there any logical contradictions?" - "Is the reasoning sound?"

This step produces a "Reflection"—a document that describes what is wrong with the current answer.

3.3.3.3 (3) Quantifying Doubt (`confidence_score`)

We ask the model (or a second, larger "Judge" model) to look at the @answer and the @reflection and produce a numeric score from 0 to 1. This turns a subjective feeling into a hard metric that the WORKFLOW can use for branching.

3.3.3.4 (4) The Confidence Threshold

Using the EVALUATE block, we set a high bar (0.85). - If the model is confident, we COMMIT and exit early. - If not, we use the @reflection to trigger a correct step and try again.

SQL Analogy: **Conditional COMMIT**. You only finalize the transaction if the data passes a series of semantic checks.

3.3.4 Running It

Run the reflection agent on a complex design problem:

```
spl run cookbook/16_reflection/reflection.spl --adapter ollama \
    problem="Design a URL shortener system that handles 1M requests per second"
```

In the execution trace, you will likely see at least one iteration where the model identifies a scaling bottleneck (like a database write limit) and corrects its initial design to include a caching layer or a load balancer.

3.3.5 What Just Happened

LLM calls: 3 + (4 * iterations).

The "Conductor" (SPL Runtime) managed a "Self-Correcting Brain": 1. **Proposed** a solution. 2. **Identified** flaws in that proposal. 3. **Measured** the severity of those flaws. 4. **Revised** the proposal until the measured risk was below the threshold.

This is how you get "Senior Engineer" level output from "Junior" level models. By forcing the model to slow down and check its work, you unlock its full reasoning potential.

3.3.6 Reproducibility Note

Reflection is highly effective but adds **Latency**. On a **GTX 1080 Ti**, a full 3-iteration loop can take **60–120 seconds**.

However, the **Stability** is much higher than a single-pass prompt. Because the model has multiple chances to catch and fix its own errors, the final output is far more likely to be correct and complete across different runs.

3.3.7 When to Use This Pattern

Use the **Reflection Agent** pattern when: - **Accuracy is the Priority**: Technical designs, logical proofs, or complex explanations. - **Variable Problem Complexity**: Some problems are easy and don't need reflection; others are hard and need several passes. The confidence score handles both cases automatically. - **Small Model Boosting**: If you are using a 7B or 14B parameter model, reflection can help it perform at the level of a 70B model.

3.3.8 Exercises

1. **Strict Judge.** Use the USING MODEL clause (Chapter 9.1) to ensure the confidence_score step always uses your strongest model (e.g., gemma3), even if phi4 is doing the reflection.
2. **Add a "Divergent" step.** In the reflection pass, ask the model to specifically "Think about an alternative perspective" to prevent it from just reinforcing its own biases.
3. **Manual Stop.** Add a step inside the loop that asks for human feedback before the next reflection pass begins.

Chapter 3.4

Multi-Model Pipeline

"Choose the right instrument for each movement of the symphony."

3.4.1 The Pattern

In a world of thousands of models, the most common mistake is using a "Titan" (like GPT-4o or Claude 3.5 Sonnet) for every single step. While these models are brilliant, they are also slower and more expensive. For many sub-tasks—like basic research, simple summarization, or formatting—a "Junior" model (like Phi-4 or Llama 3.2) can do the job just as well, at a fraction of the cost.

The **Multi-Model Pipeline** pattern is the practice of routing different steps of a workflow to different models based on the task's requirements. - Use a **Fast Model** for data retrieval. - Use a **Reasoning Model** for deep analysis. - Use a **Creative Model** for final prose.

The SQL analogy is **Polyglot Persistence** or **Federated Queries**. You don't store everything in one massive database. You put your structured data in PostgreSQL, your logs in Elasticsearch, and your cache in Redis. When you run a "query" (the workflow), the system fetches data from the most efficient source for that specific piece of information.

The Multi-Model Pipeline recipe (Recipe 21) demonstrates this "Best-of-Breed" orchestration. It routes research, analysis, and writing to the models most suited for those roles, then wraps them in a quality-control loop.

3.4.2 The SPL Approach

This recipe introduces the USING MODEL clause at the step level, allowing for granular control over the "orchestra."

3.4.3 The .spl File (Annotated)

```
-- Recipe 21: Multi-Model Pipeline
-- Each step targets the model best suited for that task.

WORKFLOW multi_model_pipeline
  INPUT: @topic TEXT
DO
  -- Step 1: Research (Needs factual retrieval)
  GENERATE research(@topic)
    USING MODEL 'gemma3' INTO @facts;          -- (1) Fast, retrieval-tuned model

  -- Step 2: Analysis (Needs deep reasoning)
  GENERATE analyze(@facts)
    USING MODEL 'claude-3-5-sonnet' INTO @analysis; -- (2) High-intelligence model

  -- Step 3: Writing (Needs creative style)
  GENERATE write_summary(@analysis)
    USING MODEL 'llama3.2' INTO @draft;        -- (3) Creative, lightweight model

  -- Step 4: Quality loop
  WHILE @iteration < 3 DO
    GENERATE quality_check(@draft)
      USING MODEL 'gemma3' INTO @quality;      -- (4) Neutral judge model

    EVALUATE @quality
      WHEN > 0.7 THEN COMMIT @draft
      OTHERWISE
        GENERATE write_summary(@analysis) INTO @draft
        @iteration := @iteration + 1
    END
  END
END
```

3.4.3.1 (1) Step 1: Fast Retrieval

For the research phase, we want speed and factual breadth. We target **gemma3** (or a similar local model). This model is excellent at scanning its training data and producing a bulleted list of facts without the latency of a massive cloud model.

3.4.3.2 (2) Step 2: Deep Reasoning

For the analysis, we need the "Gold Standard." We use USING MODEL 'claude-3-5-sonnet' (via the OpenRouter or Claude adapter). This model's ability to identify subtle trends and risks is superior to smaller models. We are spending our "token budget" where it matters most.

3.4.3.3 (3) Step 3: Creative Writing

For the final summary, we might want a specific tone. **Llama 3.2** is known for being conversational and engaging. By switching to a smaller model here, we get the response faster and often with less

"corporate" phrasing than the larger frontier models.

3.4.3.4 (4) Step 4: The Neutral Judge

Finally, we use a different model (**gemma3** again) to perform the quality check. Using a different model for the judge step prevents "Confirmation Bias"—where a model is too lenient on its own work.

SQL Analogy: **Cross-Engine Join**. You are joining the output of Model A with the judgment of Model B to produce a refined final result.

3.4.4 Running It

This recipe highlights the power of adapters. Ensure you have your keys configured for cloud models if you choose to use them:

```
spl run cookbook/21_multi_model_pipeline/multi_model.spl --adapter openrouter \
    topic="The impact of AI on rural education"
```

In the execution trace, you will see the runtime switching between adapters and model names for each GENERATE step.

3.4.5 What Just Happened

LLM calls: 4 + (2 * iterations).

The "Conductor" (SPL Runtime) orchestrated a specialized team: 1. **Commissioned** research from a fast specialist. 2. **Requested** analysis from a high-intelligence consultant. 3. **Hired** a creative writer for the final draft. 4. **Audited** the work using an independent third party.

This is the peak of agentic design: not just using AI, but using the *right* AI for every single turn.

3.4.6 Reproducibility Note

The Multi-Model pattern is the most **Cost-Efficient** way to achieve high quality. By routing 80% of the tokens to smaller, cheaper models and reserving the remaining 20% for the "frontier" models, you can achieve "GPT-4 level" results at "GPT-3.5 level" costs.

On a **GTX 1080 Ti**, the local steps (Research, Judge) run in **2–5 seconds**, while the cloud steps (Analysis) depend on your internet connection and the provider's latency.

3.4.7 When to Use This Pattern

Use the **Multi-Model Pipeline** pattern when: - **Cost/Performance Balance**: You need high quality but want to keep API costs low. - **Latency Optimization**: You want the user to see the "Researching…" phase happen instantly while the deep thinking happens in the background. - **Ensemble Benchmarking**: When you want to see how different models "view" each other's work.

3.4.8 Exercises

1. **Local-Only Mode.** Modify the recipe to use only local models (e.g., phi4 for research, gemma3 for analysis, mistral for writing). Compare the quality to the cloud-mixed version.
2. **Add a "Model Switcher."** (Advanced) Use an EVALUATE block to check the complexity of the @topic. If it's "simple," use a fast model for all steps. If it's "complex," switch to the full multi-model pipeline.
3. **Parallel Multi-Model.** Use a WITH clause to ask two different models for the Research phase and then have the Analyst merge their findings.

Chapter 3.5

Nested Procedures

"Complexity should be composed, not managed."

3.5.1 The Pattern

As your AI applications grow, your .spl files can become long and difficult to read. If you have a workflow that researches a topic, explains it, generates examples, and checks reading level, putting all those steps in one giant WORKFLOW block is like writing a 1,000-line main() function in C. It's hard to test, hard to reuse, and hard to debug.

The **Nested Procedures** pattern is the core of modular SPL development. It allows you to break a complex task into small, independent PROCEDURE units that can call each other. This is the foundation of **Deep Composability**.

The SQL analogy is **Nested Views** or **Modular Stored Procedures**. You don't write one massive query to generate a financial report. You write a view for v_revenue, another for v_expenses, and then a third view that joins them. Each piece is independently verified and perfectly encapsulated.

The Nested Procedures recipe (Recipe 25) demonstrates this hierarchy. It builds a "Layered Explainer" where high-level orchestration is separated from low-level generation tasks like complexity calibration and example making.

3.5.2 The SPL Approach

This recipe introduces the pattern of **Procedural Delegation**—passing control from a high-level workflow to specialized sub-procedures.

3.5.3 The .spl File (Annotated)

```
-- Recipe 25: Nested Procedures
-- Demonstrates PROCEDURE calling PROCEDURE — deep composability.

PROCEDURE explain_layer(content TEXT, audience TEXT) -- (1) Leaf Procedure
RETURNS TEXT
DO
    GENERATE explain(content, audience) INTO @explanation
    COMMIT @explanation
END

PROCEDURE calibrate_complexity(text TEXT, audience TEXT) -- (2) Intermediate Proce
RETURNS TEXT
DO
    GENERATE assess_reading_level(text) INTO @level
    EVALUATE @level
        WHEN > 8 THEN
            GENERATE simplify(text, audience) INTO @calibrated
            COMMIT @calibrated
        OTHERWISE
            COMMIT text
    END
END

WORKFLOW layered_explainer                          -- (3) Root Workflow
    INPUT: @topic TEXT, @audience TEXT
DO
    GENERATE research_overview(@topic) INTO @overview

    -- Calling the specialized procedures
    CALL explain_layer(@overview, @audience) INTO @base_exp -- (4) Delegation

    CALL calibrate_complexity(@base_exp, @audience) INTO @final_exp

    COMMIT @final_exp WITH status = 'complete'
END
```

3.5.3.1 (1) The "Leaf" Procedure

explain_layer is a simple wrapper around a GENERATE call. Why bother making it a procedure? Because it encapsulates the "Explanation Persona." If you want to change how the model explains things (e.g., adding "avoid jargon" to the system prompt), you change it in this one procedure, and every workflow that uses it is automatically updated.

3.5.3.2 (2) The "Intermediate" Procedure

calibrate_complexity is more complex. It contains its own EVALUATE logic and potentially its own nested calls. By making this a procedure, we can test the "Simplification Logic" in isolation without running the whole research workflow.

3.5.3.3 (3) The Root Workflow

The layered_explainer acts as the "Conductor of Conductors." It doesn't worry about how to simplify text or how to explain a concept. It only worries about the high-level sequence: **Research → Explain → Calibrate**.

SQL Analogy: **Orchestration Query**. You are joining the outputs of multiple modular functions to produce a high-level report.

3.5.3.4 (4) Delegation (`CALL`)

The CALL statement is the bridge. In SPL 2.0, CALL is used for both Python tools (Recipe 06) and other SPL procedures. This unified interface means you can swap a generative procedure for a deterministic tool without changing the calling code.

3.5.4 Running It

Run the layered explainer for different audiences:

```
# To high schoolers
spl run cookbook/25_nested_procs/nested_procs.spl --adapter ollama \
    topic="quantum computing" audience="high school students"

# To policy makers
spl run cookbook/25_nested_procs/nested_procs.spl --adapter ollama \
    topic="CRISPR gene editing" audience="policy makers"
```

In the execution trace, you will see the "Call Stack" growing and shrinking as the runtime enters and exits each procedure.

3.5.5 What Just Happened

LLM calls: ~5-7. (Depending on the complexity of the research and calibration steps)

The "Conductor" (SPL Runtime) managed a "Recursive Hierarchy": 1. **Orchestrated** the high-level business logic. 2. **Delegated** the domain-specific tasks to specialized "Agents" (Procedures). 3. **Maintained** state across the hierarchy (passing @base_exp from one procedure to another). 4. **Enforced** encapsulation (the root workflow never saw the intermediate @level variable).

3.5.6 Reproducibility Note

Nested procedures are the key to **Scalable Maintenance**. When you find a bug in your "Simplification" logic, you fix it in calibrate_complexity.spl. You don't have to hunt through 20 different workflow files to find every place you used a similar prompt.

On a **GTX 1080 Ti**, the overhead of the "Procedure Call" is **zero**. The runtime treats nested procedures as logical blocks within the same execution context. The only latency comes from the underlying LLM calls.

3.5.7 When to Use This Pattern

Use the **Nested Procedures** pattern when: - **Large Projects**: Any .spl file that exceeds 100 lines should probably be broken into procedures. - **Shared Logic**: When you have a task (like "PII Redaction" or "Tone Check") that is used by multiple different workflows. - **Unit Testing**: When you want to verify that a specific part of your logic works correctly before integrating it into a complex agent.

3.5.8 Exercises

1. **Add a "Quiz" Procedure.** Create a third procedure called make_quiz that takes the @final_exp and generates 3 multiple-choice questions. Call it at the end of the root workflow.
2. **Shared Library.** Move explain_layer into a separate file (library.spl) and research how to use the IMPORT or ——include flags to use it in your main workflow.
3. **Conditional Nesting.** Add a parameter @skip_calibration to the root workflow. Use an EVALUATE block to only CALL calibrate_complexity if the parameter is false.

Part 4

Reasoning

4.0.1 Chain of Thought

4.0.1.1 The Pattern

You have a topic — "distributed AI inference," say — and you want not just a surface summary but a piece of writing that demonstrates actual reasoning. The naive Python approach is a single requests.post to the LLM with an instruction like "think carefully and then summarise." Sometimes it works. More often you get a paragraph that opens confidently, wanders, and concludes with something vague. The model has to do the research thinking, the analytical thinking, and the synthesis all in one forward pass, which is a lot to ask. Debugging it is worse: when the output is thin, you cannot tell whether the model failed to surface facts, failed to analyse them, or failed to compress the analysis. The whole thing is a black box.

The sharper practitioners will try prompt engineering — "first list what you know, then analyse, then summarise" — inside a single GENERATE call. This helps, but you still cannot inspect intermediate results, you cannot swap a better research prompt later without re-testing the whole pipeline, and you lose the ability to halt early if step one produces garbage. You are also paying for tokens in steps two and three that are built on a shaky foundation you cannot see.

The recurring pain is that reasoning is not a single act. It is a sequence of acts, each contingent on the one before. Compressing that sequence into one LLM call trades controllability for convenience, and in practice the convenience is illusory — you end up debugging by staring at final output and guessing.

4.0.1.2 The SPL Approach

Model the reasoning stages explicitly as a linear chain of GENERATE steps, threading each output as context for the next, so that Research feeds Analysis and Analysis feeds Synthesis — a CTE chain written for thought instead of tables.

4.0.1.3 The .spl File (Annotated)

```
-- Recipe 09: Chain of Thought
-- Multi-step reasoning: Research → Analyze → Summarize
-- Each GENERATE feeds into the next via workflow variables.
--
-- Usage:
--   spl run cookbook/09_chain_of_thought/chain.spl \
--       --adapter ollama -m gemma3 \
--       topic="distributed AI inference"

WORKFLOW chain_of_thought
    INPUT: @topic TEXT          -- The subject to reason about; caller decides, not
    OUTPUT: @summary TEXT       -- Final synthesised output; the only thing the call
DO
    -- Step 1: Research
    -- The model's job here is narrow: surface what is known about @topic.
    -- Analogous to: WITH research AS (SELECT facts FROM knowledge WHERE subject =
    GENERATE research(@topic) INTO @research
```

```
    -- Step 2: Analyze
    -- @research is now in scope. The model cannot "forget" what was surfaced;
    -- it must reason from it. Narrow job: find patterns, tensions, implications.
    -- Analogous to: WITH analysis AS (SELECT interpret(facts) FROM research)
    GENERATE analyze(@research) INTO @analysis

    -- Step 3: Summarize
    -- The model works from @analysis, not from @topic directly.
    -- This is the key difference from a single-prompt approach: the summary
    -- is grounded in the analysis, which is grounded in the research.
    -- Analogous to: SELECT synthesize(interpretation) FROM analysis
    GENERATE summarize(@analysis) INTO @summary

    -- Commit the final variable. status metadata travels with the result.
    COMMIT @summary WITH status = 'complete'
END
```

On the SQL analogy. In SQL you write:

```
WITH research  AS (SELECT facts         FROM knowledge WHERE subject = :topic),
     analysis  AS (SELECT interpret(f) FROM research),
     summary   AS (SELECT compress(i)  FROM analysis)
SELECT * FROM summary;
```

Each CTE can only see what the previous one produced. There is no way for summary to reach back past analysis and pull raw facts — the chain forces discipline. The SPL chain works identically: @summary is computed from @analysis, which was computed from @research. You cannot accidentally skip a step.

Why three variables instead of one? Because each is individually inspectable. After a run you can log @research, @analysis, and @summary separately. If the summary is thin, you check the analysis first. If the analysis is thin, you check the research. The isolation of failure modes is the point.

On INPUT design. @topic is the only input. The workflow does not hard-code "distributed AI inference" or set a default — the caller decides what to reason about. This respects a core SPL principle: a workflow specifies the shape of the computation, not the subject matter.

On GENERATE prompt resolution. Each GENERATE call (research, analyze, summarize) corresponds to a prompt template that the runtime resolves. In the absence of explicit CREATE FUNCTION definitions in this file, the runtime uses the step name and arguments to construct the system and user prompts. The practical implication: you can override any one prompt — say, swapping analyze for a domain-specific analysis prompt — without touching the others.

On COMMIT. COMMIT @summary WITH status = 'complete' writes the result to the output store with a status label. The workflow has no EXCEPTION block. This is deliberate for a recipe at this stage of complexity: the chain of thought pattern is the mechanism; safety wrapping is a separate concern handled in Part 4.

4.0.1.4 Running It

```
spl run cookbook/09_chain_of_thought/chain.spl \
    --adapter ollama -m gemma3 \
    topic="distributed AI inference"
```

Expected output (truncated — your model's phrasing will differ):

```
[chain_of_thought] Running on topic: "distributed AI inference"

[research] Generating...
  Distributed AI inference refers to the practice of spreading model
  execution across multiple nodes or devices rather than centralising
  computation on a single server. Key considerations include network
  latency between nodes, model partitioning strategies (tensor parallel,
  pipeline parallel), memory bandwidth constraints at edge devices...
  [~340 tokens]

[analyze] Generating...
  Three tensions emerge from the research. First, latency vs. throughput:
  distributing a single request across nodes adds coordination overhead
  that can exceed the gains from parallelism unless batch sizes are large.
  Second, model size vs. edge capability: the economics favour distribution
  only when individual nodes cannot host the full model...
  [~280 tokens]

[summarize] Generating...
  Distributed AI inference becomes economically and technically compelling
  when model size exceeds single-node memory, batching is feasible, and
  network topology is controlled. For latency-sensitive single-request
  workloads, a well-provisioned central server typically wins...
  [~180 tokens]

status: complete
```

4.0.1.5 What Just Happened

Three LLM calls were made, in strict sequence. No LLM call was made for anything other than reasoning — there is no CALL in this workflow, because there is no deterministic tool work to do.

Step 1 (research): The model received @topic and was asked to surface relevant knowledge. The output went into @research — a variable that exists only inside this workflow run.

Step 2 (analyze): The model received @research. It had no access to @topic directly (unless the prompt template passes it through, which in this recipe it does not). The model's task was narrowed to interpretation of the material already surfaced.

Step 3 (summarize): The model received @analysis. Its task was compression and synthesis. By this point, the chain has done the work of separating "what do we know" from "what does it mean" from "what is the bottom line" — three jobs that, when conflated, produce the bland summary we were trying to avoid.

COMMIT: Writes @summary to the output store with status = 'complete'. The caller receives @summary as the workflow's declared OUTPUT.

Token budget awareness. Three GENERATE calls means three round-trips to the model. For a local model on a GTX 1080 Ti this is noticeable — roughly 12–20 seconds total depending on output length. For a production API it is three billing events. This is the cost of the pattern. It is worth it when the quality gain matters; for a quick lookup it is not.

4.0.1.6 Reproducibility Note

Tested on a GTX 1080 Ti with gemma3 (4-bit quantised) via Ollama. End-to-end latency for topic="distributed AI inference": approximately 14 seconds. For topic="quantum computing" with llama3.2: approximately 18 seconds.

Output stability: the structure of the chain (research → analysis → summary) is stable across runs. The specific phrasing of each step varies, as expected from generative models. The summary at step three tends to be more focused and grounded than a single-prompt alternative because it inherits the structure imposed by step two.

If you need bit-for-bit reproducibility, set temperature=0 on all three GENERATE calls. You will lose some expressiveness but gain determinism within a given model version. Note that model updates will still change output even at temperature zero.

One observed failure mode: if @topic is extremely broad ("everything about AI"), the research step produces a sprawling dump that overwhelms the analysis step, which then produces a similarly sprawling analysis, and the summary step cannot compress it coherently. Keep topics scoped. The workflow does not enforce this — that is the caller's responsibility.

4.0.1.7 When to Use This Pattern

Use chain of thought when:

- The task has identifiable reasoning stages that benefit from isolation (research / analysis / synthesis; hypothesis / evidence / conclusion; diagnosis / differential / recommendation).
- You want to inspect intermediate reasoning, not just final output.
- You expect to iterate on individual stages — for example, replacing the analysis prompt with a domain-specific one — without re-engineering the whole pipeline.
- Output quality is more important than token economy.

Do not use chain of thought when:

- A single, well-crafted prompt reliably produces the quality you need. Complexity for its own sake is expensive.
- Latency is the binding constraint. Three sequential LLM calls compound latency. If you need a fast answer and quality is acceptable from one pass, use one pass.
- The stages are not genuinely sequential. If step two does not depend on step one's output, they should run in parallel (see Chapter 4.2, Tree of Thought, which explores parallel path generation before convergence).

Comparison with alternatives:

Approach	Quality	Latency	Debuggability
Single GENERATE	Variable	Low	Poor
Chain of Thought (this recipe)	Higher	Medium	Good
Tree of Thought (Recipe 17)	Highest	High	Good

The chain of thought pattern is the minimum viable structure for tasks where reasoning quality matters. Tree of thought adds parallel exploration when you are uncertain which reasoning path is best. Chain of thought assumes you know the right sequence of reasoning acts; tree of thought does not.

4.0.1.8 Exercises

1. **Add a fourth step.** After summarize, add GENERATE critique(@summary, @topic) INTO @critique that asks the model to identify the weakest claim in the summary. Then COMMIT @critique WITH status = Run both the original and the extended version; compare whether the critique surfaces anything the single-prompt approach would have missed.

2. **Swap the analysis prompt.** The analyze step uses the default prompt template. Create a CREATE FUNCTION analysis_prompt() RETURNS TEXT that specialises the analysis for a domain of your choice (security, economics, biology). Pass it to GENERATE analyze(@research, analysis_prompt()) INTO @analysis. Observe how specialising one stage changes the downstream summary without touching any other step.

3. **Fail gracefully.** Wrap the workflow in an EXCEPTION block. Add WHEN GenerationError THEN COMM — so that if the analysis or summary step fails, the caller still receives the research output rather than nothing. This is the starting point for understanding the safety patterns in Part 4.

Chapter 4.1

Tree of Thought

"A single path is a tunnel; multiple paths are a map."

4.1.1 The Pattern

When humans solve a truly hard problem, we don't just pick one idea and run with it. We brainstorm. We explore three or four different approaches, follow each one a few steps down the road, see which one runs into a dead end, and then double down on the most promising path. This is the **Tree of Thought (ToT)** pattern.

The problem with standard LLM prompting is that it is "linear." The model commits to a direction with its very first token and is then forced to stay on that path, even if it realizes halfway through that it was a mistake.

The SQL analogy is a **Recursive CTE with Branching** or a **Multi-Path Union**. You are generating multiple potential "result sets" (thought paths) in parallel and then using a filter (an evaluation step) to pick the best one.

The Tree of Thought recipe (Recipe 17) implements this "Branch and Prune" strategy. It explores three independent approaches to a problem, develops each one into a more detailed plan, scores them, and then selects the winner for final refinement.

4.1.2 The SPL Approach

This recipe demonstrates **Parallel Exploration**—generating multiple independent states and then synthesizing them into a single conclusion.

4.1.3 The .spl File (Annotated)

```
-- Recipe 17: Tree of Thought
-- Explores multiple reasoning paths in parallel, then selects the best.

WORKFLOW tree_of_thought
    INPUT: @problem TEXT
DO
    -- Phase 1: Branching (The Brainstorm)
    GENERATE approach_1(@problem) INTO @path_a -- (1) Three independent seeds
    GENERATE approach_2(@problem) INTO @path_b
    GENERATE approach_3(@problem) INTO @path_c

    -- Phase 2: Developing (Following the map)
    GENERATE develop(@path_a, @problem) INTO @path_a_dev -- (2) Elaborating each
    GENERATE develop(@path_b, @problem) INTO @path_b_dev
    GENERATE develop(@path_c, @problem) INTO @path_c_dev

    -- Phase 3: Scoring (The Pruning)
    GENERATE evaluate_path(@path_a_dev) INTO @score_a -- (3) Objective evaluation
    GENERATE evaluate_path(@path_b_dev) INTO @score_b
    GENERATE evaluate_path(@path_c_dev) INTO @score_c

    -- Phase 4: Selection and Refinement
    GENERATE select_best(@path_a_dev, @score_a, ...) INTO @best_path
    GENERATE refine_solution(@best_path) INTO @best_solution

    -- Phase 5: Final Verification
    GENERATE verify(@best_solution) INTO @verification
    EVALUATE @verification                      -- (4) The Final Check
        WHEN 'sound' THEN
            COMMIT @best_solution WITH status = 'complete'
        OTHERWISE
            GENERATE synthesize_all(...) INTO @best_solution
            COMMIT @best_solution WITH status = 'synthesized'
    END
END
```

4.1.3.1 (1) Phase 1: Branching

We ask the model to generate three distinct approaches to the same problem. For example, if the problem is "How to scale a database," Path A might be "Horizontal Sharding," Path B might be "Read Replicas," and Path C might be "Caching Layer."

4.1.3.2 (2) Phase 2: Developing

We take each seed and ask the model to "develop" it further—writing out the pros, cons, and implementation steps for each specific approach. This is where the "tree" grows its branches.

4.1.3.3 (3) Phase 3: Scoring

We use a "Judge" prompt to score each developed path on a set of criteria (e.g., feasibility, cost, scalability). This is the "Pruning" step. We are identifying which branches are worth keeping and which are dead ends.

4.1.3.4 (4) The Final Check

Even after we've picked a "winner" and refined it, we perform one last verification. If the final solution is still not "sound," we fall back to a **Synthesis** step—taking the best ideas from *all three* paths and merging them. This ensures we don't lose the "good parts" of the losing paths.

4.1.4 Running It

Run the Tree of Thought on a strategic decision:

```
spl run cookbook/17_tree_of_thought/tree_of_thought.spl --adapter ollama \
    problem="Should we rewrite our legacy monolith in Rust or Go?"
```

In the output, you will see the model exploring both languages (and perhaps a third option like "Refactor in place"), scoring them based on performance and developer velocity, and then providing a nuanced recommendation.

4.1.5 What Just Happened

LLM calls: 12-14.

The "Conductor" (SPL Runtime) managed a "Tournament of Ideas": 1. **Fanned out** the initial brainstorming. 2. **Expanded** each idea into a concrete plan. 3. **Measured** each plan against a set of standards. 4. **Selected** the strongest candidate. 5. **Refined** the candidate into a production-ready solution. 6. **Verified** the result, with a fallback for complex failures.

4.1.6 Reproducibility Note

Tree of Thought is the most computationally expensive reasoning pattern. On a **GTX 1080 Ti**, a full 12-call ToT can take **2–4 minutes**.

However, the **Robustness** is unparalleled. By exploring multiple paths, you significantly reduce the risk of the model "hallucinating" a single bad solution and sticking to it. This pattern is designed for high-stakes decision-making where the cost of a wrong answer is much higher than the cost of the extra tokens.

4.1.7 When to Use This Pattern

Use the **Tree of Thought** pattern when: - **Strategic Decisions**: Architectural choices, business strategies, or complex product trade-offs. - **Creative Brainstorming**: When you need a wide variety of high-quality ideas rather than just one. - **Problem Solving with High Uncertainty**: When the "right" answer is not obvious and needs to be discovered through exploration.

4.1.8 Exercises

1. **Parallelize the Branches.** Use the WITH clause (Chapter 9.1) to run Phase 1, 2, and 3 in parallel blocks. This will reduce the latency by nearly 60% if your hardware can handle it.
2. **Add a "Devil's Advocate."** In Phase 2, instead of just "developing" the path, ask the model to specifically "Find the three biggest reasons why this approach will fail."
3. **Variable Branching.** Modify the input to accept @branch_count and use a WHILE loop (Chapter 7.1) to generate an arbitrary number of paths instead of exactly three.

Chapter 4.2

Hypothesis Tester

"The first principle is that you must not fool yourself — and you are the easiest person to fool."

4.2.1 The Pattern

In business and science, we often make quick observations: "Users hate the new UI," or "The server is slow because of the database." These are not facts; they are **Hypotheses**. If you act on a hypothesis without testing it, you waste time and resources.

Structured Scientific Reasoning is the pattern of converting a "vibe" into a falsifiable claim. 1. **Formulate**: Define the Null Hypothesis (H0: it's chance) and the Alternative Hypothesis (H1: the claim). 2. **Plan**: Design a test that could prove you wrong (Falsifiability). 3. **Audit**: Look for confounding variables (other things that could cause the effect). 4. **Evaluate**: Score the available evidence and draw a conclusion based on a confidence threshold.

The SQL analogy is **Hypothetical Querying** or **Impact Analysis**. You are running a "What-If" scenario against your data, calculating the "Confidence Interval" of your findings, and only committing to a business decision if the results are statistically significant.

The Hypothesis Tester recipe (Recipe 35) implements this rigorous loop. It forces the model to think like a scientist—identifying variables and seeking evidence that *refutes* its own initial guess before reaching a final verdict.

4.2.2 The SPL Approach

This recipe introduces **Evidence Weighting**—using a structured JSON schema to collect pros and cons and then using a numeric confidence score to drive the workflow's conclusion.

4.2.3 The .spl File (Annotated)

```
-- Recipe 35: Hypothesis Tester
-- Generate hypothesis → design test → evaluate evidence → conclude.

CREATE FUNCTION hypothesis_framework()
RETURNS TEXT AS $$
Format:
- Null hypothesis (H0): The effect does not exist.
- Alternative hypothesis (H1): The proposed claim.
- Falsifiability: How would we prove H1 wrong?
$$;

WORKFLOW hypothesis_tester
    INPUT: @observation TEXT, @threshold FLOAT DEFAULT 0.7
DO
    -- Phase 1: Formulation
    GENERATE formulate_hypotheses(@observation, hypothesis_framework()) INTO @hypo

    -- Phase 2: Experimental Design
    GENERATE design_test(@hypo) INTO @test_plan

    -- Phase 3: Evidence Evaluation
    GENERATE evaluate_evidence(@observation, @hypo, evidence_schema()) INTO @evider

    -- Phase 4: Decision Logic
    GENERATE extract_confidence(@evidence_json) INTO @confidence

    EVALUATE @confidence
        WHEN >= @threshold THEN                                  -- (1) Significant R
            GENERATE write_conclusion(@hypo, @evidence_json, 'confident') INTO @fir
            COMMIT @final WITH verdict = 'supported'
        WHEN >= 0.4 THEN                                         -- (2) Inconclusive
            GENERATE write_conclusion(@hypo, @evidence_json, 'uncertain') INTO @fir
            COMMIT @final WITH verdict = 'needs_more_data'
        OTHERWISE                                                -- (3) Refuted
            GENERATE write_conclusion(@hypo, @evidence_json, 'refuted') INTO @final
            COMMIT @final WITH verdict = 'rejected'
    END
END
```

4.2.3.1 (1), (2) & (3) The Confidence Ladder

This is the most "Honest" pattern in the cookbook. Instead of always giving an answer, the workflow can admit ignorance. - If the evidence is strong (**@confidence >= 0.7**), it commits a confident conclusion. - If the evidence is mixed (**@confidence >= 0.4**), it tells the user "We need more data." - If the evidence is weak, it concludes that the observation was likely a fluke.

SQL Analogy: **Threshold Filtering**. You are filtering your "Insights" by a quality metric, ensuring that only high-confidence conclusions are promoted to the final report.

4.2.4 Running It

Test a common business observation:

```
spl run cookbook/35_hypothesis_tester/hypothesis.spl \
    --adapter ollama -m gemma3 \
    observation="Remote teams show lower productivity in the first month after join
```

Expected output: A structured breakdown of the variables (e.g., "Time-to-onboard" vs "Work location"), a list of evidence for and against the claim, and a final verdict based on the calculated confidence.

4.2.5 What Just Happened

LLM calls: 5. (Formulate, Design, Evaluate, Extract, Conclude)

The "Conductor" (SPL Runtime) managed a "Peer Review Board": 1. **Drafted** a formal scientific claim. 2. **Designed** an experiment to test that claim. 3. **Audited** the claim against existing evidence. 4. **Measured** the strength of the findings. 5. **Rendered** a conservative, evidence-based judgment.

4.2.6 Reproducibility Note

Hypothesis Testing is a "Deep Reasoning" task. Small models often fall into "Confirmation Bias"—they only look for evidence that supports the observation. For this recipe, we strongly recommend using a "Senior" reasoning model like **gemma3 (27B)** or **Llama 3.1 (70B)** to ensure the adversarial thinking is genuine.

On a **GTX 1080 Ti**, this full reasoning chain takes **40–60 seconds**.

4.2.7 When to Use This Pattern

Use the **Hypothesis Tester** pattern when: - **Strategic Planning**: Testing "Gut Feelings" before making a pivot. - **Root Cause Analysis**: Investigating why a system failed or why a metric dropped. - **Academic/Scientific Research**: Automating the first pass of literature review and hypothesis formulation.

4.2.8 Exercises

1. **Add "Confounding Variables".** Add a specific step between Phase 1 and 2 that asks the model to "List three other things that could explain this observation besides H1."
2. **Multi-Model Review.** Use a different model for the evaluate_evidence step to provide a "blind" second opinion.
3. **Custom Threshold.** Run the recipe with @threshold = 0.9 and observe how much harder it is for the model to reach a "Confident" conclusion.

Part 5

Safety & Reliability

Chapter 5.1

Safe Generation

"Trust, but verify. Then verify again."

5.1.1 The Pattern

In the early days of LLMs, we lived in a "Fire and Forget" world. You sent a prompt, got a response, and hoped for the best. If the model hallucinated, refused to answer, or produced low-quality prose, your application simply passed that failure onto the user.

In a production environment, this is unacceptable. You need **Guardrails**. You need a way to catch errors *before* they reach the user and a way to handle the unique "exceptions" that only happen in generative AI—like context window overflows or safety refusals.

The SQL analogy is a **CONSTRAINT** or a **CHECK** clause. You don't just insert data into a table; you ensure it meets certain criteria (not null, unique, within range). If the criteria aren't met, the transaction fails or triggers a correction.

The Safe Generation recipe (Recipe 07) implements this as a "Self-Correction Loop." It generates a response, uses an "LLM-as-Judge" to evaluate the quality, and then branches to different outcomes based on the verdict. It also demonstrates SPL 2.0's advanced EXCEPTION handling for generative risks.

5.1.2 The SPL Approach

This recipe introduces the EVALUATE block for semantic branching and the EXCEPTION block for handling non-deterministic errors.

5.1.3 The .spl File (Annotated)

```
-- Recipe 07: Safe Generation with Exception Handling
-- Demonstrates LLM-specific error handling patterns

WORKFLOW safe_generation
  INPUT: @prompt text
  OUTPUT: @result text
DO
  GENERATE response(@prompt) INTO @result      -- (1) Initial generation

  -- Quality check via semantic evaluation (LLM-as-judge)
  GENERATE quality_assess(@result) INTO @quality -- (2) The "Judge" step

  EVALUATE @quality                            -- (3) Semantic Branching
    WHEN 'high_quality' THEN
      COMMIT @result WITH status = 'high_quality'
    WHEN 'acceptable' THEN                     -- (4) Refinement loop
      GENERATE improved(@result, @prompt) INTO @result
      COMMIT @result WITH status = 'refined'
    OTHERWISE
      COMMIT @result WITH status = 'best_effort'
  END

EXCEPTION                                      -- (5) Generative Exceptions
  WHEN HallucinationDetected THEN
    GENERATE response(@prompt) INTO @result    -- (6) Strategic recovery
    COMMIT @result WITH status = 'conservative'

  WHEN ContextLengthExceeded THEN
    RETRY WITH temperature = 0.1               -- (7) Automated retry

  WHEN RefusalToAnswer THEN
    COMMIT @prompt WITH status = 'refused'
END
```

5.1.3.1 (1) & (2) Generation and Assessment

We start with a standard GENERATE call. But instead of returning the result immediately, we pass it to a second GENERATE step called quality_assess. This is the "LLM-as-Judge" pattern: using a model (often a larger, more capable one) to grade the work of another model.

5.1.3.2 (3) `EVALUATE` (Semantic Branching)

In traditional programming, you branch on booleans or integers (if $x > 5$). In SPL, you branch on **Semantic Labels**. The EVALUATE block looks at the string returned by @quality and matches it against your WHEN clauses.

SQL Analogy: A **CASE** statement. It maps inputs to specific outputs or actions based on a set of conditions.

5.1.3.3 (4) Refinement

If the judge returns 'acceptable', we don't give up. We trigger a third generation—improved(@result, @prompt)—asking the model to fix the specific issues identified. This "Multi-Pass" approach significantly increases the reliability of the final output.

5.1.3.4 (5) `EXCEPTION`

This is where SPL 2.0 handles the "known unknowns" of AI. These aren't syntax errors; they are **runtime behaviors** of the model or the adapter.

5.1.3.5 (6) `HallucinationDetected`

This is a semantic exception. If the runtime detects (via internal consistency checks or external fact-checkers) that the model is making things up, it triggers this block. Here, we try one more time, perhaps with a more "conservative" prompt.

5.1.3.6 (7) `RETRY WITH ...`

Some errors are transient or can be fixed by changing parameters. If the context length is exceeded, we might RETRY with a lower temperature to encourage the model to be more concise.

5.1.4 Running It

Run the safe generation with a potentially complex prompt:

```
spl run cookbook/07_safe_generation/safe_generation.spl --adapter ollama \
    prompt="What are the trade-offs of microservices?"
```

Observe the output status. If the first pass is good, you'll see status='high_quality'. If the model needed a second try, you'll see status='refined'.

5.1.5 What Just Happened

LLM calls: 2 or 3. (Depending on the quality pass)

The "Conductor" (SPL Runtime) ensured the safety of the transaction: 1. Proposed a response. 2. Validated the proposal against a quality standard. 3. Executed a "Compensating Transaction" (improvement) if the quality was marginal. 4. Committed the result with a clear metadata tag (status).

This pattern transforms an LLM from a "black box" into a **controlled component** within a larger system.

5.1.6 Reproducibility Note

The "Judge" model needs to be highly stable. If your judge is too small (e.g., a 1B parameter model), its evaluations might be inconsistent. For this recipe, we recommend using a "Senior" model like **gemma3 (27B)** or **Claude 3.5 Sonnet** as the evaluator, even if a "Junior" model (like **phi4**) is doing the initial generation.

5.1.7 When to Use This Pattern

Use the **Safe Generation** pattern when: - **Quality is Non-Negotiable**: Customer-facing content, documentation, or legal summaries. - **Handling Refusals**: When you need to gracefully handle cases where the model's safety filters block a legitimate request. - **Self-Correction**: When the task is complex enough that the model rarely gets it perfect on the first try.

5.1.8 Exercises

1. **Strict Guardrails.** Modify the EVALUATE block so that OTHERWISE results in a REJECT or an error message rather than a best_effort commit.
2. **Add a "Banned Word" check.** Use a Python tool (Chapter 3.2) to check for specific words in @result. If found, trigger a custom exception.
3. **Two-Model Judge.** Modify the recipe to use two different models as judges and only commit if they *both* agree the quality is high.

Chapter 5.2

Guardrails Pipeline

"Safety is not an after-thought; it is the architecture."

5.2.1 The Pattern

In a production AI system, "hoping" the model stays safe is not a strategy. You need a rigorous, multi-layered defense that catches harmful content, detects private data (PII), and ensures the output is both relevant and accurate.

A single "Safety Prompt" is rarely enough. You need **Guardrails**—a series of "Gates" that every request must pass through before it is allowed to generate a response.

The SQL analogy is **Multi-Layered Security and Validation**. You have a firewall (the keyword pre-screen), Row-Level Security (the input classifier), Data Masking (PII redaction), and a final Audit (output validation). If any gate fails, the transaction is rolled back or modified before it reaches the user.

The Guardrails Pipeline recipe (Recipe 18) is the gold standard for safe AI. It implements four distinct gates, mixing deterministic code (for speed and cost) with probabilistic LLMs (for nuance).

5.2.2 The SPL Approach

This recipe demonstrates the **80/20 Rule** in action: using CALL for 80% of the safety work (keywords, regex) and GENERATE for the remaining 20% (nuanced context).

5.2.3 The .spl File (Annotated)

```
-- Recipe 18: Guardrails Pipeline
-- Input validation → safe generation → output validation.
```

```
WORKFLOW guardrails_pipeline
    INPUT: @user_input TEXT
DO
    -- Gate 1a: Keyword pre-screen (Deterministic)
    CALL classify_input_keywords(@user_input) INTO @keyword_class -- (1) The Firew

    EVALUATE @keyword_class
        WHEN STARTSWITH 'harmful' THEN
            COMMIT 'I cannot help with that.' WITH status = 'blocked_harmful'
        WHEN STARTSWITH 'off_topic' THEN
            COMMIT 'Outside my scope.' WITH status = 'blocked_off_topic'
    END

    -- Gate 1b: LLM input classification (Probabilistic)
    GENERATE classify_input(@user_input) INTO @input_class -- (2) The Nuanced Check

    EVALUATE @input_class
        WHEN 'harmful' THEN COMMIT '...' WITH status = 'blocked_harmful'
        WHEN 'off_topic' THEN COMMIT '...' WITH status = 'blocked_off_topic'
    END

    -- Gate 2: PII detection and redaction (Deterministic)
    CALL detect_pii(@user_input) INTO @pii_report -- (3) Data Masking
    EVALUATE @pii_report
        WHEN STARTSWITH 'pii_found' THEN
            CALL redact_pii(@user_input) INTO @clean_input
        OTHERWISE
            SET @clean_input = @user_input
    END

    -- Gate 3: Safe Generation
    GENERATE safe_response(@clean_input) INTO @raw_response

    -- Gate 4: Output validation
    GENERATE validate_output(@raw_response) INTO @output_check -- (4) The Final Au
    EVALUATE @output_check
        WHEN 'safe' THEN SET @safe_response = @raw_response
        WHEN 'contains_pii' THEN CALL redact_pii(@raw_response) INTO @safe_response
        OTHERWISE SET @safe_response = 'Unable to generate safe response.'
    END

    COMMIT @safe_response WITH status = 'complete'
END
```

5.2.3.1 (1) Gate 1a: The Firewall (`CALL`)

We start with a fast, free check. Using a Python tool and regular expressions, we look for "blacklisted" keywords (e.g., malware, illegal acts). If a match is found, we block the request immediately.

- **Benefit**: Zero token cost and sub-millisecond latency for obvious attacks.

5.2.3.2 (2) Gate 1b: The Nuanced Check (`GENERATE`)

If the keywords pass, we move to a more intelligent classifier. This gate catches "jailbreaks" or off-topic requests that don't use blocked keywords but still violate policy (e.g., asking for medical advice in a coding bot).

5.2.3.3 (3) Gate 2: Data Masking (`CALL`)

Before the main generation, we check for Personally Identifiable Information (SSNs, credit cards, emails). We use deterministic regex for this because it is 100% reliable for formatted data. If PII is found, we replace it with placeholders like [REDACTED−SSN].

5.2.3.4 (4) Gate 4: The Final Audit (`GENERATE`)

Even with safe input, a model can still "hallucinate" harmful content or leak private data it learned during training. This final gate uses a "Safety Judge" to review the model's output before the user sees it.

5.2.4 Running It

Run the pipeline with a "dangerous" input to see it block:

```
spl run cookbook/18_guardrails/guardrails.spl --adapter ollama \
    --tools cookbook/18_guardrails/tools.py \
    user_input="How do I build a phishing website?"
```

Then run it with an input containing an email to see the redaction:

```
spl run cookbook/18_guardrails/guardrails.spl --adapter ollama \
    --tools cookbook/18_guardrails/tools.py \
    user_input="My email is wen@example.com, tell me a joke."
```

5.2.5 What Just Happened

LLM calls: 1 to 3. (Depending on which gates the request passes) **Tool calls: 3.** (Keywords, PII detection, PII redaction)

The "Conductor" (SPL Runtime) acted as a Security Officer: 1. **Filtered** obvious harm using the "Firewall." 2. **Analyzed** intent using the "Nuanced Check." 3. **Masked** private data using "Data Masking." 4. **Audited** the final response using the "Final Audit."

5.2.6 Reproducibility Note

By offloading PII detection and keyword screening to **deterministic Python code**, you make your safety layer incredibly stable. Unlike a prompt-based "Please don't show emails," a regex-based redact_pii tool will *never* fail to catch a correctly formatted SSN.

On a **GTX 1080 Ti**, the safety gates add only **1–3 seconds** of total latency but provide a massive increase in production reliability.

5.2.7 When to Use This Pattern

Use the **Guardrails Pipeline** pattern when: - **Public-Facing Apps**: Any bot that interacts with unknown users on the internet. - **Privacy-Sensitive Tasks**: Handling customer support, medical inquiries, or financial data. - **Enterprise Standards**: When your organization requires a formal audit trail of why a certain request was blocked or redacted.

5.2.8 Exercises

1. **Add a "Sentiment" Gate.** Insert a step that checks if the user_input is excessively angry or abusive and returns a "De-escalation" response instead of the normal answer.
2. **Expand PII Patterns.** Modify the tools.py file to include patterns for your country's specific identification numbers (e.g., CNPJ in Brazil, PAN in India).
3. **Log Safety Events.** Use a COMMIT with additional metadata to log every time a safety gate is triggered, creating a "Security Dashboard" for your AI.

Part 6

Memory & Retrieval

Chapter 6.1

RAG Query

"Knowledge is not what you have in your head, but knowing where to find it."

6.1.1 The Pattern

LLMs are brilliant, but they have a "Knowledge Cutoff." They only know what they were trained on. If you ask a model about a private document, a recent news event, or your own internal company data, it will either admit ignorance or, worse, hallucinate a plausible-sounding lie.

RAG (Retrieval-Augmented Generation) is the industry-standard solution. Instead of relying on the model's memory, you provide it with the relevant "background reading" alongside your question. You fetch the right information, inject it into the prompt, and ask the model to synthesize an answer based *only* on that context.

The SQL analogy is a **JOIN against an external table** or a **Subquery**:

```
SELECT answer
FROM (
    SELECT content FROM documents
    ORDER BY similarity(content, :question) DESC
    LIMIT 3
) AS background
WHERE question = :question;
```

You aren't querying the model; you are querying a "table" of your own data and joining it with the model's reasoning capabilities.

The RAG Query recipe (Recipe 08) demonstrates the simplest possible RAG implementation in SPL. It uses the built-in rag.query function to perform a semantic search against a local vector database and feeds the results directly into the prompt.

6.1.2 The SPL Approach

This recipe introduces the rag namespace, which provides high-level access to the SPL runtime's integrated vector store.

6.1.3 The .spl File (Annotated)

```
-- Recipe 08: RAG Query
-- Retrieval-augmented generation over indexed documents.

PROMPT rag_answer                                  -- (1) name the prompt block
SELECT
    system_role('You are a knowledgeable assistant. Use the provided context to ans
                                                   -- (2) context-aware instruction
    rag.query(context.question, top_k=3) AS background,
                                                   -- (3) The Semantic Join
    context.question AS question                   -- (4) The user's query
GENERATE answer(question)                          -- (5) Grounded generation
```

6.1.3.1 (1) `PROMPT rag_answer`

Even though RAG sounds complex, the execution flow is still a single model call. The "retrieval" happens during the SELECT phase, just like a SQL join happens before the final result set is produced.

6.1.3.2 (2) `system_role(...)`

It is crucial to tell the model that it *has* context and that it should use it. This prevents the model from ignoring the retrieved data in favor of its own pre-trained weights.

6.1.3.3 (3) `rag.query(context.question, top_k=3) AS background`

This is the heart of RAG in SPL. - **rag.query**: A built-in function that takes a string, converts it into a vector (an embedding), and searches the local FAISS index for the 3 most similar chunks of text. - **top_k=3**: We only want the most relevant information. Too much context can "drown" the model or exceed its token limit.

SQL Analogy: A **Vector Similarity Search**. You are ordering your "documents" table by how closely they match the meaning of the question.

6.1.3.4 (4) & (5) `question` and `GENERATE`

We bind the question and the retrieved background to the context row. When GENERATE fires, the model sees the question *and* the relevant evidence side-by-side. The resulting answer is "grounded" in your data.

6.1.4 Running It

RAG requires a one-time setup to index your data:

```
# 1. Install dependencies
pip install numpy faiss-cpu sentence-transformers

# 2. Add a document to the index
# This chunks the file into paragraphs and stores their embeddings locally.
spl rag add /path/to/your/document.md
```

Now, run the query:

```
spl run cookbook/08_rag_query/rag_query.spl --adapter ollama \
    question="What is the main argument of this document?"
```

6.1.5 What Just Happened

LLM calls: 1. Vector Search calls: 1.

The "Conductor" (SPL Runtime) managed the retrieval: 1. Calculated the embedding for the question. 2. Searched the local `.spl/vectors.faiss` file for matches. 3. Fetched the corresponding text from the `.spl/vectors_meta.db` database. 4. Injected the found text into the background column. 5. Passed the complete, grounded context to the model for the final answer.

6.1.6 Reproducibility Note

RAG adds a layer of complexity to reproducibility: the **Embedding Model**. SPL defaults to `all-MiniLM-L6-v2`, a small but efficient model that runs on your CPU. If you change your embedding model, your search results will change, which will change the model's answer.

On a **GTX 1080 Ti**, the vector search takes **less than 50ms** for an index of thousands of paragraphs. The bottleneck remains the LLM generation time.

6.1.7 When to Use This Pattern

Use the **RAG Query** pattern when: - **Private Data**: You need to query documents that were never part of a public training set. - **Up-to-Date Info**: You need the model to know about events that happened after its knowledge cutoff. - **Fact-Checking**: You want to reduce hallucinations by forcing the model to cite its sources from the provided context.

6.1.8 Exercises

1. **Inspect the Retrieval.** Run spl rag query "your question" ——top—k 5 from the command line. This shows you exactly what the model will see *before* it generates an answer. This is the "EXPLAIN" plan for your RAG query.
2. **Filter by Metadata.** (Advanced) Research how to add metadata to your documents (e.g., author, date) and filter your rag.query to only search specific files.
3. **No-Grounding Test.** Run the recipe *before* you index any documents. Observe how the model handles the empty context. Does it admit it doesn't know, or does it try to guess? This is a great way to test the "honesty" of a model.

Chapter 6.2

Memory Conversation

"The difference between a tool and a partner is memory."

6.2.1 The Pattern

Most LLM interactions are "stateless." The model doesn't remember who you are, what you said five minutes ago, or what your preferences are. Every time you start a new session, you have to re-explain everything. This makes for a very frustrating "Groundhog Day" experience for the user.

Memory-Augmented Conversation solves this by providing the model with a "Long-Term Memory." Before each turn, the system looks up facts about the user from a persistent database and injects them into the prompt. It also extracts *new* facts from the current turn to update that memory for the future.

The SQL analogy is a **User Profile Table** or a **Session Store**. Every time a user visits your website, you fetch their user_id from a cookie and join it with a profiles table to personalize their experience.

The Memory Conversation recipe (Recipe 19) implements this "Persistent Persona" pattern. It uses the memory.get function to fetch a user profile, uses an LLM to merge new facts into that profile, and then generates a response that is aware of the user's history and preferences.

6.2.2 The SPL Approach

This recipe introduces the memory namespace and the pattern of "Fact Extraction and Merging."

6.2.3 The .spl File (Annotated)

```
-- Recipe 19: Memory-Augmented Conversation
-- Conversational agent that remembers facts from prior turns.
```

```
WORKFLOW memory_conversation
    INPUT: @user_input TEXT
DO
    -- Phase 1: Retrieval (Remembering)
    SELECT memory.get('chat_user_profile') AS profile INTO @profile -- (1) The Long
    SELECT memory.get('chat_history') AS history INTO @history

    -- Phase 2: Fact Extraction (Learning)
    GENERATE extract_facts(@user_input) INTO @new_facts -- (2) Identifying new info

    EVALUATE @new_facts
        WHEN 'no_new_facts' THEN -- do nothing
        OTHERWISE
            GENERATE merge_profile(@profile, @new_facts) INTO @profile -- (3) Upda
    END

    -- Phase 3: Contextual Response
    GENERATE contextual_reply(@user_input, @profile, @history) INTO @response

    -- Update history (Short-term context)
    @history := @history + '\nUser: ' + @user_input + '\nAI: ' + @response
    GENERATE trim_history(@history, 10) INTO @history

    COMMIT @response WITH status = 'complete'
END
```

6.2.3.1 (1) Phase 1: Retrieval (`memory.get`)

We start by querying the built-in SPL memory store (a local SQLite database). We fetch two things: - **chat_user_profile**: A summary of what we know about the user (e.g., "Name: Alice, Role: Data Scientist"). - **chat_history**: The last few turns of the conversation to maintain flow.

SQL Analogy: **SELECT FROM session_state**. You are pulling the existing context for this specific user.

6.2.3.2 (2) Phase 2: Fact Extraction

Instead of just replying, we ask the model to "listen" for new information. If the user says "I prefer Python over R," the model identifies this as a new fact and produces @new_facts.

6.2.3.3 (3) Updating the State (`merge_profile`)

We take the *old* profile and the *new* facts and ask the model to merge them into a single, updated profile. This prevents the memory from becoming a disorganized list of sentences; it becomes a structured, evolving "biography" of the user.

6.2.4 Running It

Run the conversation over multiple turns. Notice how the model "learns" as you go:

```
# Turn 1: Introduce yourself
spl run cookbook/19_memory_conversation/memory_chat.spl \
    user_input="My name is Alice and I'm a data scientist"

# Turn 2: Test the memory
spl run cookbook/19_memory_conversation/memory_chat.spl \
    user_input="What is my name and what do I do?"
```

6.2.5 What Just Happened

LLM calls: 3. (Extract, Merge, Reply) **Memory calls: 2.** (Get profile, Get history)

The "Conductor" (SPL Runtime) managed a "Persistent Identity": 1. **Recalled** the user's past. 2. **Observed** new details in the current input. 3. **Synthesized** the new and old information into an updated world-view. 4. **Responded** with a "personalized" voice.

6.2.6 Reproducibility Note

The reliability of memory depends on the **Extraction Quality**. If your model is too small, it might miss subtle facts or misinterpret them. We recommend using a model with high "Information Density" (like **gemma3**) for the merge_profile step to ensure the memory remains clean and accurate.

On a **GTX 1080 Ti**, the memory lookup adds **zero** noticeable latency (it's a local SQLite query), but the extra LLM calls for extraction and merging add about **10–15 seconds** per turn.

6.2.7 When to Use This Pattern

Use the **Memory Conversation** pattern when: - **Personal Assistants**: Bots that help with scheduling, coding, or writing over long periods. - **Educational Tutors**: A bot that remembers which concepts a student has already mastered and where they are struggling. - **Customer Support**: A bot that knows a user's purchase history or past complaints without being told every time.

6.2.8 Exercises

1. **Inspect the Memory.** Use the command line spl memory get chat_user_profile to see the raw text the model has stored about you.

2. **Add a "Mood" Memory.** Modify the workflow to track the user's emotional state over time (e.g., "Frustrated," "Curious") and adjust the @response tone accordingly.
3. **Reset Logic.** Add a special input command (e.g., user_input="Forget everything") that triggers a memory.delete call to wipe the profile and start over.

Part 7

Multi-Agent Systems

Chapter 7.1

Debate Arena

"The truth is a spark that flies when two stones of opinion are struck together."

7.1.1 The Pattern

When a single model answers a question, you get a single perspective. That perspective is shaped by the model's training data, its safety filters, and its inherent biases. If you are exploring a complex, nuanced topic—like "Should AI be open-sourced?"—a single response is rarely enough. You need to see the "pro" and the "con" argued with equal vigor.

The **Debate Arena** pattern uses multiple LLM instances (or personas) to argue opposing sides of a motion. By forcing the models to rebut each other's points over several rounds, you expose the depth of the topic and the weaknesses in each argument.

The SQL analogy is a **Self-Join with State** or a **Recursive CTE**. You are taking the "output" of one row (the pro argument) and using it as the "input" for the next row (the con rebuttal), repeating the process until a termination condition is met.

The Debate Arena recipe (Recipe 11) implements a multi-round debate. Two personas trade opening statements and rebuttals, while a third "Judge" persona monitors the history and declares a winner. This is a powerful technique for stress-testing ideas and discovering edge cases.

7.1.2 The SPL Approach

This recipe introduces the WHILE loop for iterative reasoning and the pattern of "State Accumulation" (building a history string over multiple steps).

7.1.3 The .spl File (Annotated)

```
-- Recipe 11: Debate Arena
-- Two personas argue opposing sides, then a judge picks the winner.

WORKFLOW debate_arena
    INPUT: @topic TEXT, @max_rounds integer DEFAULT 3
DO
    @round := 0
    @pro_history := ''                          -- (1) State initialization
    @con_history := ''

    -- Opening statements
    GENERATE pro_argument(@topic, 'opening statement') INTO @pro
    GENERATE con_argument(@topic, 'opening statement') INTO @con

    @pro_history := @pro                        -- (2) Seeding the history
    @con_history := @con

    WHILE @round < @max_rounds DO                -- (3) Iterative Reasoning
        -- Pro rebuts Con's latest point
        GENERATE pro_argument(@topic, @con_history) INTO @pro_rebuttal
        @pro_history := @pro_history + '\n---\n' + @pro_rebuttal

        -- Con rebuts Pro's latest point
        GENERATE con_argument(@topic, @pro_history) INTO @con_rebuttal
        @con_history := @con_history + '\n---\n' + @con_rebuttal

        @round := @round + 1
    END

    -- Judge evaluates both sides
    GENERATE judge_debate(@topic, @pro_history, @con_history) INTO @verdict

    COMMIT @verdict                             -- (4) Final Judgment
END
```

7.1.3.1 (1) & (2) State Management

In a debate, context is everything. You cannot rebut an argument you haven't seen. We initialize @pro_history and @con_history as strings and "seed" them with the opening statements. This is the "Working Memory" of the workflow.

7.1.3.2 (3) `WHILE` (Iterative Reasoning)

This is the first time we've seen a loop in SPL. The WHILE loop allows the workflow to "think" for multiple turns. - In the first turn, the models establish their positions. - In subsequent turns, they refine their positions based on the opponent's "evidence."

SQL Analogy: A **Recursive CTE** (WITH RECURSIVE). Each iteration of the loop builds upon the result set of the previous iteration.

7.1.3.3 (4) The Final Judgment

After the rebuttals are complete, we call the judge_debate step. This step receives the *entire* history of the conversation. The judge is instructed to look for logical fallacies, strength of evidence, and clarity of thought to decide which side "won" the debate.

7.1.4 Running It

Pick a controversial topic and let the models fight it out:

```
spl run cookbook/11_debate_arena/debate.spl --adapter ollama \
    topic="Remote work is better than office work"
```

Watch the execution trace. You will see the pro_argument and con_argument steps repeating as the loop progresses.

7.1.5 What Just Happened

LLM calls: 2 + (2 * max_rounds) + 1. (With 3 rounds, that's 9 calls)

The "Conductor" (SPL Runtime) managed a dynamic conversation: 1. Set the stage with opening statements. 2. Iteratively "passed the microphone" between the pro and con personas. 3. Appended the new arguments to the cumulative history strings. 4. Terminated the loop after the specified number of rounds. 5. Presented the full "transcript" to the judge for a final verdict.

7.1.6 Reproducibility Note

Debates are highly non-deterministic. A single strong point in the opening statement can shift the entire trajectory of the conversation.

On a **GTX 1080 Ti**, a 3-round debate can take **30–90 seconds**, as each rebuttal depends on the previous generation finishing. This is a sequential process—it cannot be fanned out in parallel because the context of Round 2 depends on the output of Round 1.

7.1.7 When to Use This Pattern

Use the **Debate Arena** pattern when: - **Red Teaming**: Argue against your own product's features or security assumptions to find weaknesses. - **Complex Decision Making**: When you have two valid paths (e.g., "SQL vs. NoSQL") and want to explore the trade-offs in depth. - **Educational Tools**: Create an interactive tutor that can argue multiple sides of a historical or scientific debate.

7.1.8 Exercises

1. **Change the Personas.** (Advanced) Research how to use CREATE FUNCTION (Chapter 9.1) to give the Pro and Con sides specific "backgrounds" (e.g., "You are a CEO" vs. "You are a junior developer").
2. **Add a Moderator.** Insert a step inside the WHILE loop where a "Moderator" model summarizes the debate so far and asks a specific follow-up question.
3. **Variable Rounds.** Modify the workflow to accept @max_rounds from the command line, and try a "Lightning Round" (1 round) vs. an "Exhaustive Debate" (5 rounds).

Chapter 7.2

Multi-Agent Collaboration

"The best reports are written by many hands but one mind."

7.2.1 The Pattern

In a complex project, you don't ask one person to do everything. You have a Researcher to find the facts, an Analyst to interpret the data, and a Writer to polish the prose. Each role requires a different "mindset" and set of skills. When you try to force an LLM to be all three at once, it often produces a "Jack-of-all-trades" response that is mediocre in every dimension.

Multi-Agent Collaboration is the practice of breaking a workflow into specialized roles (agents) and delegating tasks between them. Each agent is a separate "persona" with its own system instructions and expertise.

The SQL analogy is a **Modular Architecture** or **Database Views**. Instead of one giant, 500-line query, you build specialized views (v_research, v_analysis) and join them together in a final report. Each view does one thing perfectly.

The Multi-Agent Collaboration recipe (Recipe 14) demonstrates this modularity. It defines three distinct PROCEDURE blocks—Researcher, Analyst, and Writer—and coordinates them via a central WORKFLOW. This ensures that each phase of the project gets the specialized attention it deserves.

7.2.2 The SPL Approach

This recipe introduces the use of **Procedures as Agents** and the pattern of "Linear Delegation" (passing the output of one agent as the input to the next).

7.2.3 The .spl File (Annotated)

```
-- Recipe 14: Multi-Agent Collaboration
-- Specialized agents (Researcher, Analyst, Writer) collaborate on a report.

PROCEDURE researcher(topic TEXT)                -- (1) Agent 1: The Researcher
DO
    GENERATE research_facts(topic) INTO @facts
    GENERATE identify_key_themes(@facts) INTO @themes
    COMMIT @facts + '\n\nKey Themes:\n' + @themes
END

PROCEDURE analyst(research TEXT, topic TEXT) -- (2) Agent 2: The Analyst
DO
    GENERATE analyze_trends(research) INTO @trends
    GENERATE assess_risks(research, topic) INTO @risks
    COMMIT 'Trends: ' + @trends + '\n\nRisks: ' + @risks
END

PROCEDURE writer(research TEXT, analysis TEXT, topic TEXT) -- (3) Agent 3: The Wri
DO
    GENERATE draft_report(topic, research, analysis) INTO @draft
    GENERATE critique(@draft) INTO @feedback
    GENERATE revise_report(@draft, @feedback) INTO @final
    COMMIT @final
END

WORKFLOW multi_agent_report
    INPUT: @topic TEXT
DO
    CALL researcher(@topic) INTO @research     -- (4) Linear Delegation
    CALL analyst(@research, @topic) INTO @analysis
    CALL writer(@research, @analysis, @topic) INTO @report

    COMMIT @report WITH status = 'complete'
END
```

7.2.3.1 (1), (2) & (3) Procedures as Agents

In SPL, a PROCEDURE is the perfect container for an "Agent." - It has its own inputs and outputs. - It can contain its own internal logic (like the Writer's "draft-critique-revise" loop). - It can be tested and debugged in isolation before being added to the main workflow.

By separating the Researcher from the Analyst, we prevent "Context Contamination." The Researcher doesn't need to worry about trends or risks; its only job is to find facts.

7.2.3.2 (4) Linear Delegation

The WORKFLOW acts as the Project Manager. It calls the researcher, waits for the facts, and then hands those facts to the analyst. This is the "Waterfall" model of agentic workflows.

SQL Analogy: **Pipelining**. The output of the first stage becomes the filter for the second stage.

7.2.4 Running It

Run the collaborative report on a technical topic:

```
spl run cookbook/14_multi_agent/multi_agent.spl --adapter ollama \
    topic="The impact of quantum computing on cybersecurity"
```

In the execution trace, you will see the distinct "handoffs" between the three procedures.

7.2.5 What Just Happened

LLM calls: ~8-10. (Depending on the internal steps of each procedure)

The "Conductor" (SPL Runtime) managed a professional team: 1. **Fact-Finding**: The Researcher gathered raw data. 2. **Synthesis**: The Analyst turned data into insights. 3. **Polishing**: The Writer turned insights into a human-readable report. 4. **Handoffs**: The runtime ensured that the right context was passed to the right agent at the right time.

7.2.6 Reproducibility Note

This recipe is highly stable because each step is small and focused. By decomposing the task, you reduce the "reasoning load" on the model in any single call.

On a **GTX 1080 Ti**, the total time for a full report is about **45–75 seconds**. While this is slower than a single-prompt summary, the **Quality Improvement** is significant. You get a report that is deeper, more structured, and more accurate.

7.2.7 When to Use This Pattern

Use the **Multi-Agent Collaboration** pattern when: - **High-Stakes Reports**: Whitepapers, strategic plans, or technical documentation. - **Multi-Step Expertise**: When a task requires distinct phases (e.g., "Research → Code → Document"). - **Quality Benchmarking**: When you want to see if adding an "Analyst" or "Critic" step actually improves the final output of your "Writer."

7.2.8 Exercises

1. **Add a "Critic" Agent.** Create a fourth PROCEDURE called critic that reviews the final report and suggests one last round of improvements.

2. **Parallel Research.** (Advanced) If your research can be split into two topics (e.g., "Technical" and "Economic"), use a WITH clause to call two researchers in parallel.
3. **Model Specialization.** Use the USING MODEL clause (Chapter 9.1) to assign different models to different roles (e.g., use a "fast" model for Research and a "strong" model for Analysis).

Chapter 7.3

Ensemble Voting

"The wisdom of the crowd is more reliable than the genius of the individual."

7.3.1 The Pattern

In the world of statistics and machine learning, an **Ensemble** is a collection of models that work together to solve a single problem. The core idea is simple: if you ask one person a question, they might be wrong. If you ask five people and take the average of their answers, you are much more likely to be right. This is known as "The Wisdom of the Crowd."

For LLMs, this pattern is incredibly powerful for reducing "hallucinations." A single model might confidently state a fact that is slightly off. But if you generate five independent answers, the correct facts will likely appear in the majority, while the hallucinations will be unique to each run and can be "voted out."

The SQL analogy is a **UNION ALL with a GROUP BY and HAVING COUNT(*) > 1**. You are collecting multiple result sets and then filtering for the "consensus" that appears across most of them.

The Ensemble Voting recipe (Recipe 20) generates five independent candidate answers, scores each one for accuracy and clarity, identifies the "Consensus" themes across all five, and finally synthesizes a "Winning" response.

7.3.2 The SPL Approach

This recipe demonstrates **Multi-Sample Generation** and **Consensus-Driven Synthesis**.

7.3.3 The .spl File (Annotated)

```
-- Recipe 20: Ensemble Voting
-- Generates multiple independent answers, then uses majority voting to pick the be

WORKFLOW ensemble_voting
    INPUT: @question TEXT
DO
    -- Phase 1: Multi-Sampling (The Crowd)
    GENERATE answer_candidate(@question) INTO @candidate_1 -- (1) Five independent
    GENERATE answer_candidate(@question) INTO @candidate_2
    GENERATE answer_candidate(@question) INTO @candidate_3
    GENERATE answer_candidate(@question) INTO @candidate_4
    GENERATE answer_candidate(@question) INTO @candidate_5

    -- Phase 2: Scoring (Individual Quality)
    GENERATE score_candidate(@candidate_1) INTO @score_1 -- (2) Measuring each
    GENERATE score_candidate(@candidate_2) INTO @score_2
     ...

    -- Phase 3: Consensus (Finding the Wisdom)
    GENERATE find_consensus(@candidate_1, @candidate_2, ...) INTO @consensus -- (3)

    -- Phase 4: Selection and Polishing
    GENERATE select_winner(@scores, @consensus) INTO @best_candidate
    GENERATE polish(@best_candidate, @consensus) INTO @final_answer -- (4) The Fina

    COMMIT @final_answer WITH status = 'complete', candidates = 5
END
```

7.3.3.1 (1) Phase 1: Multi-Sampling

We run the *exact same* prompt five times. In a stateless environment, we rely on the model's inherent "Temperature" (randomness) to produce slightly different phrasings and perspectives in each run.

7.3.3.2 (2) Phase 2: Scoring

Before we look for consensus, we score each candidate individually. - "Is this answer coherent?" - "Does it address all parts of the question?" - "Are there obvious factual errors?"

7.3.3.3 (3) Phase 3: Finding Consensus

This is the "Voting" step. We pass all five candidates to a "Synthesizer" model and ask it: "What are the common facts and arguments that appear in at least three of these versions?" This identifies the "Grounded Truth" that most of the runs agree on.

7.3.3.4 (4) Phase 4: Polishing

Finally, we take the highest-scoring candidate and "polish" it using the consensus themes. If the winner missed a detail that the other four versions caught, we add it back in. The result is a

"Super-Answer" that is more robust than any single model could produce.

7.3.4 Running It

Run the ensemble on a controversial or factual topic:

```
spl run cookbook/20_ensemble_voting/ensemble.spl --adapter ollama \
    question="What are the main causes of the Great Depression?"
```

Observe the output. You will notice that the final answer is exceptionally balanced and thorough, as it has been "filtered" through five independent reasoning passes.

7.3.5 What Just Happened

LLM calls: 13. (5 candidates + 5 scores + 1 consensus + 1 winner + 1 polish)

The "Conductor" (SPL Runtime) managed a "Delphi Method" process: 1. **Surveyed** the crowd (the 5 candidates). 2. **Audited** each response for quality. 3. **Extracted** the consensus agreement. 4. **Selected** the strongest voice. 5. **Enhanced** that voice with the collective wisdom of the group.

7.3.6 Reproducibility Note

Ensemble Voting is the "Gold Standard" for stability. While a single run might vary by 20–30% in its details, the **Ensemble Result** is remarkably consistent. The "noise" of individual runs is cancelled out, leaving only the "signal" of the underlying model's knowledge.

On a **GTX 1080 Ti**, this recipe takes **2–4 minutes**. It is a heavy workload, but for high-accuracy requirements, it is the most reliable tool in the cookbook.

7.3.7 When to Use This Pattern

Use the **Ensemble Voting** pattern when: - **Fact-Critical Queries**: Medical, legal, or historical questions where being "mostly right" is not enough. - **Reducing Hallucination**: When you suspect the model might "make things up" and want to verify facts across multiple samples. - **Consensus Building**: When you need a balanced view of a topic with multiple valid perspectives.

7.3.8 Exercises

1. **Parallel Execution.** (Required for speed) Use the WITH clause (Chapter 9.1) to run all 5 candidates and all 5 scores in parallel blocks.

2. **Diversity Prompting.** Instead of running the same prompt five times, give each candidate a slightly different "Persona" (e.g., "Answer as an economist," "Answer as a historian," "Answer as a sociologist").
3. **Weighted Voting.** Modify the select_winner step to give more "weight" to candidates that have higher individual scores.

Chapter 7.4

Socratic Tutor

"The highest form of teaching is not giving answers, but asking the right questions."

7.4.1 The Pattern

Most AI tutors act like talking encyclopedias. You ask "Why is the sky blue?" and they give you a three-paragraph lecture on Rayleigh scattering. This is efficient for information retrieval, but it is terrible for *learning*. Learning happens when a student is forced to think, reason, and arrive at a conclusion themselves.

The **Socratic Tutor** pattern implements the oldest pedagogical technique in history. Instead of explaining, the AI acts as a guide. It asks a question, listens to the student's response, identifies a gap in their logic, and then asks a *better* question to help them close that gap.

The SQL analogy is an **Iterative Refinement Query**. You are querying a "Student Mind" (the user) and using the results to filter and focus your next "Query" (the question), repeating until the "Result Set" (the student's understanding) meets your quality threshold.

The Socratic Tutor recipe (Recipe 32) simulates this dialogue. It doesn't just ask questions; it **simulates** a student's responses to test its own effectiveness, scores the student's emerging understanding, and adapts its questioning strategy in real-time.

7.4.2 The SPL Approach

This recipe demonstrates **Persona-Driven Branching**—using a strict system role to constrain the model's behavior while using an EVALUATE block to change the pedagogical strategy.

7.4.3 The .spl File (Annotated)

```
-- Recipe 32: Socratic Tutor
-- Ask guiding questions rather than giving answers directly.

CREATE FUNCTION socratic_persona(student_level TEXT)
RETURNS TEXT AS $$
You are a Socratic tutor.
Core rules:
1. Ask exactly ONE clear question at a time.
2. NEVER state facts or give direct answers.
3. Build on what the student just said.
Student level: {student_level}
$$;

WORKFLOW socratic_tutor
    INPUT: @topic TEXT, @student_level TEXT DEFAULT 'high school'
DO
    -- Phase 1: Initiation
    GENERATE opening_question(@topic, socratic_persona(@student_level)) INTO @q1

    -- Phase 2: Dialogue Simulation (Testing the logic)
    GENERATE simulate_student(@q1, @student_level) INTO @s1 -- (1) The "Dummy Stude
    GENERATE followup_question(@q1, @s1) INTO @q2
    GENERATE simulate_student(@q2, @student_level) INTO @s2

    -- Phase 3: Assessment
    GENERATE assess_understanding(@s1, @s2) INTO @score    -- (2) Measuring Learnin

    -- Phase 4: Adaptive Branching
    EVALUATE @score
        WHEN > 7 THEN                                      -- (3) High Understanding
            GENERATE consolidation_question(@s2) INTO @q3
        OTHERWISE                                          -- (4) Low Understanding
            GENERATE hint_question(@q2, @s2) INTO @q3
    END

    -- Phase 5: Final Performance
    GENERATE simulate_student(@q3) INTO @s3
    COMMIT compile_dialogue(@q1, @s1, @q2, @s2, @q3, @s3) WITH status = 'complete'
END
```

7.4.3.1 (1) The "Dummy Student" (`simulate_student`)

To build a truly effective tutor, we don't just write prompts; we test them. In this workflow, we use one model instance to act as the Tutor and another to act as a "Simulated Student" at a specific level (e.g., middle school). - **Why?** This allows us to "dry-run" the educational path. If the tutor is too confusing for the simulated student, it will likely be too confusing for a real human.

7.4.3.2 (2) Measuring Learning (`assess_understanding`)

We use a "Judge" prompt to look at the student's two previous answers. - "Are they getting closer to the truth?" - "Are they identifying the right variables?" This step turns the "Vibe" of a conversation into a hard numeric score.

7.4.3.3 (3) & (4) Adaptive Branching

This is where the pedagogy lives. - If the student is succeeding (**Score > 7**), the tutor asks a "Consolidation" question to cement the knowledge. - If the student is struggling (**Otherwise**), the tutor pivots to a "Hint" question to simplify the concept.

SQL Analogy: **Conditional Routing**. You are routing the "Transaction" (the lesson) through different logic paths based on the "Data" (the student's performance).

7.4.4 Running It

Test the tutor on a scientific concept:

```
spl run cookbook/32_socratic_tutor/socratic_tutor.spl \
    --adapter ollama --tools cookbook/32_socratic_tutor/tools.py \
    topic="Why do objects fall at the same speed?" student_level="middle school"
```

Expected output: A three-turn dialogue where the AI guides the student from "gravity pulls everything" toward the more nuanced understanding of mass and acceleration.

7.4.5 What Just Happened

LLM calls: 7. (3 questions + 3 simulated responses + 1 assessment)

The "Conductor" (SPL Runtime) managed a "Pedagogical Loop": 1. **Established** a strict role-play boundary. 2. **Facilitated** a multi-turn reasoning chain. 3. **Monitored** the quality of the student's progress. 4. **Modified** the difficulty level dynamically. 5. **Verified** the final outcome.

7.4.6 Reproducibility Note

The Socratic Tutor is highly dependent on the **Persona Guardrails**. If the model "breaks character" and gives the answer, the learning value is lost. Larger models like **gemma3** are much better at maintaining this discipline than smaller models.

On a **GTX 1080 Ti**, a 3-turn simulation takes **30–50 seconds**. This is a powerful way to generate "Synthetic Training Data" for educational apps or to test different teaching strategies.

7.4.7 When to Use This Pattern

Use the **Socratic Tutor** pattern when: - **Educational Apps**: Building tools that help students think rather than just cheat. - **Corporate Training**: Helping employees discover new processes or compliance rules through reasoning. - **Problem Discovery**: When you have a complex problem and want to "think through it" by having an AI ask you guiding questions.

7.4.8 Exercises

1. **Add a "Subject" Bias.** Modify the socratic_persona to accept a @subject parameter (e.g., "Physics," "History") and observe how it uses different analogies.
2. **Extend the Loop.** Use a WHILE loop (Chapter 7.1) to allow the dialogue to continue for up to 10 turns, or until the @understanding_score reaches 9.
3. **Strict Mode.** Add an EXCEPTION handler that triggers if the model ever says the phrase "The answer is" and forces a RETRY with a stricter safety prompt.

Chapter 7.5

Interview Sim

"To master a skill, you must first survive the simulation."

7.5.1 The Pattern

High-stakes interactions—like job interviews, sales pitches, or crisis management—require more than just knowledge. They require "Grace Under Pressure." The best way to prepare for these moments is through simulation. But human-led simulation is expensive and time-consuming. You need a partner who can play a specific role, stay in character, and provide objective feedback after the session.

The **Interactive Interview Simulation** pattern uses multiple LLM instances to create a "closed-loop" training environment. 1. **Interviewer**: One model instance plays the role of a professional interviewer, asking probing technical questions. 2. **Candidate**: A second model instance (the "Dummy Candidate") answers the questions to test the interview's difficulty. 3. **Evaluator**: A third model instance (the "Judge") reviews the entire transcript and scores it based on a professional rubric.

The SQL analogy is a **Three-Way Join with an Audit Log**. You are joining the "Interviewer Persona" with the "Candidate Persona" over a series of "Turn IDs," and then auditing the resulting "Conversation Table" for quality and consistency.

The Interview Sim recipe (Recipe 33) implements this triad. It allows you to simulate technical interviews for any role (Senior SWE, Data Scientist, etc.) and provides a structured evaluation report that highlights strengths and weaknesses.

7.5.2 The SPL Approach

This recipe demonstrates **Multi-Role Orchestration**—managing three distinct personas in a single workflow while using deterministic tools to aggregate scores.

7.5.3 The .spl File (Annotated)

```
-- Recipe 33: Interview Simulator
-- Two-persona structured Q&A with evaluation.

WORKFLOW interview_sim
    INPUT: @role TEXT, @focus TEXT, @candidate_profile TEXT
DO
    -- Phase 1: Planning
    GENERATE generate_questions(@role, @focus) INTO @questions_json -- (1) Upfront

    -- Phase 2: The Interview (Sequential Generation)
    CALL extract_question(@questions_json, 1) INTO @q1
    GENERATE answer(@q1, @candidate_profile) INTO @a1 -- (2) Role-play generation

    CALL extract_question(@questions_json, 2) INTO @q2
    GENERATE answer(@q2, @candidate_profile) INTO @a2

    -- Phase 3: Evaluation (The Judge)
    GENERATE score(@q1, @a1, evaluation_rubric()) INTO @score1 -- (3) Applying the
    GENERATE score(@q2, @a2, evaluation_rubric()) INTO @score2

    -- Phase 4: Aggregation (Deterministic)
    CALL aggregate_scores(@score1, @score2) INTO @totals -- (4) Python Math Tool

    -- Phase 5: Synthesis
    GENERATE write_report(@totals, @role) INTO @final_report
    COMMIT @final_report WITH status = 'complete'
END
```

7.5.3.1 (1) Upfront Planning (`generate_questions`)

Instead of asking one question at a time, we ask the "Interviewer" model to generate a set of three questions upfront. - **Why?** This ensures the interview has a logical flow (e.g., easy → medium → hard) rather than being a random list of disconnected queries.

7.5.3.2 (2) Role-play Generation

When the "Candidate" model answers, it doesn't just give the "right answer." It answers based on its @candidate_profile. If the profile says "Junior Developer," the model might give a shallow answer or admit it doesn't know. - This creates a **Realistic Simulation**. We aren't just testing the facts; we are testing the interaction.

7.5.3.3 (3) Applying the Rubric (`evaluation_rubric`)

We use a formal JSON rubric to score the answers. This ensures that the evaluation is objective and consistent across different interview runs. We score dimensions like "Technical Accuracy," "Communication," and "Experience."

7.5.3.4 (4) Python Math Tool (`aggregate_scores`)

As always, we hand the numbers back to Python. Python sums the scores, calculates the average, and flags any "Red Flags" found in the JSON feedback. This ensures the final @totals are mathematically perfect.

7.5.4 Running It

Run a simulation for a Senior SWE role:

```
spl run cookbook/33_interview_sim/interview_sim.spl \
    --adapter ollama --tools cookbook/33_interview_sim/tools.py \
    role="Senior Software Engineer" focus="Distributed Systems" \
    candidate_id="alice_senior_swe"
```

Expected output: A full transcript of the three-question interview, followed by a structured evaluation report with a total score (e.g., 32/40) and specific feedback for the candidate.

7.5.5 What Just Happened

LLM calls: 8–10. (Question set + 3 answers + 3 scores + 1 final report) **Tool calls: 5.** (Role loading, Candidate loading, Question extraction, Score aggregation)

The "Conductor" (SPL Runtime) managed a "HR Assessment Center": 1. **Sourced** the correct expertise (Role Context). 2. **Choreographed** a three-turn dialogue between two distinct personas. 3. **Audited** every turn against a professional standard. 4. **Calculated** the performance metrics. 5. **Synthesized** a comprehensive hiring recommendation.

7.5.6 Reproducibility Note

This recipe is highly stable because the questions are generated upfront. The variability comes from the "Candidate's" responses.

On a **GTX 1080 Ti**, a full simulation takes **60–90 seconds**. This is an incredible tool for **Interview Preparation** or for **Hiring Managers** to test their own question sets before using them on real candidates.

7.5.7 When to Use This Pattern

Use the **Interview Sim** pattern when: - **Personal Coaching**: Preparing for a specific job or a high-stakes presentation. - **Content Generation**: Creating "Synthetic Transcripts" for training materials or educational videos. - **Model Evaluation**: Testing how different models perform in a specific persona (e.g., "How does Llama 3 handle being a CEO vs. a Developer?").

7.5.8 Exercises

1. **Add a "Live" mode.** (Advanced) Modify the workflow to replace the @candidate_profile with a context.user_response parameter, allowing a real human to type the answers in a terminal while the AI asks the questions.
2. **Change the Difficulty.** Add a @difficulty parameter to the interviewer_persona and observe how the questions change from "basic syntax" to "architectural trade-offs."
3. **Stress Test.** Give the "Candidate" a profile that says "You are very defensive and don't like being questioned." Observe how the "Interviewer" handles the conflict.

Part 8

Applications

Chapter 8.1

Map-Reduce Summarizer

"To understand the mountain, you must first understand the stones."

8.1.1 The Pattern

Every LLM has a "Context Window"—a limit on how much text it can process at once. If you try to feed a 50-page technical report into a model with a small window, the model will simply "forget" the beginning of the document or crash entirely. Even with large-context models, processing a massive document in one go often leads to "lost-in-the-middle" syndrome, where the model misses key details buried in the center of the text.

Map-Reduce is the classic distributed computing pattern for this problem. 1. **Map**: Split the large document into smaller, manageable chunks and summarize each chunk independently. 2. **Reduce**: Combine those individual summaries into a single, cohesive final report.

The SQL analogy is a **GROUP BY with an Aggregate Function**:

```
SELECT REDUCE(summary)
FROM (
    SELECT MAP(chunk) AS summary
    FROM document_chunks
) AS subquery;
```

You are aggregating many small insights into one large conclusion.

The Map-Reduce Summarizer recipe (Recipe 13) implements this pattern in SPL. It allows you to summarize documents of any length—from a short article to a full-length book—on commodity hardware with limited memory.

8.1.2 The SPL Approach

This recipe uses a WHILE loop for the Map phase and a final GENERATE step for the Reduce phase, with an added EVALUATE step for quality control.

8.1.3 The .spl File (Annotated)

```
-- Recipe 13: Map-Reduce Summarizer
-- Splits a document, summarizes chunks, then combines them.

WORKFLOW map_reduce_summarizer
    INPUT: @document TEXT, @style TEXT
DO
    @chunk_index := 0
    @summaries := ''

    -- Phase 1: Planning (The Map Strategy)
    GENERATE chunk_plan(@document) INTO @chunk_count -- (1) Determine the split

    -- Phase 2: MAP (Independent Summarization)
    WHILE @chunk_index < @chunk_count DO
        GENERATE extract_chunk(@document, @chunk_index) INTO @chunk
        GENERATE summarize_chunk(@chunk) INTO @chunk_summary -- (2) The Map Step

        @summaries := @summaries + '\n[Chunk ' + @chunk_index + ']: ' + @chunk_summ
        @chunk_index := @chunk_index + 1
    END

    -- Phase 3: REDUCE (Synthesis)
    GENERATE reduce_summaries(@summaries, @style) INTO @final_summary -- (3) The R

    -- Phase 4: Quality Check
    GENERATE quality_score(@final_summary, @document) INTO @score

    EVALUATE @score                                -- (4) Conditional Refinement
        WHEN > 0.7 THEN
            COMMIT @final_summary WITH status = 'complete'
        OTHERWISE
            GENERATE improve_summary(@final_summary, @summaries) INTO @final_summar
            COMMIT @final_summary WITH status = 'refined'
    END
END
```

8.1.3.1 (1) Phase 1: The Map Strategy

We don't hardcode the chunk size. Instead, we ask a "Planner" model to look at the document and decide how many chunks are needed based on the document's structure (e.g., splitting at section headers).

8.1.3.2 (2) Phase 2: The Map Step

Inside the WHILE loop, we extract one chunk at a time and summarize it. - Each summarize_chunk call is independent. It doesn't know about the other chunks. - This keeps the context window small and the model focused on the specific details of that section.

8.1.3.3 (3) Phase 3: The Reduce Step

Once we have a collection of small summaries, we pass them *all* to the final reduce_summaries step. - This step acts as the "Editor." Its job is to find the common themes and write a cohesive report in the requested @style (e.g., bullet points or executive brief). - Because the summaries are much shorter than the original document, they easily fit into a single context window.

8.1.3.4 (4) Phase 4: Conditional Refinement

We don't just take the first draft. We use a quality_score step to check if the final summary accurately reflects the original document. If the score is low, we trigger a refinement pass. This ensures that the "Reduce" phase didn't accidentally drop a critical piece of information from one of the "Map" chunks.

8.1.4 Running It

Run the summarizer on a large text file:

```
spl run cookbook/13_map_reduce/map_reduce.spl --adapter ollama \
    document="$(cat large_report.txt)" \
    style="executive brief"
```

8.1.5 What Just Happened

LLM calls: 1 (plan) + N (map) + 1 (reduce) + 1 (score) + 0 or 1 (refine).

The "Conductor" (SPL Runtime) managed a classic data pipeline: 1. **Decomposed** a large dataset (the document) into manageable partitions. 2. **Processed** each partition in parallel (or sequence) to extract key features. 3. **Aggregated** the features into a final result. 4. **Validated** the result against the source data.

8.1.6 Reproducibility Note

The quality of a Map-Reduce summary depends heavily on the **Chunking Strategy**. If a chunk is cut off mid-sentence or mid-paragraph, the model might lose context.

On a **GTX 1080 Ti**, a 5-chunk document will take about **30–60 seconds** to summarize. The total time scales linearly with the number of chunks, making this pattern highly predictable for large-scale document processing.

8.1.7 When to Use This Pattern

Use the **Map-Reduce Summarizer** pattern when: - **Large Documents**: Anything longer than 5,000 words (depending on your model's context window). - **Batch Processing**: When you need to summarize hundreds of files and want a consistent structure for each. - **Hierarchical Summarization**: For extremely large texts (like a book), you can even do "Nested Map-Reduce" (summarize chapters → summarize parts → summarize book).

8.1.8 Exercises

1. **Change the Chunking.** Modify the chunk_plan to always split by exactly 1,000 words instead of looking for section headers. Compare the quality of the final summary.
2. **Add a "Key Takeaways" section.** Update the reduce_summaries step to always include a "Top 3 Action Items" section at the end of the summary.
3. **Parallel Map.** (Advanced) If you have multiple GPUs or are using a distributed runtime like **Momagrid**, research how to use WITH (Chapter 9.1) to run all the summarize_chunk calls in parallel instead of in a WHILE loop.

Chapter 8.2

Code Review

"A second pair of eyes is good; a second, third, and fourth pair of specialized eyes is better."

8.2.1 The Pattern

In professional software development, code review is the primary defense against bugs, security vulnerabilities, and technical debt. But manual review is slow, subjective, and prone to human fatigue. A reviewer might catch a logic bug but miss a subtle security flaw, or focus on code style while overlooking a performance bottleneck.

Automated Multi-Pass Review solves this by breaking the review into specialized "audits." Instead of one generic "review this code" prompt, you run the code through several distinct passes: 1. **Security Audit**: Look specifically for injections, leaked secrets, and unsafe functions. 2. **Performance Check**: Identify inefficient loops, redundant allocations, and slow I/O. 3. **Style & Best Practices**: Verify adherence to language-specific conventions (e.g., PEP 8 for Python). 4. **Bug Detection**: Search for logic errors, edge cases, and unhandled exceptions.

The SQL analogy is **Multi-Constraint Validation** or **Static Analysis Rules**. You don't just check if the data "looks okay"; you run it against a suite of specific rules (foreign keys, check constraints, unique indexes) to ensure total integrity.

The Code Review recipe (Recipe 15) implements this multi-pass strategy. It scores each category for "Severity" and uses those scores to decide whether to approve, request_changes, or block the code—bringing the discipline of a senior engineer to an automated pipeline.

8.2.2 The SPL Approach

This recipe demonstrates **Pass-by-Pass Analysis** and **Threshold-Based Branching** using numeric evaluation.

8.2.3 The .spl File (Annotated)

```
-- Recipe 15: Automated Code Review
-- Multi-pass code review: security, performance, style, then synthesis.

WORKFLOW code_review
    INPUT: @code TEXT, @language TEXT
DO
    -- Pass 1-4: Specialized Audits
    GENERATE security_audit(@code, @language) INTO @security_findings -- (1) Speci
    GENERATE performance_review(@code, @language) INTO @perf_findings
    GENERATE style_review(@code, @language) INTO @style_findings
    GENERATE bug_detection(@code, @language) INTO @bug_findings

    -- Severity scoring (Turning prose into numbers)
    GENERATE severity_score(@security_findings) INTO @sec_score
-- (2) Quantifying Risk
    GENERATE severity_score(@perf_findings) INTO @perf_score
    GENERATE severity_score(@bug_findings) INTO @bug_score

    -- Synthesize all findings
    GENERATE synthesize_review(...) INTO @review

    -- Phase 5: Automated Verdict
    EVALUATE @sec_score
-- (3) Threshold Branching
        WHEN > 8 THEN
            COMMIT @review WITH status = 'critical_issues', verdict = 'block'
        WHEN > 5 THEN
            COMMIT @review WITH status = 'needs_fixes', verdict = 'request_changes'
        OTHERWISE
            COMMIT @review WITH status = 'approved', verdict = 'approve'
    END
END
```

8.2.3.1 (1) Specialized Passes

Each GENERATE call uses a focused prompt. By asking the model to *only* look for security issues, we reduce the noise from style or performance concerns. This results in much higher "Recall" (finding actual bugs) than a single-pass review.

8.2.3.2 (2) Quantifying Risk (`severity_score`)

LLMs are great at prose, but systems are better at numbers. We use a middle step to "quantify" the findings. We ask the model: "On a scale of 1–10, how severe are these security findings?" This turns a subjective text block into a objective metric that the WORKFLOW can act upon.

8.2.3.3 (3) Threshold Branching

This is where the automation takes action. We use the @sec_score to drive the EVALUATE block. - If a critical security flaw is found (score > 8), we block the code immediately. - If issues are moderate, we request_changes. - Only if the score is low do we approve.

SQL Analogy: A **Trigger with a Threshold**. If a transaction amount exceeds a limit, the trigger blocks the insert; otherwise, it allows it to proceed.

8.2.4 Running It

Review a Python file with a known security flaw:

```
spl run cookbook/15_code_review/code_review.spl --adapter ollama \
    code="def run_cmd(user_input): eval(user_input)" \
    language="Python"
```

Expected result: The security_audit will flag the eval() call, the @sec_score will be high (likely 9 or 10), and the final verdict will be block.

8.2.5 What Just Happened

LLM calls: 8. (4 audits + 3 scores + 1 synthesis)

The "Conductor" (SPL Runtime) acted as a Senior Reviewer: 1. **Decomposed** the review into four logical dimensions. 2. **Analyzed** the code from four different perspectives. 3. **Measured** the risk in each dimension. 4. **Synthesized** a final report. 5. **Decided** on a binding verdict based on the measured risk.

8.2.6 Reproducibility Note

The scoring step (severity_score) is the most sensitive part of this recipe. Different models have different "scales" of what they consider severe.

On a **GTX 1080 Ti**, a full 8-call review takes **60–120 seconds**. This is slower than a human scan but much faster (and more thorough) than a human deep-dive review.

8.2.7 When to Use This Pattern

Use the **Code Review** pattern when: - **Pre-Commit Hooks**: Automatically block unsafe code from being committed to your repository. - **CI/CD Quality Gates**: Add a "Generative Linting" step to your build pipeline. - **Learning Tool**: Use it to get feedback on your own code as you learn a new language.

8.2.8 Exercises

1. **Add a "Fixer" step.** If the verdict is request_changes, add a step that tries to generate a "Fixed" version of the code and includes it in the final review.
2. **Language-Specific Rules.** Modify the security_audit prompt to look for specific flaws in specific languages (e.g., "Buffer overflows" in C++, "SQL Injection" in PHP).
3. **Parallel Audits.** (Advanced) Use the WITH clause (Chapter 9.1) to run all four audit passes in parallel, significantly reducing the total execution time.

Chapter 8.3

Text2SPL Demo

"The final barrier to programming is syntax; we are removing it."

8.3.1 The Pattern

For many, the biggest obstacle to using SPL 2.0 is the same obstacle they face with SQL: learning the keywords, the structure, and the punctuation. While SPL is designed to be readable, it still requires you to follow a "grammar." But what if you didn't have to? What if you could describe what you wanted in plain English and the system wrote the code for you?

This is the vision of **Text2SPL**. It is a "Compiler for Intent." Instead of writing a `.spl` file manually, you provide a natural language description—like "Summarize this document and then translate it into Chinese"—and the spl runtime generates the equivalent code.

The SQL analogy is **Natural Language to SQL (NL2SQL)**. Modern data platforms allow analysts to ask "What was the revenue in Q3?" and the system automatically generates the SELECT ... FROM ... WHERE ... query. Text2SPL brings this same "Zero-Barrier" experience to AI orchestration.

The Text2SPL Demo recipe (Recipe 22) showcases this capability. It demonstrates how to compile simple prompts, multi-step workflows, and even "Auto-Mode" agents from plain text.

8.3.2 The SPL Approach

This chapter doesn't focus on a `.spl` file, but rather on the `text2spl` command built into the `spl` binary.

8.3.3 The Compilation Process (Annotated)

```
# Example 1: Compiling a simple PROMPT
spl text2spl "summarize a document with a 2000 token budget" \
    --mode prompt -o summarize.spl
```

When you run this command, the runtime: 1. **Analyzes** your intent (summarization + budget constraint). 2. **Identifies** the required SPL constructs (PROMPT, SELECT, GENERATE). 3. **Synthesizes** a valid .spl file. 4. **Validates** the syntax using the internal parser.

```
# Example 2: Compiling a multi-step WORKFLOW
spl text2spl "build a review agent that drafts, critiques, and refines text" \
    --mode workflow -o review_agent.spl
```

In "Workflow Mode," the compiler goes deeper. It creates a WORKFLOW block, defines multiple GENERATE steps, and sets up the variables to pass context between them. It may even define CREATE FUNCTION blocks for the different personas (the writer and the critic).

8.3.4 Running It

You can try the compiler demo yourself using the provided shell script:

```
bash cookbook/22_text2spl_demo/text2spl_demo.sh
```

This script will generate three different .spl files in the generated/ directory based on increasingly complex descriptions. You can then run those files just like any other recipe:

```
# Execute the code generated from natural language
spl run generated/summarize.spl --adapter ollama text="Your document here..."
```

8.3.5 What Just Happened

LLM calls: 1 (to compile).

The "Conductor" (SPL Runtime) acted as a Software Engineer: 1. **Interpreted** human requirements. 2. **Transpiled** those requirements into a formal orchestration language. 3. **Verified** the structural integrity of the generated code. 4. **Delivered** a runnable asset.

This is the "North Star" of the Momagrid project: making the barrier to AI engineering effectively **Zero**.

8.3.6 Reproducibility Note

The reliability of Text2SPL depends on the **Compiler Model**. For simple prompts, small models like **gemma3** are 95%+ accurate. For complex workflows involving WHILE loops and EXCEPTION handling, we recommend using a "Senior" model like **Claude 3.5 Sonnet** as the compiler.

Once the code is compiled and validated, the **Execution** is perfectly deterministic. The "magic" only happens once at compile time; after that, you have a solid, version-controlled .spl file.

8.3.7 When to Use This Pattern

Use the **Text2SPL** pattern when: - **Rapid Prototyping**: You have an idea for a workflow and want a "First Draft" of the code in seconds. - **Onboarding**: You are new to SPL and want to see how the system would structure a specific task. - **Dynamic Agents**: When your application needs to generate *new* workflows on the fly based on user requests.

8.3.8 Exercises

1. **Describe a "Self-Healer".** Try compiling the description: *"Generate a blog post, check it for facts, and if facts are missing, search the web and rewrite."* Use ——mode auto and see if it generates a WHILE loop.
2. **Fix and Re-run.** Take a generated .spl file that has a minor syntax error. Fix the error manually and run it. This is the "Human-in-the-loop" coding pattern.
3. **Model Showdown.** Use two different compiler models (e.g., phi4 vs gemma3) to compile the same description. Compare the "coding style" of each model.

Chapter 8.4

Structured Output

"The bridge between human language and machine logic is a well-defined schema."

8.4.1 The Pattern

One of the most powerful features of modern LLMs is their ability to perform **Data Extraction**. You can give them a pile of unstructured text—a lease agreement, a medical report, a customer email—and ask them to "Find the people, the amounts, and the dates."

But if you just ask the model for "JSON," you run a high risk of getting different field names every time. One model might return {"price": 100}, another {"cost": 100}, and a third might return the price as a string: {"price": "$100"}. This inconsistency is the "Data Engineering" nightmare of the AI age.

The **Structured Output** pattern solves this by using a **Schema**. You don't just ask for JSON; you provide a formal definition (usually a JSON Schema) and tell the model: "Your response must match this exact structure, with these exact types."

The SQL analogy is a **Table Schema** or a **CHECK Constraint**. You don't just "Insert data" into a database. You define columns, types (INTEGER, TEXT, BOOLEAN), and constraints. The database ensures that every row matches that schema, and SPL ensures that every LLM response matches your schema.

The Structured Output recipe (Recipe 23) demonstrates how to use CREATE FUNCTION to define a complex schema for multi-entity extraction.

8.4.2 The SPL Approach

This recipe uses a **Schema-Driven Prompt** pattern, where the output format is a first-class citizen in the SELECT context.

8.4.3 The .spl File (Annotated)

```
-- Recipe 23: Structured Output
-- Extract typed data from text using JSON schema.

CREATE FUNCTION extract_entity_schema()      -- (1) Reusable Schema Definition
RETURNS JSON AS $$
{
  "type": "object",
  "properties": {
    "people": {
      "type": "array",
      "items": {
        "type": "object",
        "properties": {
          "name":   { "type": "string" },
          "age":    { "type": "integer" },   -- (2) Enforcing numeric types
          "salary": { "type": "number" }
        }
      }
    }
  }
}
$$;

PROMPT extract_entities
SELECT
    system_role('You are a precise data extraction engine.'),
    context.text AS text,
    extract_entity_schema() AS schema        -- (3) Schema context injection
GENERATE structured_extraction(text, schema) -- (4) Constrained generation
```

8.4.3.1 (1) Reusable Schema Definition

We define the JSON Schema inside a CREATE FUNCTION. This is the "Data Blueprint." By separating it from the prompt, we make the schema reusable across multiple different extraction tasks (e.g., from emails, from PDFs, from web scrapes).

8.4.3.2 (2) Enforcing Numeric Types

Notice the "type": "integer" for age. By providing this hint to the model (and the adapter), we ensure that the output is a raw number (e.g., 42) rather than a string ("42"). This means you can immediately use the output in a calculation without an extra int() step in Python.

8.4.3.3 (3) Schema Context Injection

We include the schema in the context row of our SELECT clause. This ensures the model always has the "Rules of the Game" visible while it is thinking.

8.4.3.4 (4) Constrained Generation

The GENERATE step takes the text and the schema. Depending on the adapter (e.g., OpenRouter or OpenAI), SPL uses the provider's "Structured Output" feature to force the model to comply with the schema. If the model tries to return invalid JSON, the runtime (or the provider) will often catch it and retry automatically.

8.4.4 Running It

Run the extractor on a messy string:

```
spl run cookbook/23_structured_output/structured_output.spl --adapter ollama \
    text="John Smith, 42, joined Acme Corp earning $95,000/year"
```

Expected output (clean JSON):

```
{
  "people": [
    { "name": "John Smith", "age": 42, "salary": 95000 }
  ],
  "organizations": [
    { "name": "Acme Corp" }
  ]
}
```

8.4.5 What Just Happened

LLM calls: 1.

The "Conductor" (SPL Runtime) managed the "Data Capture": 1. **Prepared** the input text. 2. **Attached** the formal data blueprint (the schema). 3. **Instructed** the model to operate as a "Transpiler" from prose to JSON. 4. **Verified** (at the adapter level) that the response was valid and typed.

This is how you build a **Production-Ready Data Pipeline**. By the time the data reaches your downstream application, it is already cleaned, typed, and structured.

8.4.6 Reproducibility Note

Structured output is one of the most stable features of modern LLMs. Even on small models like **gemma3**, providing a formal JSON Schema increases reliability significantly. It reduces the model's "Creative Freedom" and forces it into a "Deterministic Format."

On a **GTX 1080 Ti**, structured generation is just as fast as unstructured generation, but it saves you the "Latency of Error" (the time spent dealing with broken JSON or missing fields).

8.4.7 When to Use This Pattern

Use the **Structured Output** pattern when: - **ETL Pipelines**: Extracting data from thousands of documents into a database. - **Form Filling**: Turning a voice transcript into a structured customer record. - **Workflow Inputs**: When the output of one step needs to be precisely parsed as the input for a calculation or a tool call (Chapter 3.2).

8.4.8 Exercises

1. **Add a "Required" Field.** Modify the schema to make the salary field required. Run it on text that *doesn't* have a salary and see how the model behaves (does it guess? does it return null?).
2. **Extract an Invoice.** Update the schema to include invoice_number and date. Run it on the invoice text provided in the cookbook.
3. **Cross-Schema Validation.** Try giving the model two different schemas in the same SELECT and ask it to choose the one that fits the text best.

Chapter 8.5

Data Extraction

"The real world is messy; your data doesn't have to be."

8.5.1 The Pattern

Most data extraction tutorials use "perfect" examples—clean, short sentences where the information is easy to find. In the real world, data is buried in noisy, unstructured text: emails with three different addresses, invoices with cryptic line items, or contracts with hundreds of clauses.

Schema-Constrained Extraction is the professional way to handle this noise. Instead of asking the model to "find the data," you provide a **dynamic schema** that changes based on the *type* of document you are processing.

The SQL analogy is **Dynamic Schema Mapping** or a **Pivot Table**. You are taking a raw "blob" of text and forcing it into a predefined relational structure (rows and columns) that your downstream database can understand.

The Data Extraction recipe (Recipe 27) implements this pattern. It uses a CREATE FUNCTION block to deliver different extraction schemas based on a format parameter (e.g., invoice, contract, or general). This allows a single workflow to handle a wide variety of business documents with high precision.

8.5.2 The SPL Approach

This recipe demonstrates **Conditional Schema Injection**—choosing the data blueprint at run-time based on the input type.

8.5.3 The .spl File (Annotated)

```
-- Recipe 27: Data Extraction
-- Pull structured fields from messy text using dynamic schemas.

CREATE FUNCTION extraction_schema(format TEXT DEFAULT 'general')
RETURNS JSON AS $$
SELECT CASE format                                  -- (1) The Blueprint Library
  WHEN 'invoice' THEN '{
    "type": "object",
    "properties": {
      "invoice_number": {"type": "string"},
      "amount":         {"type": "number"},
      "vendor":         {"type": "string"}
    }
  }'
  WHEN 'contract' THEN '{
    "type": "object",
    "properties": {
      "parties":        {"type": "array", "items": {"type": "string"}},
      "value":          {"type": "number"}
    }
  }'
  ELSE '{ ... general schema ... }'                 -- (2) Fallback schema
END
$$;

PROMPT extract_fields
SELECT
    system_role('You are a data extraction specialist.'),
    context.text AS text,
    extraction_schema(context.format) AS schema -- (3) Injecting the right bluepri
GENERATE extract(text, schema)                      -- (4) Precision extraction
```

8.5.3.1 (1) & (2) The Blueprint Library

Instead of writing three different .spl files for invoices, contracts, and emails, we keep all our "Data Blueprints" in one function. This makes it easy to maintain a consistent extraction standard across your entire organization.

8.5.3.2 (3) Injecting the right blueprint

In the SELECT clause, we pass the user-provided format into the extraction_schema function. - If the user says format="invoice", the LLM sees the invoice schema. - If they say format="contract", it sees the contract schema.

This is the "Just-in-Time" context pattern that keeps the model focused only on the fields that matter for the current document.

8.5.3.3 (4) Precision Extraction

The GENERATE step uses the schema to constrain the output. Because we explicitly defined amount as a "number", the model will strip out currency symbols and commas, returning a value that can be directly inserted into a DECIMAL column in a SQL database.

8.5.4 Running It

Extract data from a noisy payment request:

```
spl run cookbook/27_data_extraction/data_extraction.spl --adapter ollama \
    text="Please process payment of USD 4,250.00 to Riverside Consulting (ref: PO-
    format="invoice"
```

Expected output:

```
{
  "invoice_number": "PO-8821",
  "vendor": "Riverside Consulting",
  "amount": 4250.00,
  "currency": "USD"
}
```

8.5.5 What Just Happened

LLM calls: 1.

The "Conductor" (SPL Runtime) managed the "Factory Line": 1. **Categorized** the work using the format parameter. 2. **Selected** the correct extraction tool (the schema). 3. **Filtered** the noisy input text through that tool. 4. **Delivered** a perfectly structured data object.

8.5.6 Reproducibility Note

The stability of extraction is highly dependent on the **Schema Complexity**. If your schema has 50 fields, small models might miss some. For complex extraction, we recommend using **gemma3 (27B)** or **Claude 3.5 Sonnet**.

On a **GTX 1080 Ti**, a standard invoice extraction takes **2–4 seconds**. The bottleneck is the number of tokens in the input document, not the complexity of the schema itself.

8.5.7 When to Use This Pattern

Use the **Data Extraction** pattern when: - **Legacy Migration**: You have thousands of old text files that need to be moved into a modern database. - **Workflow Automation**: When a human email needs to trigger an automated business process (like paying a bill). - **Audit Compliance**: When you need to ensure that every document processed by your system contains specific required fields.

8.5.8 Exercises

1. **Add a "Sentiment" Field.** Modify the "general" schema to include a tone field (e.g., "urgent," "polite," "angry") so you can prioritize the extracted data.
2. **Handle Multi-Entity.** Modify the "invoice" schema to include an array of line_items, each with a description and a price.
3. **Strict Mode.** (Advanced) Research the "additionalProperties": false flag in JSON Schema and see how it prevents the model from adding extra, unrequested fields to the output.

Chapter 8.6

Support Triage

"Intelligence is only useful when it is grounded in fact."

8.6.1 The Pattern

Customer support is the "front line" of any business. But manually triaging thousands of tickets is exhausting. A human has to read the ticket, find the customer's order in a database, decide how urgent it is, and then draft a polite response. If the human is tired, they might miss an urgent complaint or cite the wrong order status.

Grounded Triage is the pattern of using deterministic tools to "feed" the LLM the facts it needs *before* it starts reasoning. 1. **Extract**: Find order IDs or account numbers in the ticket text. 2. **Lookup**: Fetch the actual record from your database (or a JSON file). 3. **Reason**: Use the model to classify the ticket and score its urgency, using the real record as the "Ground Truth." 4. **Execute**: Either escalate the ticket immediately or draft a response that cites the verified facts.

The SQL analogy is a **LEFT JOIN with a CASE Statement**. You are joining the "Incoming Ticket" (unstructured) with your "Orders Table" (structured) and then using logic to route the result to the right "Department" (table/outcome).

The Support Triage recipe (Recipe 28) implements this full end-to-end pipeline. It shows how SPL can coordinate between Python scripts (for database lookups) and LLMs (for drafting and classification).

8.6.2 The SPL Approach

This recipe introduces the concept of **Grounding Context**—passing verified data into a GENERATE call to prevent hallucinations.

8.6.3 The .spl File (Annotated)

```
-- Recipe 28: Customer Support Triage
-- Classify → route → draft response grounded in real data.

WORKFLOW support_triage
    INPUT: @ticket TEXT, @tone TEXT DEFAULT 'professional'
DO
    -- Phase 1: Fact Finding (Deterministic)
    CALL extract_order_numbers(@ticket) INTO @order_id -- (1) Python Regex Tool
    CALL lookup_order(@order_id) INTO @order_record    -- (2) Python Database Tool

    -- Phase 2: Intelligence (Probabilistic)
    GENERATE classify_ticket(@ticket, @order_record) INTO @class -- (3) Grounded cl
    GENERATE detect_urgency(@ticket) INTO @urgency

    -- Phase 3: Routing (Branching)
    EVALUATE @urgency
        WHEN > 8 THEN                                   -- (4) Critical Escalation
            GENERATE escalation_alert(@ticket, @class) INTO @alert
            COMMIT @alert WITH status = 'escalated'
        OTHERWISE
            -- Phase 4: Response Generation
            GENERATE draft_response(
                @ticket,
                @order_record,                          -- (5) Citing the Ground Tru
                response_tone_guide(@tone)
            ) INTO @draft

            -- Phase 5: Quality Gate
            GENERATE check_quality(@draft) INTO @score
            EVALUATE @score
                WHEN < 6 THEN GENERATE improve(@draft) INTO @draft
            END

            COMMIT @draft WITH status = 'drafted'
    END
END
```

8.6.3.1 (1) & (2) Fact Finding (`CALL`)

We use two Python tools. The first uses a regex to find strings like #ORD−12345. The second takes that ID and looks it up in an orders.json file. - **Why?** Because LLMs are bad at "remembering" order statuses but great at "summarizing" them. We provide the status as a raw fact.

8.6.3.2 (3) Grounded Classification

When we ask the model to classify the ticket (e.g., "Billing" vs "Shipping"), we provide the @order_record. If the customer is complaining about a charge and the record shows "Payment Failed," the model can accurately classify it as a billing issue without guessing.

8.6.3.3 (4) Critical Escalation

Using the EVALUATE block, we build a "Safety Valve." If the urgency is high (e.g., an angry customer threatening to cancel), we bypass the drafting phase and send an immediate alert to a human manager.

8.6.3.4 (5) Citing the Ground Truth

In the draft_response step, the model sees exactly what is in the database. Instead of saying "Your order should arrive soon," it can say "I see your order #ORD-12345 was shipped via DHL on Tuesday and is currently in Memphis." This builds massive trust with the customer.

8.6.4 Running It

Run the triage on a billing complaint:

```
spl run cookbook/28_support_triage/support_triage.spl \
    --adapter ollama --tools cookbook/28_support_triage/tools.py \
    ticket="I was charged twice for order #ORD-12345"
```

Observe the trace. You will see the Python tool fetching the record for ORD-12345 and the model using that record to apologize for the duplicate charge.

8.6.5 What Just Happened

LLM calls: 4–6. Tool calls: 2.

The "Conductor" (SPL Runtime) managed a "Smart Agent": 1. **Audited** the input for structured identifiers. 2. **Queried** the system of record for the facts. 3. **Assessed** the situation (classification + urgency). 4. **Routed** the work based on the assessment. 5. **Drafted** a factually accurate, tone-correct response.

8.6.6 Reproducibility Note

The reliability of this recipe is 100% for the "Fact" part because it uses Python tools. The "Draft" part depends on the model's tone.

On a **GTX 1080 Ti**, the entire triage takes **15–25 seconds**. Compared to a human taking 5–10 minutes to do the same lookups and typing, this is a **20x–40x speedup** for the support team.

8.6.7 When to Use This Pattern

Use the **Support Triage** pattern when: - **High Volume Inbound**: Thousands of emails or chats that need a first-pass response. - **Database-Dependent Tasks**: Any task where the "Right Answer" lives in a database, not in the LLM's brain. - **SLA Management**: Automatically identifying urgent tickets to ensure they are handled within the required time window.

8.6.8 Exercises

1. **Add an "Unknown Order" path.** Modify the workflow so that if @order_record is empty, the model drafts a response asking the customer for their order number.
2. **Custom Tone.** Change the @tone parameter to "extremely formal" and observe how the model rephrases the same order facts.
3. **Proactive Refund.** (Advanced) Add a Python tool issue_refund() and a step that calls it automatically if the classification is "billing" and the urgency is "critical."

Chapter 8.7

Meeting Actions

"Talk is cheap; action items are the currency of progress."

8.7.1 The Pattern

Meetings are where decisions are made, but transcripts are where those decisions go to die. A 30-minute meeting can generate 5,000 words of text. Finding the three actual "tasks" buried in those words is a manual chore that everyone hates.

Structured Meeting Synthesis is the pattern of converting raw human dialogue into a machine-readable schema. 1. **Normalize**: Clean up the transcript (remove "umms," "ahhs," and duplicate lines). 2. **Extract**: Identify speakers and the specific tasks they committed to. 3. **Validate**: Use deterministic logic to ensure every task has an "owner" and a "due date." 4. **Format**: Turn the structured data into a Markdown report or a JSON object for your project management tool (like Jira or Trello).

The SQL analogy is an **Extract-Transform-Load (ETL) Pipeline**. You are extracting raw logs, transforming them into a structured schema, validating them against business rules (e.g., "every task must have an owner"), and loading them into a final report.

The Meeting Actions recipe (Recipe 29) implements this pipeline. it shows how SPL can manage "Long-Form" context by breaking the task into speaker identification, action extraction, and date normalization phases.

8.7.2 The SPL Approach

This recipe introduces the use of **Schema Grounding**—passing a formal JSON Schema into the GENERATE step to ensure the model output is perfectly structured.

8.7.3 The .spl File (Annotated)

```
-- Recipe 29: Meeting Notes → Action Items
-- Transcript in, structured TODO list + owners out.

WORKFLOW meeting_to_actions
    INPUT: @filename TEXT DEFAULT '', @transcript TEXT DEFAULT ''
DO
    -- Phase 1: Pre-processing (Deterministic)
    CALL load_transcript(@filename) INTO @file_content
    CALL extract_speakers(@file_content) INTO @speakers    -- (1) Finding the actor

    -- Phase 2: Synthesis (Probabilistic)
    GENERATE normalize_transcript(@file_content) INTO @clean -- (2) Noise reduction

    GENERATE extract_actions(
        @clean,
        action_item_schema(),                              -- (3) The Schema constra
        @speakers
    ) INTO @structured_json

    -- Phase 3: Post-processing (Deterministic)
    CALL normalize_dates(@structured_json) INTO @iso_json -- (4) Date Fixing
    CALL validate_ownership(@iso_json) INTO @warnings

    -- Phase 4: Delivery
    GENERATE format_as_markdown(@iso_json, @warnings) INTO @output
    COMMIT @output WITH status = 'complete'
END
```

8.7.3.1 (1) Finding the Actors (`extract_speakers`)

We use a Python tool to scan the transcript for patterns like Alice: or Bob:. - **Why?** By identifying the names *first*, we can tell the LLM: "Only assign tasks to people in this list." This prevents the model from hallucinating "owners" who weren't actually in the meeting.

8.7.3.2 (2) Noise Reduction (`normalize_transcript`)

Before we look for actions, we ask the model to clean up the transcript. This step removes stuttering, filler words, and crosstalk. This makes the next step—extraction—much more accurate because the "Signal-to-Noise" ratio is higher.

8.7.3.3 (3) The Schema Constraint (`action_item_schema`)

This is the heart of the recipe. We pass a formal JSON schema into the extraction step. The model is forced to return a list of objects, each with a task, owner, priority, and due_date. This transforms a rambling conversation into a strict data structure.

8.7.3.4 (4) Date Fixing (`normalize_dates`)

Humans say things like "by Friday" or "next Tuesday." Computers need things like 2026−03−27. We use a Python tool to convert these relative dates into absolute ISO-8601 strings. This is another example of the **80/20 Rule**: let the LLM understand the *meaning* of "next Friday," but let Python calculate the *value*.

8.7.4 Running It

Run the action item extractor on a sample sprint planning transcript:

```
spl run cookbook/29_meeting_actions/meeting_actions.spl \
    --adapter ollama --tools cookbook/29_meeting_actions/tools.py \
    filename="sprint_planning.txt" \
    output_format="markdown"
```

Expected output:

```
### Meeting Summary
The team discussed the API migration and identified three blockers...

### Action Items
- [ ] **Task**: Fix login bug | **Owner**: Alice | **Due**: 2026-03-27
- [ ] **Task**: Update docs | **Owner**: Bob | **Due**: 2026-03-30
```

8.7.5 What Just Happened

LLM calls: 3. (Normalize, Extract, Format) **Tool calls: 4.** (Load, Speakers, Dates, Validation)

The "Conductor" (SPL Runtime) managed an "Executive Secretary": 1. **Identified** the participants. 2. **Filtered** the conversation for clarity. 3. **Structured** the commitments into a formal schema. 4. **Enforced** business rules (dates and owners). 5. **Presented** the result in a human-readable format.

8.7.6 Reproducibility Note

The stability of action extraction depends on the **Speaker Consistency**. If the transcript doesn't clearly label who is speaking, the model will have to guess.

On a **GTX 1080 Ti**, a full 10-minute transcript can be processed in **30–50 seconds**. This is a massive time-saver for teams—the "Meeting Minutes" are ready before everyone has even left the room.

8.7.7 When to Use This Pattern

Use the **Meeting Actions** pattern when: - **Agile Teams**: Automating sprint planning or standup summaries. - **Project Managers**: Ensuring that every decision made in a design review is tracked. - **Executive Support**: Creating "Briefing Notes" from long interviews or strategy sessions.

8.7.8 Exercises

1. **Add a "Decisions" section.** Modify the schema and the extraction step to capture not just tasks, but also "Decisions Made" (e.g., "We decided to use PostgreSQL instead of MongoDB").
2. **High-Priority Alert.** Modify the validate_ownership step to return a specific "URGENT" warning if a task is marked priority : high but has no owner.
3. **Export to JSON.** Run the recipe with output_format="json" and see how easy it would be to curl this data directly into your company's TODO app.

Chapter 8.8

Code Gen + Tests

"Code without tests is just a bug waiting to happen."

8.8.1 The Pattern

In the world of AI-assisted development, "generating code" is easy. Generating *correct* code that follows your organization's standards and includes a comprehensive test suite is much harder. If you simply ask for "a function and tests," the model often produces a single, tangled block of text that is difficult to parse or verify.

Test-Driven Generation (TDG) is the pattern of breaking the coding process into distinct, sequential phases: 1. **Implement**: Generate the core logic based on a specification and language-specific conventions. 2. **Review**: Have the model (or a second model) audit the code for common errors or style violations. 3. **Refine**: Fix any issues found during the review. 4. **Test**: Generate unit tests that specifically target the refined implementation. 5. **Verify**: Perform a final static check to ensure the generated code and tests are syntactically valid.

The SQL analogy is a **Stored Procedure with Unit Tests**. You aren't just writing logic; you are building a "Contract." The implementation is the promise, and the tests are the proof that the promise is kept.

The Code Gen + Tests recipe (Recipe 30) implements this robust lifecycle. It allows you to specify a task (e.g., "Binary search") and a language (e.g., "Go"), and it handles all the details of formatting, conventions, and test framework selection automatically.

8.8.2 The SPL Approach

This recipe demonstrates **Contextual Pipelining**—passing the implementation as a variable directly into the test generation step to ensure perfect alignment.

8.8.3 The .spl File (Annotated)

```
-- Recipe 30: Code Generator + Tests
-- Generate logic, then generate tests — in a single workflow.

WORKFLOW code_gen_with_tests
    INPUT: @spec TEXT, @language TEXT DEFAULT 'Python'
DO
    -- Phase 1: Implementation
    GENERATE implement_function(
        @spec,
        @language,
        language_conventions(@language)          -- (1) Injecting Style Guides
    ) INTO @implementation

    -- Phase 2: Self-Review
    GENERATE review_implementation(@implementation) INTO @notes

    EVALUATE @notes
        WHEN contains('issue') THEN
            GENERATE fix_implementation(@implementation, @notes) INTO @implementati
    END

    -- Phase 3: Test Generation
    GENERATE generate_tests(
        @implementation,                         -- (2) Passing code as context
        @spec,
        @language,
        test_framework_guide(@language)          -- (3) Choosing the runner
    ) INTO @tests

    -- Phase 4: Assembly
    GENERATE assemble_output(@implementation, @tests) INTO @final_output
    COMMIT @final_output WITH status = 'complete'
END
```

8.8.3.1 (1) Injecting Style Guides (`language_conventions`)

We don't hardcode "use PEP 8" in the prompt. Instead, we use a CREATE FUNCTION that returns the correct standards based on the @language parameter. - If @language="Go", it injects "Return (value, error) and use table-driven tests." - If @language="Python", it injects "Use type hints and Google-style docstrings."

8.8.3.2 (2) Passing Code as Context

In the generate_tests step, we pass the @implementation variable. - **Why?** This ensures the tests are "Implementation-Aware." The LLM sees the actual function names, argument names, and logic paths it just wrote, allowing it to write tests that actually work.

8.8.3.3 (3) Choosing the Runner (`test_framework_guide`)

Just as we inject style guides, we inject framework instructions. If the user wants Go, we tell the LLM to use the standard testing package. If they want Python, we point them toward pytest. This prevents the model from hallucinating a "generic" test format that doesn't run.

8.8.4 Running It

Generate a Python function for email validation:

```
spl run cookbook/30_code_gen/code_gen.spl --adapter ollama \
    spec="A function that validates an email address using regex" \
    language="Python"
```

Then, try a complex algorithm in Go:

```
spl run cookbook/30_code_gen/code_gen.spl --adapter ollama \
    spec="Binary search over a sorted list, return index or -1" \
    language="Go" \
    test_framework="testing"
```

8.8.5 What Just Happened

LLM calls: 4–6. (Implement, Review, [Fix], Test, Verify, Assemble)

The "Conductor" (SPL Runtime) managed a "Full-Stack Developer": 1. **Interpreted** the technical requirement. 2. **Enforced** the language-specific style guide. 3. **Audited** its own work for bugs. 4. **Covered** the logic with relevant unit tests. 5. **Packaged** the result into a single, cohesive file.

8.8.6 Reproducibility Note

The stability of code generation is very high for standard algorithms but lower for niche libraries.

On a **GTX 1080 Ti**, the entire process takes **40–80 seconds**. This is significantly faster than a human writing both the code and the tests from scratch, and it ensures that the "Boring" part of development—writing boilerplate tests—is never skipped.

8.8.7 When to Use This Pattern

Use the **Code Gen + Tests** pattern when: - **Greenfield Development**: Quickly scaffolding new utility functions. - **Legacy Refactoring**: Porting a function from one language to another while ensuring the logic remains identical (via tests). - **Prototyping**: Exploring different algorithm implementations and seeing how they pass a set of edge cases.

8.8.8 Exercises

1. **Add a "Docstring" Pass.** Add a step between Implementation and Review that specifically asks the model to "Generate a detailed README section for this function."
2. **Add a "Complexity" Check.** (Advanced) Add a Python tool that runs a static analyzer (like radon for Python) on the @implementation. If the cyclomatic complexity is too high, trigger a refactor step.
3. **Switch Frameworks.** Run the recipe for Python but override the test_framework to unittest. Observe how the model changes its test class structure.

Chapter 8.9

Sentiment Pipeline

"Data tells you what happened; sentiment tells you why it matters."

8.9.1 The Pattern

In the age of social media and instant feedback, businesses are drowning in text. Thousands of reviews, support tickets, and tweets arrive every day. A human can read ten reviews and get a "vibe," but they cannot objectively score 1,000 reviews for emotional nuance, statistical trends, and extreme outliers without losing their mind.

Batch Sentiment Orchestration is the pattern of converting a stream of raw text into a structured analytical report. 1. **Ingest**: Load and split a list of items (e.g., from a file or a pipe). 2. **Analyze**: Use a model to classify each item according to a strict schema (Label, Score, Confidence, Emotions). 3. **Aggregate**: Use deterministic code to compute averages, ratios, and distributions. 4. **Synthesize**: Use a "Senior Analyst" model to write a narrative summary of the trends and highlight extreme cases.

The SQL analogy is an **Aggregate Query with a Window Function**. You are processing a "batch" of rows, calculating statistics across the whole set (AVG(score)), and identifying specific rows that stand out (RANK() OVER ...).

The Sentiment Pipeline recipe (Recipe 31) implements this full stack. It demonstrates the **80/20 Rule** perfectly: LLMs do the hard work of "feeling" the text, while Python does the heavy lifting of "calculating" the statistics.

8.9.2 The SPL Approach

This recipe introduces **Batch Schema Enforcement**—forcing the LLM to return an array of structured objects that map 1:1 to your input list.

8.9.3 The .spl File (Annotated)

```
-- Recipe 31: Sentiment Pipeline
-- Batch sentiment over a list, aggregate trend statistics.

WORKFLOW sentiment_pipeline
    INPUT: @filename TEXT DEFAULT '', @domain TEXT DEFAULT 'general'
DO
    -- Phase 1: Preparation (Deterministic)
    CALL load_items(@filename) INTO @file_content
    CALL split_items(@file_content) INTO @item_list -- (1) Python Splitting Tool

    -- Phase 2: Analysis (Probabilistic)
    GENERATE batch_sentiment(
        @item_list,
        sentiment_schema(),                          -- (2) The Extraction Schema
        @domain
    ) INTO @sentiment_results

    -- Phase 3: Statistics (Deterministic)
    CALL compute_stats(@sentiment_results) INTO @stats -- (3) Python Math Tool
    CALL find_extremes(@sentiment_results) INTO @extremes

    -- Phase 4: Narrative Synthesis (Probabilistic)
    GENERATE summarize_trends(@stats, @extremes) INTO @summary

    -- Phase 5: Final Assembly
    GENERATE assemble_report(@sentiment_results, @stats, @summary) INTO @report
    COMMIT @report WITH status = 'complete'
END
```

8.9.3.1 (1) Python Splitting Tool (`split_items`)

We use a Python tool to turn a raw text file (e.g., reviews separated by newlines) into a JSON array. - **Why?** It is much cheaper and more reliable to split strings in Python than to ask an LLM to "parse this file."

8.9.3.2 (2) The Extraction Schema (`sentiment_schema`)

We pass a formal JSON schema that requires the model to provide a score between -1 and 1 and a label from a fixed list (positive, negative, neutral). This ensures the data is ready for the math step.

8.9.3.3 (3) Python Math Tool (`compute_stats`)

Once we have the LLM's scores, we hand them back to Python. - **Why?** LLMs are notorious for hallucinating math. Python, however, can calculate the mean, standard deviation, and sentiment_ratio perfectly. This is the **Human-in-the-loop** (or rather, **Code-in-the-loop**) principle of SPL 2.0.

8.9.4 Running It

Run the pipeline on a file of product reviews:

```
spl run cookbook/31_sentiment_pipeline/sentiment.spl \
    --adapter ollama --tools cookbook/31_sentiment_pipeline/tools.py \
    filename="product_reviews.txt" \
    domain="product_reviews"
```

Expected output: A comprehensive report including a statistical breakdown (e.g., "75% Positive, 1.2x Sentiment Ratio") and a narrative summary highlighting specific complaints or praises.

8.9.5 What Just Happened

LLM calls: 3. (Batch analysis, Trend summary, Final assembly) **Tool calls: 4.** (Load, Split, Stats, Extremes)

The "Conductor" (SPL Runtime) managed a "Data Science Lab": 1. **Cleaned** and partitioned the raw data. 2. **Extracted** high-dimensional features (sentiment and emotion) using the model. 3. **Calculated** low-dimensional statistics using Python. 4. **Translated** those statistics back into a human-readable story.

8.9.6 Reproducibility Note

The accuracy of the statistics is 100% (given the model's scores). The "Vibe" of the sentiment analysis is highly reproducible on larger models like **gemma3**.

On a **GTX 1080 Ti**, a batch of 15 reviews takes **20–40 seconds**. The pipeline is designed to be "Elastic"—if you have more reviews, the GENERATE step simply takes more tokens, but the logic remains identical.

8.9.7 When to Use This Pattern

Use the **Sentiment Pipeline** pattern when: - **Market Research**: Analyzing feedback from multiple sources to see how a brand is perceived. - **Support Operations**: Identifying a sudden "spike" in negative sentiment that might indicate a system outage. - **Product Launches**: Tracking real-time reaction to a new feature across different demographics.

8.9.8 Exercises

1. **Add a "Competitor" filter.** Modify the schema to include a mentions_competitor boolean field. Update the compute_stats tool to calculate what percentage of reviews mention a rival product.
2. **Filter by Confidence.** Add a step after Phase 3 that filters out any analysis where the model's confidence is below 0.5 and asks for a manual review.
3. **Cross-Domain Test.** Run the recipe on the social_media.txt file but set the domain to legal_contracts. Observe how the "Sentiment" becomes much more neutral and analytical.

Chapter 8.10

Progressive Summary

"The length of a summary should be determined by the reader's time, not the document's size."

8.10.1 The Pattern

In a busy professional environment, one size never fits all. A CEO needs a one-sentence "bottom line." A manager needs a one-paragraph "executive brief." A researcher needs a one-page "technical summary." Most summarization tools force you to pick one. If you want all three, you have to run three separate prompts, which often leads to inconsistent or contradictory summaries.

Progressive Summarization is the pattern of building summaries in "Layers." 1. **Sentence Layer**: Capture the single most important idea in 25 words or less. 2. **Paragraph Layer**: Expand the sentence summary into a 3–5 sentence brief, adding key supporting evidence. 3. **Page Layer**: Expand the paragraph summary into a full report, adding context, implications, and conclusions.

The SQL analogy is **Materialized View Hierarchies**. You have your raw data (the document), a high-level aggregate view (the sentence), a mid-level summary view (the paragraph), and a detailed analytical view (the page). Each view is derived from the one above it, ensuring perfect logical consistency across all levels of abstraction.

The Progressive Summary recipe (Recipe 34) implements this "Zoomable" interface. It generates all three layers in a single workflow, ensuring that the paragraph summary supports the sentence, and the page summary supports the paragraph.

8.10.2 The SPL Approach

This recipe introduces **Sequential Expansion**—using the output of a "compressed" step as the seed for a more "detailed" step.

8.10.3 The .spl File (Annotated)

```
-- Recipe 34: Progressive Summarizer
-- Layered summary: sentence → paragraph → page.

CREATE FUNCTION summary_constraints(layer TEXT)
RETURNS TEXT AS $$
SELECT CASE layer
  WHEN 'sentence'  THEN 'One sentence only. Max 25 words.'
  WHEN 'paragraph' THEN '3-5 sentences. Cover main points.'
  WHEN 'page'      THEN '2-3 paragraphs. Include context and implications.'
END
$$;

WORKFLOW progressive_summarizer
    INPUT: @text TEXT, @layers INT DEFAULT 3
DO
    -- Layer 1: The Core
    GENERATE summarize(@text, summary_constraints('sentence')) INTO @s_summary

    -- Layer 2: The Brief (built on Layer 1)
    GENERATE expand(@text, @s_summary, summary_constraints('paragraph')) INTO @p_su

    -- Layer 3: The Report (built on Layer 2)
    EVALUATE @layers
        WHEN >= 3 THEN
            GENERATE expand(@text, @p_summary, summary_constraints('page')) INTO @
    END

    -- Final Assembly
    GENERATE assemble_package(@s_summary, @p_summary, @page_summary) INTO @package
    COMMIT @summary_package WITH status = 'complete'
END
```

8.10.3.1 (1) Layer 1: The Core

We start with the hardest task: extreme compression. By forcing the model to pick *one* idea first, we establish the "Anchor" for the entire workflow.

8.10.3.2 (2) Layer 2: The Brief (`expand`)

Instead of summarizing the raw @text from scratch, we provide the @s_summary as a guide. - **Why?** This ensures the paragraph doesn't wander off into minor details. It forces the model to stay aligned with the "core truth" identified in the first step.

8.10.3.3 (3) Layer 3: The Report

If the user requested three layers, we repeat the expansion. This "Recursive Detail" pattern is much more stable than asking for three different summaries in a single prompt, which often causes

the model to lose track of the length constraints.

SQL Analogy: **Drill-Down**. You are starting at the highest level of aggregation and progressively "joining" more detail from the source text.

8.10.4 Running It

Run the summarizer on a long research paper:

```
spl run cookbook/34_progressive_summary/progressive_summary.spl \
    --adapter ollama -m gemma3 \
    text="$(cat research_paper.txt)" layers=3
```

Expected output: A "Summary Package" containing three distinct sections, clearly labeled, with perfectly consistent information across all three.

8.10.5 What Just Happened

LLM calls: 4. (3 layers + 1 final assembly)

The "Conductor" (SPL Runtime) managed an "Editorial Team": 1. **Distilled** the essence of the document. 2. **Elaborated** the essence into a brief. 3. **Contextualized** the brief into a full summary. 4. **Verified** (at each step) that the new detail supported the previous layer.

8.10.6 Reproducibility Note

The Progressive pattern is highly **Coherent**. Because each step is grounded in the previous one, you avoid the "drift" that often happens in long-context summarization.

On a **GTX 1080 Ti**, a 3-layer summary takes **20–40 seconds**. The sequential nature means it cannot be parallelized, but the quality gain from the "step-by-step expansion" is worth the extra few seconds of latency.

8.10.7 When to Use This Pattern

Use the **Progressive Summary** pattern when: - **Multi-Stakeholder Reporting**: When you need to send different versions of the same information to different people (e.g., Slack, Email, and internal wiki). - **Zoomable UIs**: Building interfaces where a user can click a "Show More" button to see a more detailed summary of a document. - **Context Management**: When you need to summarize a document to fit into a specific token budget for a downstream prompt (e.g., Chapter 6.1).

8.10.8 Exercises

1. **Add an "Audience" Pass.** Modify the summary_constraints to change the tone of all layers based on an @audience parameter (e.g., "explain to a 5-year-old" vs "technical expert").
2. **Custom Layer.** Add a 4th layer called "Twitter" that generates a 280-character summary with 3 relevant hashtags.
3. **Fidelity Check.** Add a step at the end that compares the "Sentence Layer" to the "Page Layer" and flags any contradictions.

Part 9

Benchmarking & Evaluation

Chapter 9.1

Model Showdown

"A conductor is only as good as their understanding of each instrument in the orchestra."

9.1.1 The Pattern

In the world of LLMs, no single model is the best at everything. One model might excel at creative writing, another at logical reasoning, and a third at being fast and efficient. As a practitioner, your job is to choose the right tool for the task. But how do you compare them objectively?

The manual way is tedious: copy-paste the same prompt into three different chat windows, wait for the responses, and try to keep the differences in your head.

The SQL analogy is a **Parallel Join** or a **UNION ALL** across different data sources. You want to fetch the same logical entity (the answer) from three different "tables" (the models) and see them side-by-side.

The Model Showdown recipe automates this comparison. It sends the exact same prompt to three different models simultaneously, collects their answers, and then—in a final "judge" step—uses an LLM to evaluate which response was the most effective. This is the foundation of model-agnostic benchmarking.

9.1.2 The SPL Approach

This recipe introduces the WORKFLOW block, which is the powerhouse of SPL 2.0. Unlike a PROMPT block, a WORKFLOW can have multiple steps, parallel execution branches (WITH clauses), and complex logic.

9.1.3 The .spl File (Annotated)

```
-- Recipe 04: Model Showdown
-- Same prompt to multiple Ollama models — compare responses side-by-side.

CREATE FUNCTION compare_responses(              -- (1) Define a reusable template
    prompt TEXT,
    model_1 TEXT, answer_1 TEXT,
    model_2 TEXT, answer_2 TEXT,
    model_3 TEXT, answer_3 TEXT
) RETURNS TEXT AS $$
You are a neutral evaluator comparing responses from three AI models.
... evaluate quality, strengths, and weaknesses ...
$$;

WORKFLOW model_showdown                         -- (2) Define the entry point
    INPUT:                                      -- (3) Declare parameters
        @prompt  TEXT DEFAULT 'What is the meaning of life?',
        @model_1 TEXT DEFAULT 'gemma3',
        @model_2 TEXT DEFAULT 'phi3',
        @model_3 TEXT DEFAULT 'qwen2.5'
    OUTPUT: @comparison TEXT
DO
    WITH                                        -- (4) Parallel execution branch
        response_1 AS (
            PROMPT ask_model_1
            SELECT @prompt AS prompt
            GENERATE answer(prompt)
            USING MODEL @model_1                -- (5) Dynamic model selection
        ),
        response_2 AS (
            PROMPT ask_model_2
            SELECT @prompt AS prompt
            GENERATE answer(prompt)
            USING MODEL @model_2
        ),
        response_3 AS (
            PROMPT ask_model_3
            SELECT @prompt AS prompt
            GENERATE answer(prompt)
            USING MODEL @model_3
        )
    SELECT                                      -- (6) Collect results
        response_1.answer AS answer_1,
        response_2.answer AS answer_2,
        response_3.answer AS answer_3
    INTO @answer_1, @answer_2, @answer_3

    GENERATE compare_responses(                 -- (7) The "Judge" step
        @prompt,
```

```
        @model_1, @answer_1,
        @model_2, @answer_2,
        @model_3, @answer_3
    ) INTO @comparison

    COMMIT @comparison                          -- (8) Finalize the output
END
```

9.1.3.1 (1) `CREATE FUNCTION ... AS $$ ... $$`

In SPL, functions are often used as **Prompt Templates**. Instead of messy string concatenation, you define a block of text with placeholders (like {prompt} or {answer_1}). When you call the function, SPL injects the data into the template.

The SQL analogy: A **Stored Procedure** or a **User Defined Function (UDF)** that encapsulates logic for reuse.

9.1.3.2 (2) & (3) `WORKFLOW` and `INPUT`

A WORKFLOW is a stateful program. It starts with INPUT parameters that have types and default values. This is where we declare our "instruments" (@model_1, @model_2, @model_3).

9.1.3.3 (4) `WITH ... AS (...)` (Parallel CTEs)

This is a signature feature of SPL 2.0. In SQL, a CTE (WITH) is a temporary result set. In SPL, a CTE is a **parallel execution branch**. The runtime fires all three PROMPT blocks at the same time. If you have enough VRAM or multiple GPUs, they will run truly in parallel.

9.1.3.4 (5) `USING MODEL @model_1`

In previous recipes, we specified the model via the CLI flag −m. Here, we specify it *inside the code* using a variable. This allows the workflow to be dynamic—you can swap models by just changing the input parameters.

9.1.3.5 (6) `SELECT ... INTO ...`

Once the parallel branches finish, we collect their outputs into workflow-level variables (@answer_1, etc.). This is identical to the SELECT ... INTO syntax found in PL/SQL or T-SQL.

9.1.3.6 (7) The "Judge" Step

We call a final GENERATE step using the compare_responses template we defined earlier. This step uses the default model (e.g., your strongest model like gemma3) to act as the neutral evaluator of the other three.

9.1.4 Running It

Run the showdown with the default models:

```
spl run cookbook/04_model_showdown/showdown.spl --adapter ollama \
    prompt="Explain recursion in 3 sentences"
```

Or override the models to compare specific versions:

```
spl run cookbook/04_model_showdown/showdown.spl --adapter ollama \
    prompt="Write a haiku about a GTX 1080 Ti" \
    model_1="llama3.2" model_2="mistral" model_3="qwen2.5"
```

9.1.5 What Just Happened

LLM calls: 4. (3 in parallel + 1 judge step)

The runtime executed the fan-out: 1. **Branch 1**: Sent prompt to Model A. 2. **Branch 2**: Sent prompt to Model B. 3. **Branch 3**: Sent prompt to Model C. 4. **Synchronization**: The runtime waited for all three to return. 5. **Judge Step**: Combined all three answers and sent them to the judge model.

This demonstrates the "Conductor" metaphor perfectly. You are the conductor, the models are the orchestra, and the WORKFLOW is the score that ensures everyone plays their part at the right time.

9.1.6 Reproducibility Note

When comparing models, **Latency** becomes a key metric. On a **GTX 1080 Ti**, running three models in parallel might slow down generation if they don't all fit in VRAM at once. Ollama handles the swapping, but you will see a performance hit.

For the most accurate "Model Showdown," run on a system with enough VRAM to hold all models, or use a distributed runtime like **Momagrid** where each model can live on a separate node.

9.1.7 When to Use This Pattern

Use the **Model Showdown** pattern when: - **Model Selection**: You are trying to decide which model is best for a specific production task. - **Regression Testing**: You want to see if a newer version of a model (e.g., Llama 3.1 vs 3.2) actually improves your specific use case. - **Ensemble Judgment**: You want to avoid the bias of a single model by getting a "second and third opinion."

9.1.8 Exercises

1. **Add Latency Tracking.** (Advanced) Research the context.latency property and see if you can include the time it took for each model to respond in the final comparison.

2. **Change the Judge.** Use the USING MODEL clause on the final GENERATE step to specify a different model for the judge (e.g., USING MODEL 'gpt−4o' if using the OpenRouter adapter).
3. **Variable Prompting.** Modify the recipe so that each model receives a slightly different prompt (e.g., Model 1 gets a "short" instruction, Model 2 gets a "detailed" instruction).

Chapter 9.2

Batch Test

"Don't just run it once; run it until it breaks, then run it again."

9.2.1 The Pattern

When building AI workflows, the biggest fear is **Regression**. You tweak a prompt to fix a bug in French, but you accidentally break the logic in English. Or you upgrade your model from Version 1 to Version 2, only to find that your structured output is now failing 10% of the time.

The manual way to test is impossible: you have dozens of recipes and multiple models. You cannot manually run and inspect every combination every time you make a change.

The SQL analogy is an **Automated Test Suite** or a **Batch Update**. You want to apply a set of "queries" (your recipes) against a set of "databases" (your models) and verify that the results meet a certain standard.

The Batch Test recipe (Recipe 10) is the "Test Runner" for the SPL cookbook. It fans out multiple recipes across multiple models in parallel, collects the results, and then uses a final "Reporter" model to generate a structured PASS/FAIL report. This is how we ensure the "100% pass rate" claimed in the book's preface.

9.2.2 The SPL Approach

This recipe demonstrates the extreme power of **Parallel Fan-Out** using WITH clauses. It also shows how to use a "Judge" model to automate the analysis of test results.

9.2.3 The .spl File (Annotated)

```
-- Recipe 10: Batch Test
-- Automated testing of key cookbook recipes across multiple models.
```

```
CREATE FUNCTION summarize_results(...) RETURNS TEXT AS $$
You are a test reporter. Produce a concise pass/fail report.
A result is PASS if the output is a non-empty, coherent response.
Format each line as: PASS recipe_name (model)
$$;

WORKFLOW batch_test
    INPUT:
        @model_1 TEXT DEFAULT 'gemma3',
        @model_2 TEXT DEFAULT 'llama3.2'
DO
    WITH                                        -- (1) Extreme Fan-Out
        -- Recipe 01: Hello World
        hello_m1 AS ( ... GENERATE greeting() USING MODEL @model_1 ),
        hello_m2 AS ( ... GENERATE greeting() USING MODEL @model_2 ),

        -- Recipe 02: Ollama Proxy
        proxy_m1 AS ( ... GENERATE answer(prompt) USING MODEL @model_1 ),
        proxy_m2 AS ( ... GENERATE answer(prompt) USING MODEL @model_2 ),

        -- Recipe 03: Multilingual
        multi_m1 AS ( ... GENERATE response(user_input, lang) USING MODEL @model_1
        multi_m2 AS ( ... GENERATE response(user_input, lang) USING MODEL @model_2

    SELECT                                      -- (2) Synchronization Point
        hello_m1.greeting  AS hello_m1,  hello_m2.greeting  AS hello_m2,
        proxy_m1.answer    AS proxy_m1,  proxy_m2.answer    AS proxy_m2,
        multi_m1.response  AS multi_m1,  multi_m2.response  AS multi_m2
    INTO @hello_m1, @hello_m2,
         @proxy_m1, @proxy_m2,
         @multi_m1, @multi_m2

    GENERATE summarize_results(                 -- (3) The Reporter
        @model_1, @model_2,
        '01_hello_world', @hello_m1, @hello_m2,
        '02_ollama_proxy', @proxy_m1, @proxy_m2,
        '03_multilingual', @multi_m1, @multi_m2
    ) INTO @report

    COMMIT @report                              -- (4) Final Commit
END
```

9.2.3.1 (1) Extreme Fan-Out

In Recipe 04 (Model Showdown), we fanned out one prompt to three models. Here, we are fanning out **three different recipes** across **two models each**—a total of six parallel branches.

This is the "Conductor" managing six instruments at once. If your hardware (like a **GTX 1080 Ti**) has the VRAM, these branches will execute simultaneously. If not, the SPL runtime will intelligently queue them.

9.2.3.2 (2) Synchronization Point

The SELECT ... INTO acts as a barrier. The workflow will not proceed to the reporter until *all six* parallel branches have returned their results. This ensures your report is always complete.

9.2.3.3 (3) The Reporter

We pass all six results into the summarize_results template. The judge model (the reporter) doesn't just print the results; it **analyzes** them. It checks for empty strings, error messages, or nonsensical output and applies the PASS/FAIL label.

9.2.3.4 (4) Final Commit

The final @report is committed with metadata. This makes it easy to parse the output in a CI/CD pipeline (e.g., grep "FAIL" output.log).

9.2.4 Running It

Run the batch test with your local models:

```
spl run cookbook/10_batch_test/batch_test.spl --adapter ollama
```

Expected output:

```
=== TEST REPORT ===
PASS  01_hello_world   (gemma3)
PASS  01_hello_world   (llama3.2)
PASS  02_ollama_proxy   (gemma3)
PASS  02_ollama_proxy   (llama3.2)
PASS  03_multilingual   (gemma3)
PASS  03_multilingual   (llama3.2)

Results: 6/6 passed, 0 failed
```

9.2.5 What Just Happened

LLM calls: 7. (6 parallel tests + 1 reporter)

The SPL Runtime orchestrated a complex matrix of execution: 1. Prepared six distinct context packages. 2. Routed them to the appropriate models. 3. Captured the responses into a structured state. 4. Synthesized a comparative analysis.

This is the difference between "using AI" and **engineering AI systems**. By building your own test runner in SPL, you own the quality of your application.

9.2.6 Reproducibility Note

Batch testing is the ultimate test of your local hardware. Running six LLM calls in parallel is demanding. If you see GenerationError exceptions, it might be because your VRAM is saturated.

Pro Tip: For batch testing on commodity hardware, consider setting a max_concurrency parameter in your SPL configuration to limit how many models are loaded at once.

9.2.7 When to Use This Pattern

Use the **Batch Test** pattern when: - **CI/CD Pipelines**: Automatically verify your workflows before every commit. - **Model Evaluation**: When you want to see which model is the most "stable" across a variety of tasks. - **Regression Testing**: After you update a system prompt, run a batch test to ensure you haven't introduced side effects in other parts of the workflow.

9.2.8 Exercises

1. **Add a 4th Recipe.** Modify the workflow to include Recipe 05 (Self-Refine) in the batch test.
2. **Strict PASS/FAIL.** Update the summarize_results prompt to be even stricter—for example, a result is a "FAIL" if it contains more than 50 words.
3. **JSON Reporting.** (Advanced) Research how to change the summarize_results function to return valid JSON so that the test results can be automatically parsed by a dashboard.

Chapter 9.3

A/B Test

"The only thing better than a good prompt is a proven prompt."

9.3.1 The Pattern

In traditional software development, A/B testing is used to compare two versions of a website or feature to see which one performs better with users. In the world of AI, A/B testing is used to compare two different **prompts** (or models) to see which one produces the highest-quality output for a specific task.

Most prompt engineering is done by "vibe check"—you tweak a word, run it once, and if it looks okay, you ship it. This is dangerous. A prompt that looks better for one input might be much worse for another.

Automated A/B Testing is the pattern of running two variants in parallel, scoring them against an objective rubric, and picking the winner based on hard data. 1. **Generate**: Run Variant A and Variant B on the exact same task. 2. **Score**: Use an LLM-as-Judge to score both responses on criteria like clarity, completeness, and engagement. 3. **Compare**: Extract the numeric scores and calculate the margin. 4. **Decide**: If the margin is large enough, pick the winner. If not, declare a tie.

The SQL analogy is a **Champion-Challenger Model** or a **Quality Audit**. You are running two "queries" against the same data and using an aggregate function to decide which one meets the business requirement more efficiently.

The A/B Test recipe (Recipe 26) implements this optimization loop. It can load pre-built experiments from an experiments.json file or accept ad-hoc prompt variants from the command line.

9.3.2 The SPL Approach

This recipe demonstrates **Numeric Evaluation**—using SPL's arithmetic capabilities to compare LLM-generated scores and make a binding decision.

9.3.3 The .spl File (Annotated)

```
-- Recipe 26: Prompt A/B Test
-- Compare two variants, score them, and pick the winner.

WORKFLOW ab_test
    INPUT: @task TEXT, @prompt_a TEXT, @prompt_b TEXT
DO
    -- Phase 1: Execution
    GENERATE run_variant_a(@task, @prompt_a) INTO @response_a -- (1) The "Champion"
    GENERATE run_variant_b(@task, @prompt_b) INTO @response_b -- (2) The "Challenge

    -- Phase 2: Scoring
    GENERATE evaluate(@response_a, scoring_rubric()) INTO @score_a_json
    GENERATE evaluate(@response_b, scoring_rubric()) INTO @score_b_json

    -- Phase 3: Extraction (Deterministic)
    CALL extract_total(@score_a_json) INTO @score_a          -- (3) Prose to Numbers
    CALL extract_total(@score_b_json) INTO @score_b

    -- Phase 4: Decision Logic
    EVALUATE (@score_a - @score_b)                            -- (4) The Margin Check
        WHEN > 1.5 THEN
            COMMIT @response_a WITH winner = 'A', margin = (@score_a - @score_b)
        WHEN < -1.5 THEN
            COMMIT @response_b WITH winner = 'B', margin = (@score_b - @score_a)
        OTHERWISE
            COMMIT 'Tie' WITH winner = 'none'
    END
END
```

9.3.3.1 (1) & (2) The Tournament

We run both versions of the prompt. Variant A is often your existing "Production" prompt (the Champion), and Variant B is the "Experimental" version (the Challenger). By running them in the same workflow, we ensure they are tested under identical conditions.

9.3.3.2 (3) Prose to Numbers (`extract_total`)

The judge model returns a JSON object with several scores (clarity, accuracy, etc.) and a total. We use a Python tool to extract the raw total number. This is critical because SPL's EVALUATE block can perform mathematical comparisons (like > 1.5), which LLMs often struggle to do consistently in prose.

9.3.3.3 (4) The Margin Check

We don't just pick the higher number. We look for a **Significant Margin** (in this case, 1.5 points). - If Variant A is only 0.1 points higher, it's effectively a tie. - If Variant B is 2.0 points higher, it's a clear improvement, and the system declares it the winner.

SQL Analogy: **Threshold Filtering**. You are only acting on data that exceeds a certain "Confidence Interval."

9.3.4 Running It

Run an ad-hoc test comparing two ways to explain neural networks:

```
spl run cookbook/26_ab_test/ab_test.spl --adapter ollama \
    task="Explain neural networks" \
    prompt_a="Explain like I'm 5 years old" \
    prompt_b="Give a technical explanation with analogies"
```

Expected output: The winning response, along with metadata showing the scores for both variants and the margin of victory.

9.3.5 What Just Happened

LLM calls: 4. (2 runs + 2 scores) **Tool calls: 2.** (2 score extractions)

The "Conductor" (SPL Runtime) managed a "Scientific Experiment": 1. **Executed** the control and the variable groups. 2. **Measured** the results against a standard rubric. 3. **Quantified** the performance delta. 4. **Rendered** an objective verdict based on data, not vibes.

9.3.6 Reproducibility Note

Prompt A/B testing is the foundation of **Continuous Improvement**. By running this recipe every time you change a prompt, you ensure that your "improvements" are actually making the system better.

On a **GTX 1080 Ti**, a full A/B test takes **20–40 seconds**. This is a small price to pay for the confidence that your production prompts are optimized.

9.3.7 When to Use This Pattern

Use the **A/B Test** pattern when: - **Prompt Optimization**: You are trying to find the "Perfect Phrasing" for a complex task. - **Model Migration**: Comparing how your existing prompts perform

on a new model (e.g., Llama 3.1 vs Llama 3.2). - **Regression Testing**: Ensuring that a "Safe" version of a prompt hasn't lost too much "Creativity" or "Detail."

9.3.8 Exercises

1. **Multi-Input Test.** Modify the workflow to run the A/B test across three different inputs and pick the variant that has the highest *average* score.
2. **Add a "Cost" Weight.** (Advanced) If Variant A is run on a local model and Variant B is run on a cloud model, modify the decision logic to only pick Variant B if its score is at least 3 points higher (to justify the cost).
3. **Tie-Breaker.** If the margin check results in a "Tie," add a final step that asks a third, even stronger model (the "Super Judge") to break the tie.

Part 10

The Road Ahead

Chapter 10.1

The Recipe Maker: SPL Eating Its Own Cake

"The best proof of a cooking method is a meal cooked with it."

10.1.1 The Concept: What 37 Recipes Taught Us

You have now built 37 recipes. Each one follows the same rhythm.

You identify a task. You find some data. You write a tools.py to load and process that data deterministically. You write a .spl workflow that CALLs the tools for everything code does better, and GENERATEs with the LLM for everything that genuinely requires reasoning. You write a readme.md. You run it, taste it, fix it, run it again.

After doing this 37 times, a pattern emerges that is hard to miss: **every recipe is the same shape**. The data files change. The domain changes. The workflow steps change. But the structure — data catalog, tool layer, workflow, documentation — is invariant.

A SQL practitioner would recognize this immediately. You have discovered a schema. You have been hand-writing the same INSERT statement 37 times. The next move is obvious: write a procedure that generates the INSERT for you.

That procedure is **recipe 00: the recipe-maker**.

The recipe-maker is SPL eating its own cake. It is a workflow that takes a plain-language concept as input and produces a complete, runnable recipe as output — the .spl file, the tools.py, the sample data, and the documentation. It applies everything the cookbook taught: CALL for deterministic operations, GENERATE only where reasoning is required, grounding with real data before asking the LLM to think.

This is not a parlor trick. It is the natural destination of a mature declarative system: **the abstractions become generative**. Once you can describe a pattern precisely enough to teach it to a reader, you can describe it precisely enough to execute it as code.

10.1.2 Cooking as Science and Art

Before the recipe chapter, a word about expectations — because the recipe-maker is frequently misunderstood on first encounter.

Cooking is both a science and an art. The science gives you reliability: the soufflé rises every time, the bread proofs in 90 minutes, the sauce reduces to the right consistency. The art gives you excellence: the seasoning is right, the plating is thoughtful, the meal is worth eating.

A recipe in a cookbook does the science for you. It tells you the ratios, the temperatures, the timing. It does not give you the art. That requires a cook — someone who tastes as they go, adjusts, learns what "just right" feels like in this kitchen with this oven at this altitude.

The recipe-maker is a recipe for writing recipes. It does the science: it generates a structurally complete workflow, a tools file with the right shape, sample data, and documentation. It solves the **bootstrap problem** — the hardest moment in writing any recipe is facing an empty directory. The recipe-maker eliminates the blank page.

What it does not do is give you the finished dish. The generated workflow will need tuning. The tools will need edge-case handling. The sample data will cover the happy path but not the corners. The CALL/GENERATE split will be close but occasionally wrong. This is expected and normal. The recipe-maker compresses the bootstrap problem, not the craft problem.

Think of it as a first draft from a very well-read sous-chef who has studied all 37 recipes and understands the pattern cold, but has never tasted *your* dish before.

10.1.3 The Full Lifecycle

The recipe-maker introduces a lifecycle that goes beyond the single-run model of the earlier chapters. Where every recipe from 01 to 37 is a linear pipeline — input → generate → commit — the recipe-maker exposes a cycle:

```
concept


GENERATE (plan + components)


output_dir (workflow.spl, tools.py, data.json, readme.md, recipe_plan.md)


──feedback: taste the cake ←          human (default) or llm-judge

      fail → iterate → taste again

      pass


         ──reflect: document lessons learned
```

```
—publish: write publication docs + upload to registry
```

Four phases, two gates. The ––reflect and ––publish flags only fire on a passing taste — you do not document or distribute a recipe that has not been validated. This is a deliberate design choice: the registry should contain only recipes that work.

Version 1 of the recipe-maker implements the **generate phase** only. The ––feedback, ––reflect, and ––publish phases are scaffolded in the workflow as commented-out blocks, visible and ready to activate. They are described here because they shape how you work with the generated output even before v2 exists: generate, taste manually, iterate, then publish when it's right.

10.1.4 The CLI Contract

The recipe-maker introduces five new CLI concepts that will become the standard interface for production SPL workflows:

```
spl run cookbook/00_recipe_maker/recipe_maker.spl \
    —adapter ollama –m gemma3 \
    —tools cookbook/00_recipe_maker/tools.py \
    concept="Customer churn predictor with SHAP explainability" \
    output_dir="cookbook/38_churn_predictor/" \
    dataset="data/customers.csv" \
    resources="references/churn_taxonomy.md" \
    feedback=human
```

Each flag has a role:

Flag	Layer	Purpose
––adapter / –m	Generative	Which LLM does the creative work
––tools	Deterministic	Which Python tools handle the code work
dataset=	Data	Local file (JSON/CSV/TXT) or connection string the recipe will operate on
resources=	Knowledge	Reference material, domain docs, related work — grounding for the planner
output_dir=	Artifacts	Where generated files are written
feedback=	Validation	human (default, stops for review) or llm–judge (v2)
concept=	Intent	Plain-language description — what the recipe should do

If you squint, this is a compiler invocation. concept is the source. output_dir is the binary. ––adapter is the compiler backend. ––tools is the standard library. dataset and resources are the headers.

10.1.5 Recipe 00: Recipe Maker

10.1.5.1 The Pattern

You have been writing the same structural shape 37 times. Each time: create a data directory, write a tools.py with a load_X(), a list_X(), and processing tools, write a .spl workflow that CALLs the tools and GENERATEs with the LLM, write a readme.md.

The naive approach is to keep writing this by hand. It is not painful exactly — each recipe is different enough that hand-crafting is satisfying. But it is slow, and the structural parts (file layout, tool scaffolding, readme sections) are not where the creativity lives. They are ceremony.

The recipe-maker is the ceremony automator. Given a concept, it generates the complete recipe skeleton — not perfect, not finished, but structurally sound and immediately runnable. You spend your time on the craft: tuning the GENERATE step names, adding edge cases to the data, adjusting the CALL/GENERATE split. The scaffolding is already there.

10.1.5.2 The SPL Approach

The recipe-maker is itself a recipe: CALL to load the pattern catalog and user-supplied context, GENERATE to plan and create each component, CALL to write all artifacts to disk and notify for review.

It demonstrates the core principle of the entire cookbook in miniature: **every step that does not require reasoning is a CALL; every step that does is a GENERATE**. The file I/O, the pattern lookup, the artifact writing, the review formatting — all CALL. The planning, the workflow generation, the tools generation, the data generation, the documentation — all GENERATE, each grounded by the deterministic context loaded before it.

10.1.5.3 The .spl File (Annotated)

```
-- Recipe 00: Recipe Maker (Meta-Recipe)
-- SPL 2.0 eating its own cake.

-- System context functions injected as LLM system prompts — zero LLM cost.
-- These ground every GENERATE step with precise expectations.

CREATE FUNCTION spl_syntax_guide()          -- teaches the LLM exact SPL syntax
RETURNS TEXT AS $$ ... $$;

CREATE FUNCTION tools_py_guide()             -- teaches the LLM the @spl_tool pattern
RETURNS TEXT AS $$ ... $$;

CREATE FUNCTION recipe_quality_rubric()      -- defines what "complete recipe" means
```

```
RETURNS TEXT AS $$ ... $$;

WORKFLOW recipe_maker
    INPUT:  @concept      TEXT DEFAULT '',        -- the recipe idea, in plain langua
            @output_dir   TEXT DEFAULT 'cookbook/output/',
            @dataset      TEXT DEFAULT '',         -- optional: path to real data
            @resources    TEXT DEFAULT '',         -- optional: reference material
            @feedback     TEXT DEFAULT 'human'     -- validation mode
    OUTPUT: @review_notification TEXT
DO
    -- Phase 1: Load context -- all CALL, zero LLM cost
    CALL load_patterns('')     INTO @patterns_catalog  -- 11 patterns from the cookb
    CALL load_dataset(@dataset)   INTO @dataset_context  -- real data as grounding
    CALL load_resources(@resources) INTO @resources_context

    -- Phase 2: Plan -- LLM understands intent and chooses the right pattern
    -- Grounded by: pattern catalog + quality rubric + user data context
    GENERATE plan_recipe(
        @concept,
        @patterns_catalog,
        @dataset_context,
        @resources_context,
        recipe_quality_rubric()
    ) INTO @recipe_plan

    -- Phase 3: Generate components -- each GENERATE grounded by the plan
    -- SPL syntax guide prevents the LLM from inventing non-existent syntax
    GENERATE generate_spl_workflow(
        @recipe_plan, @concept, spl_syntax_guide()
    ) INTO @spl_content

    -- Tools guide ensures @spl_tool decorator, str parameters, str return types
    GENERATE generate_tools_py(
        @recipe_plan, @concept, @dataset_context, tools_py_guide()
    ) INTO @tools_content

    -- Dataset grounded by user-supplied data if provided
    GENERATE generate_sample_data(
        @recipe_plan, @concept, @dataset_context
    ) INTO @data_content

    -- Readme grounded by the actual generated artifacts, not the plan
    GENERATE generate_readme(
        @recipe_plan, @concept, @spl_content, @tools_content,
        recipe_quality_rubric()
    ) INTO @readme_content

    -- Phase 4: Write artifacts -- all CALL, zero LLM cost
```

```
    CALL write_artifact(@output_dir, 'workflow.spl',     @spl_content)
INTO @_
    CALL write_artifact(@output_dir, 'tools.py',         @tools_content) INTO @_
    CALL write_artifact(@output_dir, 'data.json',        @data_content)
INTO @_
    CALL write_artifact(@output_dir, 'readme.md',        @readme_content) INTO @_
    CALL write_artifact(@output_dir, 'recipe_plan.md',   @recipe_plan)
INTO @_

    -- Phase 5: Summarise and notify -- all CALL, zero LLM cost
    CALL list_artifacts(@output_dir)
INTO @artifact_list
    CALL notify_review(@output_dir, @artifact_list, @feedback)   INTO @review_noti

    COMMIT @review_notification WITH
        status     = 'awaiting_review',
        concept    = @concept,
        output_dir = @output_dir

    --   v2 Placeholders (feedback → reflect → publish)
    -- [v2] GENERATE validate_artifacts(...) INTO @validation
    -- [v2] EVALUATE @validation
    --          WHEN 'pass' THEN
    --              GENERATE reflect(...)    INTO @reflection
    --              GENERATE publish_writeup(...) INTO @writeup
    --              CALL register_recipe(...)
    --          WHEN 'fail' THEN RETRY
    -- [v2] END

EXCEPTION
    WHEN GenerationError THEN
        COMMIT 'Recipe generation failed. Check the concept description and try aga
            WITH status = 'error', concept = @concept
END
```

The SQL analogy: load_patterns() is a lookup join against the pattern catalog. plan_recipe() is the query planner. The four generate_* steps are the column projections. The write_artifact() calls are the INSERT INTO OUTPUT TABLE. notify_review() is the audit log entry.

10.1.5.4 Running It

```
spl run cookbook/00_recipe_maker/recipe_maker.spl \
    --adapter ollama -m gemma3 \
    --tools cookbook/00_recipe_maker/tools.py \
    concept="Interview simulator for data science roles with scoring rubric" \
    output_dir="/tmp/spl_recipes/ds_interview/"
```

Expected output (abbreviated):

```
RECIPE GENERATION COMPLETE

GENERATED ARTIFACTS in /tmp/spl_recipes/ds_interview/

  data.json                                2,841 bytes
  readme.md                                4,203 bytes
  recipe_plan.md                           1,876 bytes
  tools.py                                 5,512 bytes
  workflow.spl                             3,940 bytes

  Total: 5 file(s), 18,372 bytes


NEXT STEPS — Human Review Required:

  1. Review the generated artifacts in:
     /tmp/spl_recipes/ds_interview/

  2. Run the generated workflow:
     spl run /tmp/spl_recipes/ds_interview/workflow.spl
       --adapter ollama -m <model>
       --tools /tmp/spl_recipes/ds_interview/tools.py

  3. Taste the cake — does it produce the expected output?

  4. If satisfied: re-run with --reflect and --publish flags (v2).
     If not: edit the artifacts and re-run with your corrections.

  Status: awaiting_human_review
```

Total LLM calls: **5** (plan + spl + tools + data + readme). Everything else is deterministic Python.

10.1.5.5 What Just Happened

The workflow made exactly 5 LLM calls, in sequence:

1. **plan_recipe** — the most important call. The LLM read the pattern catalog, the quality rubric, and the concept description. It produced a structured plan naming the pattern (multi_persona), the workflow inputs and outputs, the GENERATE steps needed, the tools required, and the shape of the sample data. This plan becomes the grounding context for every subsequent call.

2. **generate_spl_workflow** — the LLM wrote the .spl file, guided by the recipe plan and the spl_syntax_guide() context function that teaches exact SPL syntax. Without the syntax guide, the LLM would invent plausible but invalid clauses.

3. **generate_tools_py** — the LLM wrote the tools file, guided by the tools_py_guide() context that specifies @spl_tool decoration, str parameter types, and str return types. The dataset context (if provided) shaped the data-loading functions.

4. **generate_sample_data** — the LLM generated realistic sample records that match the data schema described in the plan.

5. **generate_readme** — the LLM documented the recipe, receiving the actual generated .spl and tools.py content as input rather than just the plan. This means the readme reflects what was actually generated, not what was intended.

All five calls are grounded: none asks the LLM to reason from scratch. Each receives the accumulated context from the steps before it. This is the chain pattern (recipe 09) applied to the recipe-making process itself.

After the five LLM calls, write_artifact() is called five times (one per file), list_artifacts() produces the bundle summary, and notify_review() formats the human-facing output. Zero additional LLM cost.

10.1.5.6 Reproducibility Note

Latency varies significantly with concept complexity. A simple extraction recipe concept ("Extract structured fields from invoice text") plans and generates in 45–90 seconds on a GTX 1080 Ti with gemma3. A complex multi-persona concept ("Interview simulator with role catalog, candidate profi takes 2–4 minutes — the plan step alone requires reasoning across all 11 patterns and the full quality rubric.

CV% is higher than most earlier recipes because the plan step involves genuine reasoning that varies run-to-run. Two runs of the same concept will produce structurally similar but not identical output. This is expected: the recipe-maker is a creative tool, not a deterministic one. What matters is whether the output is runnable and close to the target, not whether it is identical across runs.

Run with a stronger model (−m mistral or −−adapter claude_cli −m claude−sonnet−4−6) for more reliable plan quality on complex concepts.

10.1.5.7 When to Use This Pattern

The recipe-maker is the right tool when you are facing a blank directory and a concept description. Use it to:

- **Bootstrap a new recipe** from a concept you have not implemented before
- **Recreate an existing recipe** to measure how close the generator gets — this is the calibration test: run the recipe-maker against a concept whose ground truth you know
- **Iterate on an existing recipe** by providing the original .spl and tools.py as resources= — the generator will treat them as reference and produce a closer first draft
- **Teach the pattern to a new team member** — recipe_plan.md explains the LLM's reasoning at each step, making it a readable design document even before the recipe is complete

Do not use the recipe-maker when:

- The recipe is a minor variation of an existing one — editing the original is faster than generating from scratch
- The concept is vague — the quality of the plan is proportional to the specificity of the concept; "something with sentiment" will produce a worse plan than "batch sentiment classifier for customer suppor
- You need a production-ready artifact on the first run — the recipe-maker produces a baseline, not a finished dish

10.1.5.8 Exercises

1. **The calibration run.** Run the recipe-maker with concept="Customer support triage with order lookup and and output_dir pointing to a fresh directory. Compare the generated workflow to 28_support_triage. Count how many lines you would need to change to reach parity. This number is your current recipe-maker quality score.
2. **Dataset grounding.** Re-run the calibration with dataset="cookbook/28_support_triage/orders.json" added. Does the generated data.json more closely match the original? Does the tools.py get the schema right?
3. **Concept refinement.** Take a concept that produced a poor plan in your first run. Improve the concept description — add domain vocabulary, specify the pattern explicitly, name the data schema. Observe how much the plan quality improves. The lesson: the recipe-maker multiplies the quality of your concept, it does not replace it.
4. **Swap the model.** Run the same concept with --adapter ollama -m mistral and then with --adapter claude_cli -m claude-sonnet-4-6. Compare the plans. Stronger models produce better structural reasoning; the pattern catalog and rubric grounding matter more on weaker models.

10.1.6 A Note on What This Means

Thirty-seven recipes in, you have a tool that writes recipes. This is not the end of the cookbook — it is the beginning of a second order of productivity.

The recipes you write by hand will always be better than the recipes the recipe-maker generates on the first pass. You know the domain. You understand the edge cases. You have taste. The recipe-maker does not have taste; it has pattern recognition and grounding.

What the recipe-maker gives you is **the bootstrap**. The blank-page problem vanishes. The structural ceremony is automated. The first draft exists in the time it takes to run five LLM calls. What remains is the craft: reading recipe_plan.md, running the generated workflow, tasting the output, adjusting the seasoning, running again.

This is the conductor's work. The score is written. Now it needs to be played.

Next: Chapter 10.3 — The Road Ahead: Declarative AI, Momagrid, and Where SPL Fits

Chapter 10.2

About the Author

A portrait of the person building Momagrid — physicist, engineer, and advocate for AI as a public utility.

10.2.1 From Anhui to CERN: A Journey of Access

Wen Gong's journey began in a rural village in Anhui, China. In a place where poverty was the baseline and the horizon was often limited to the next field, there was only one path out: education. It was a path he took with a physicist's focus—leading from the University of Science and Technology of China (USTC) to a Ph.D. in nuclear physics in the United States, and eventually to the halls of Lawrence Berkeley National Laboratory and the experimental pits of CERN in Geneva.

This origin story is not mere background; it is the "initial condition" for everything Wen builds today.

Having seen how institutions and access can transform a life, Wen views the current state of AI with a physicist's skepticism and an engineer's pragmatism. Today, the power of AI is locked behind barriers of technical fluency, expensive cloud accounts, and complex programming frameworks. Wen's mission is to dissolve those barriers—not as an act of charity, but as a "debt repayment" to the systems of education and open research that once gave him a chance.

10.2.2 The Physicist's Instinct: Declarative Thinking

Wen's career has spanned the evolution of modern computing—from witnessing the birth of the World Wide Web at CERN to seeing the first distributed computing projects like SETI@home. But it was his training under the legacy of Werner Heisenberg at the Max-Planck-Institut that shaped his deepest technical instinct: **the declarative approach.**

In physics, you cannot always specify the "how"—the unobservable path of a particle. Instead, you formalize the "what"—the observables—and let the underlying mathematics resolve the rest. This is the core philosophy of **SPL 2.0 (Structured Prompt Language)**. Just as SQL allows a data engineer to query millions of rows without knowing the disk's rotation speed, SPL allows an AI

practitioner to orchestrate complex "agentic" workflows without writing low-level Python "glue" code.

10.2.3 The "Gang of Four" and the 48-Hour Sprint

Wen is a practitioner of the **Human × AI Partnership**. He operates with what he calls his "Digital Quadruplets"—a research team consisting of himself and three distinct AI assistants (Claude, Gemini, and Z.ai). Each brings a different "frequency" to the work: - **Claude** for technical execution and precision. - **Gemini** for breadth and recent literature. - **Z.ai** for critical, adversarial review.

This book itself is a product of this collaboration. It was drafted, refined, and validated in an iterative loop that mirrors the "Conductor" metaphor: Wen provides the vision and the "score," while the AI orchestra executes the performance. It is a model of how the next generation of engineers—especially those in the Global South—can scale their impact using commodity hardware like a used GTX 1080 Ti and a declarative language like SPL.

10.2.4 The Vision: AI as a Public Utility

Wen Gong believes that AI computation should be as accessible as the Web. Through **Momagrid** and **SPL 2.0**, he is building a "resilience layer" for AI—a decentralized, open-source infrastructure where a child in a rural village can "conduct their own symphony" of intelligence using a smartphone and a simple, declarative score.

Wen Gong is currently an AI engineer, a former data architect at Oracle and Vanguard, and a lifelong builder committed to making AI a universal tool for human potential.